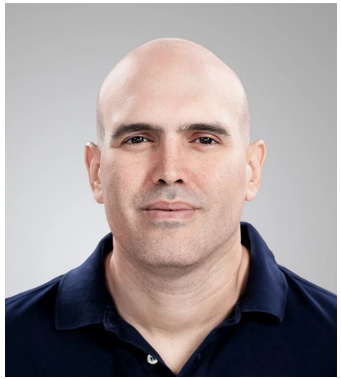
Fundamentals of Deep Learning

Antonio Rueda-Toicen

June 2025

About me

- AI Researcher at Hasso Plattner Institute
- Organizer of the Berlin Computer Vision Group
- Instructor at Nvidia’s Deep Learning Institute and Berlin’s Data Science Retreat



LinkedIn

Main challenges

- Fruit classification challenge on Kaggle (private link)
  - This link is only visible after you have logged in to Kaggle
- Digit recognizer challenge on Kaggle (public link)
  - MNIST exploration with FiftyOne (Google Colab)
  - Fruit Classification with ResNet18 and FiftyOne (Google Colab - evaluation)

Agenda

- Intro to deep learning
  - DL in the machine learning project-cycle
  - What is DL and why should we use it?
    - Neural networks
    - Perceptrons
    - The feedforward pass
    - Activation functions
    - Backpropagation
    - Stochastic gradient descent
    - Convolutional neural networks
    - Transformers
    - Data augmentation
    - Binary vs multiclass classifiers
    - Transfer learning and embeddings
  - Our goal: build an image classifier of fruits in PyTorch

Private Kaggle Competition

1. Register on Kaggle.com
2. Verify your account through SMS to gain access to GPUs (instructions here)
3. Access the competition here:

   https://www.kaggle.com/t/b4dbb9add11c4da0962b837929799d52

   The competition is private and will only be visible after you have registered on Kaggle

4. Starter notebook (download and upload to Kaggle)

Goal: achieve over 92% accuracy

File: test_398.png

File: test_12.png

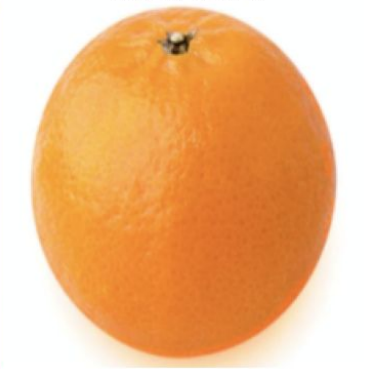
File: test_173.png

File: test_277.png

File: test_116.png

File: test_360.png

File: test_334.png



File: test_102.png

What is “**deep**” learning?

- It’s a fuzzy term for artificial neural networks with “many” hidden layers.

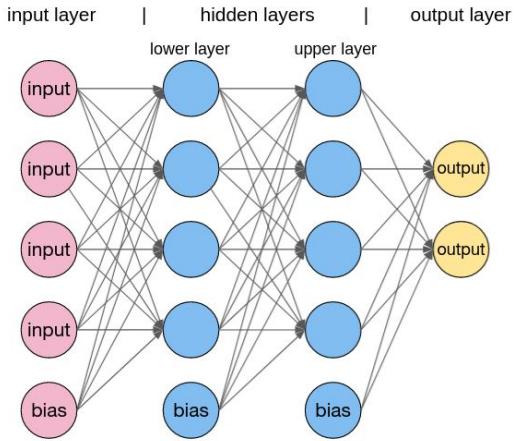


Image source

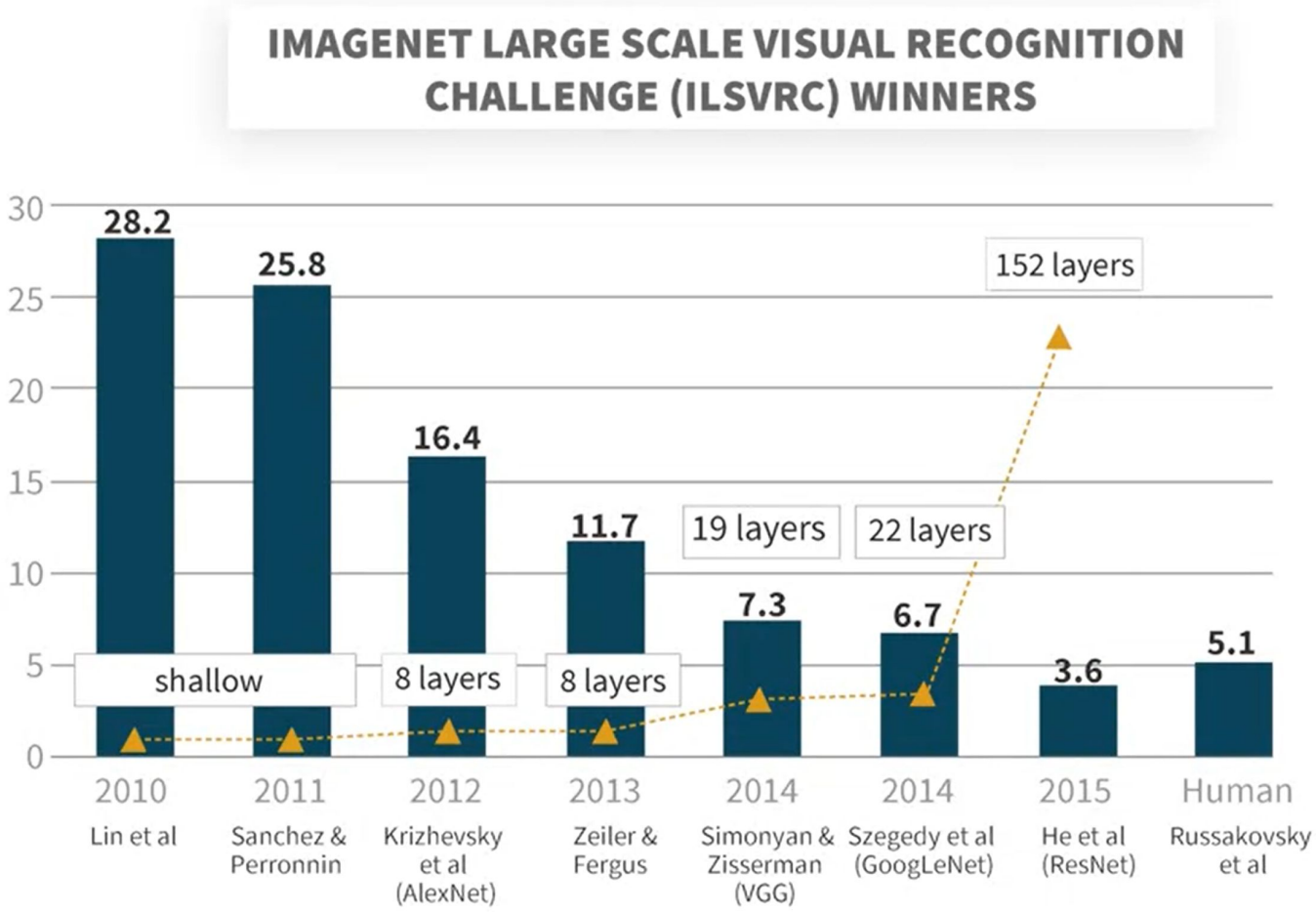
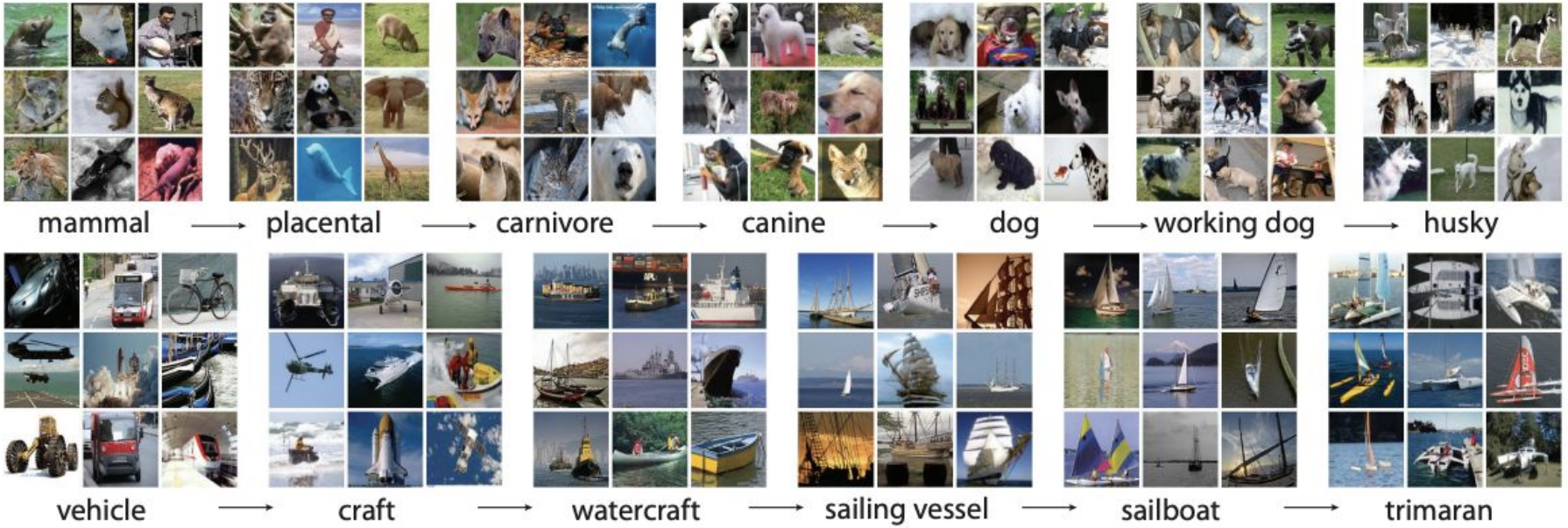


Image source

Y axis = error rate on Imagenet classification



Source: [Introduction to Imagenet]

What is an artificial neural network?

In 1943 Warren McCulloch, a neurophysiologist, and Walter Pitts, a logician, teamed up to develop a mathematical model of an artificial neuron. They declared that:

> *Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms.* (Pitts and McCulloch; A Logical Calculus of the Ideas Immanent in Nervous Activity)

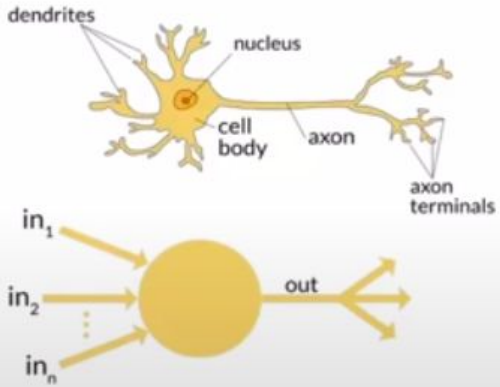
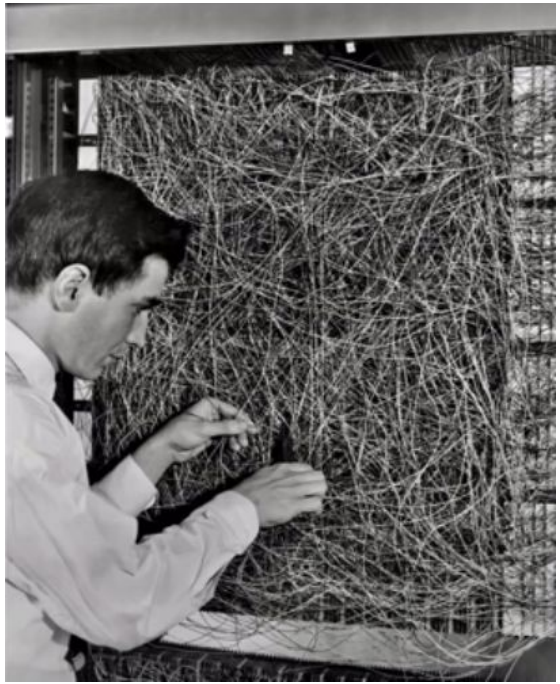


Image and text from 'Deep Learning for Coders' by Jeremy Howard and Sylvain Gugger

Artificial "neurons" are also called "processing units"

Recommended read on McCulloch and Pitts: "The Man Who Tried to Redeem the World with Logic"

What is an artificial neural network?

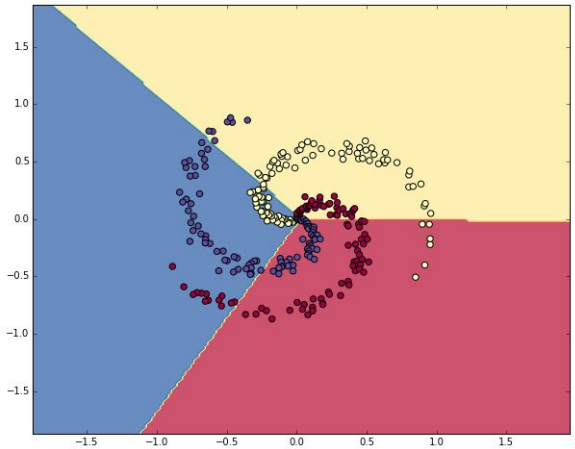


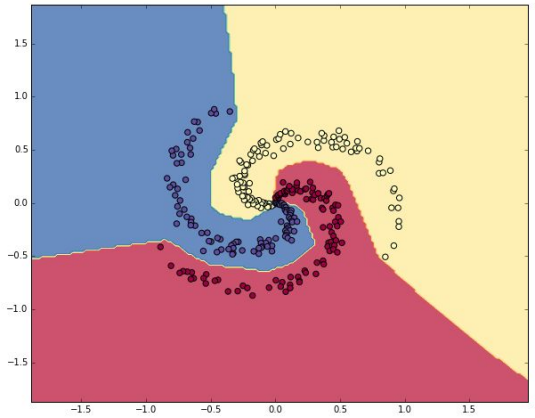
"we are about to witness the birth of such a machine – a machine capable of perceiving, recognizing and identifying its surroundings without any human training or control".
Frank Rosenblatt

Image: Frank Rosenblatt with the Mark I Perceptron at Cornell University (1961) source

Deep learning is the art of making squiggly functions

- Squiggly function = ‘sophisticated decision boundary’
- More weights and non linear activations ~ more squiggly boundaries





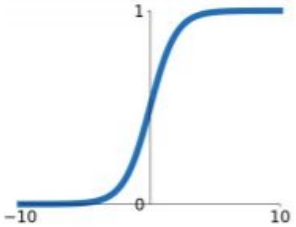
https://cs231n.github.io/neural-networks-case-study
Experiment notebook

Activation functions

ReLU is a good default choice for most problems

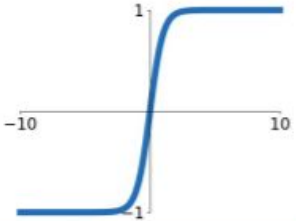
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



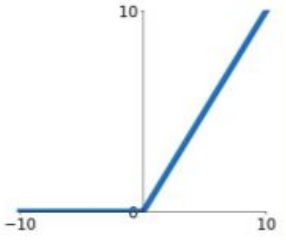
tanh

$$\tanh(x)$$



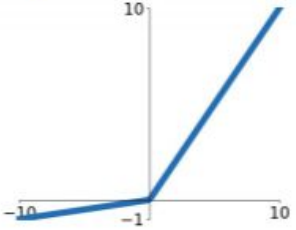
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

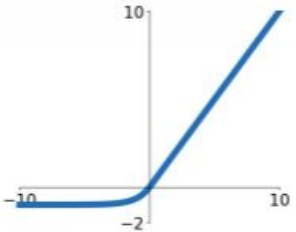


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



A: Using ReLU as a final activation function prevents negative predictions for the age.

Creating predictions

$$\hat{y} = f(X; \theta)$$

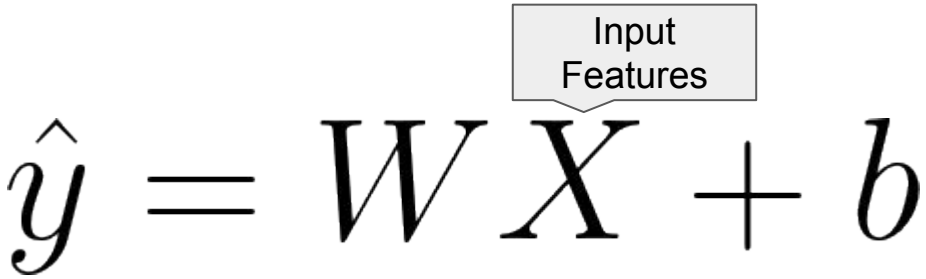
Neural networks are functions that learn parameters **θ** (weights and biases) to make a prediction **ŷ** for given input features **X**

Creating predictions

A simple neural network can be represented by a linear regression model, where the output **ŷ** is defined by the equation **ŷ = WX + b**.

Here, **W** represents the weights, **X** the input features, and **b** the bias term.

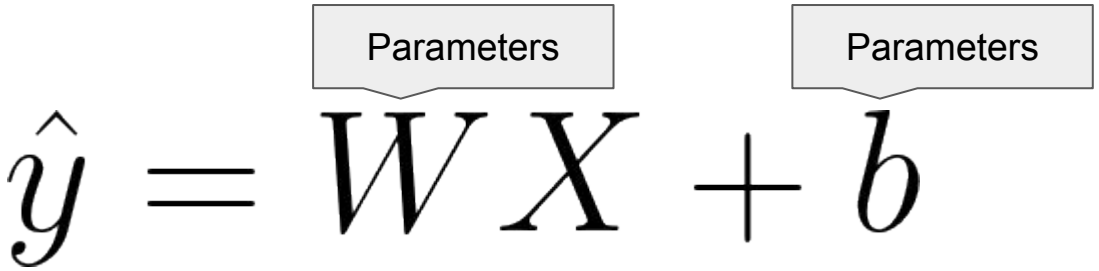
Creating predictions



A simple neural network can be represented by a linear regression model, where the output **ŷ** is defined by the equation **ŷ = WX + b**.

Here, **W** represents the weights, **X** the input features, and **b** the bias term.

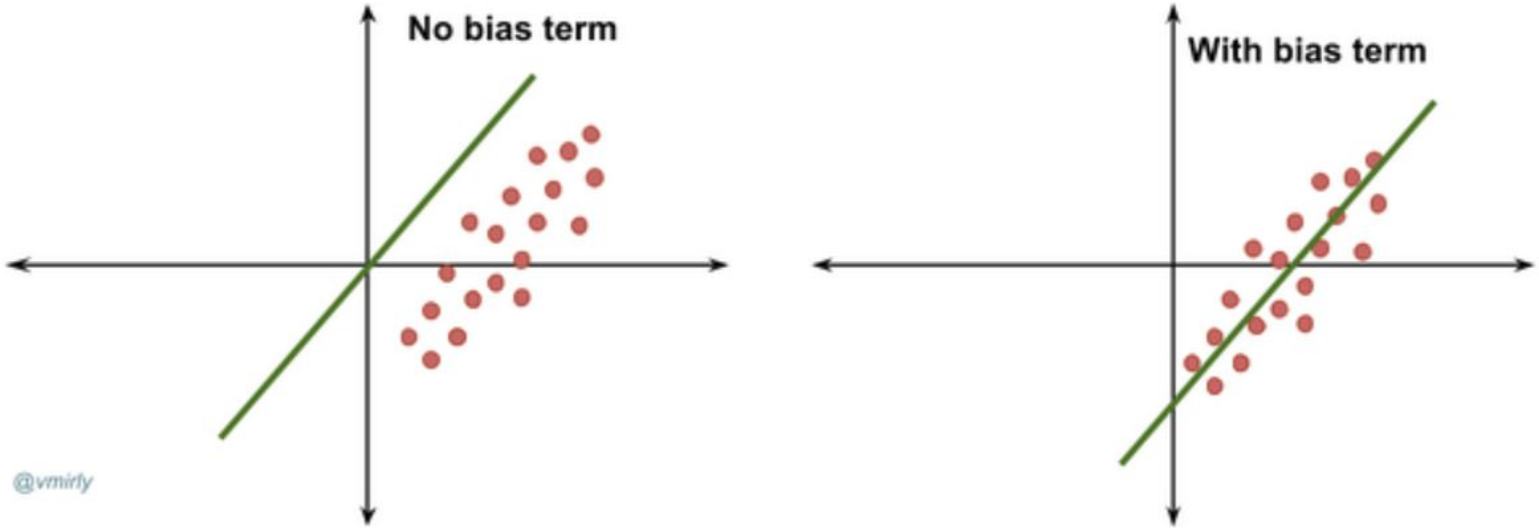
Creating predictions



A simple neural network can be represented by a linear regression model, where the output **ŷ** is defined by the equation **ŷ = WX + b**.

Here, **W** represents the weights, **X** the input features, and **b** the bias term.

Bias units



Source: [Introducing linear regression the easy way]

Fully connected networks

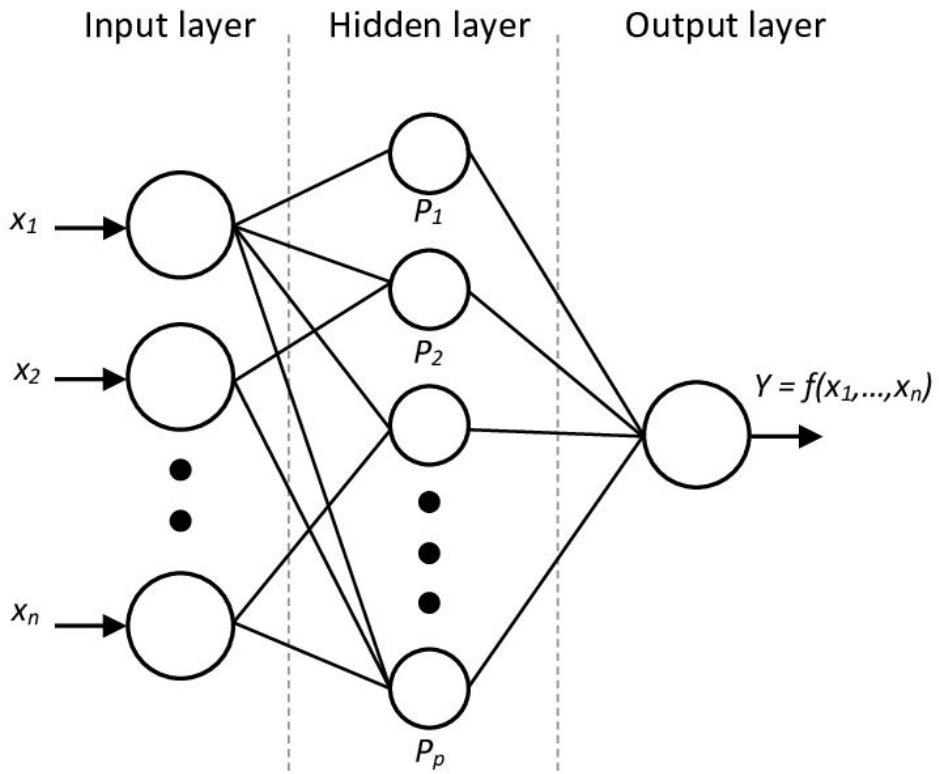
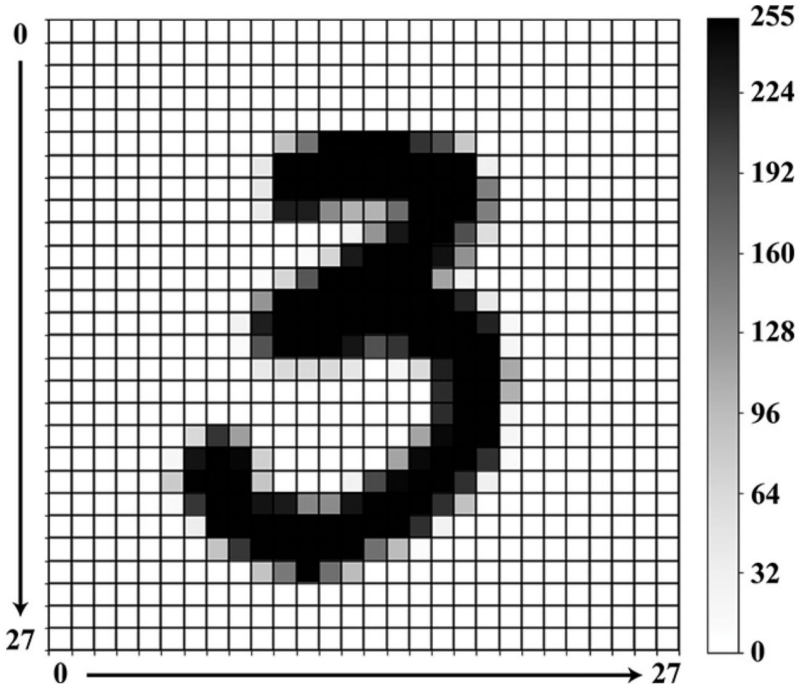


Image representation



what the computer “sees”

Image from Deep Learning Illustrated (O’reilly, 2019)

Images as tensors

Image source

What a human sees

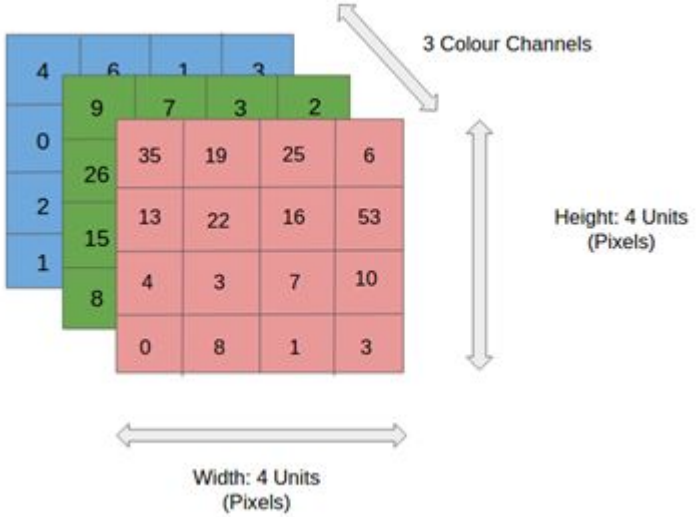
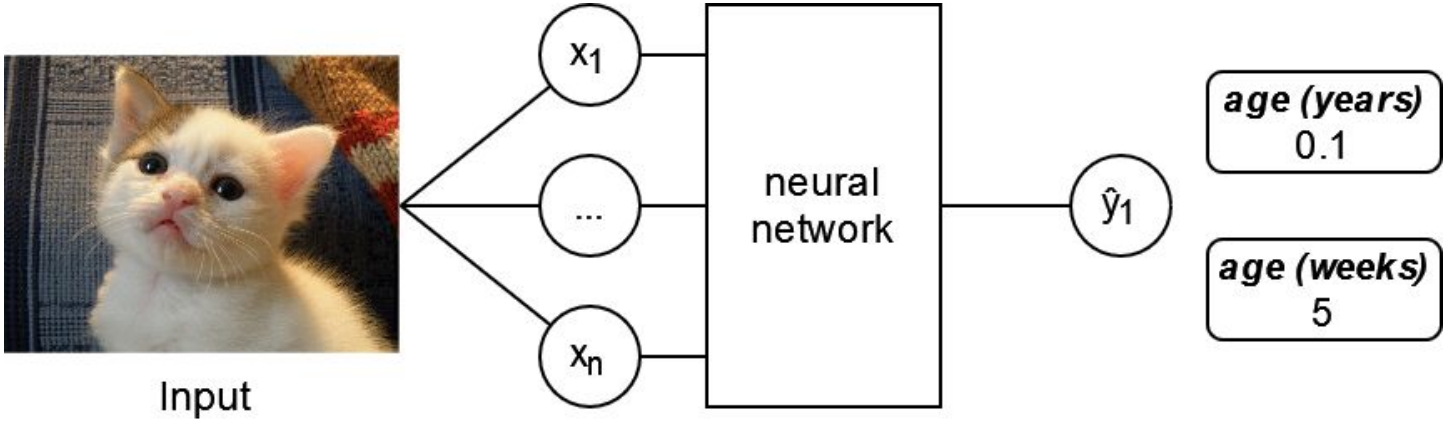


Image source

What the computer ‘sees’

Regression and classification - Intro to NN design



Regression

Predicting the age of a cat with a neural network

In code

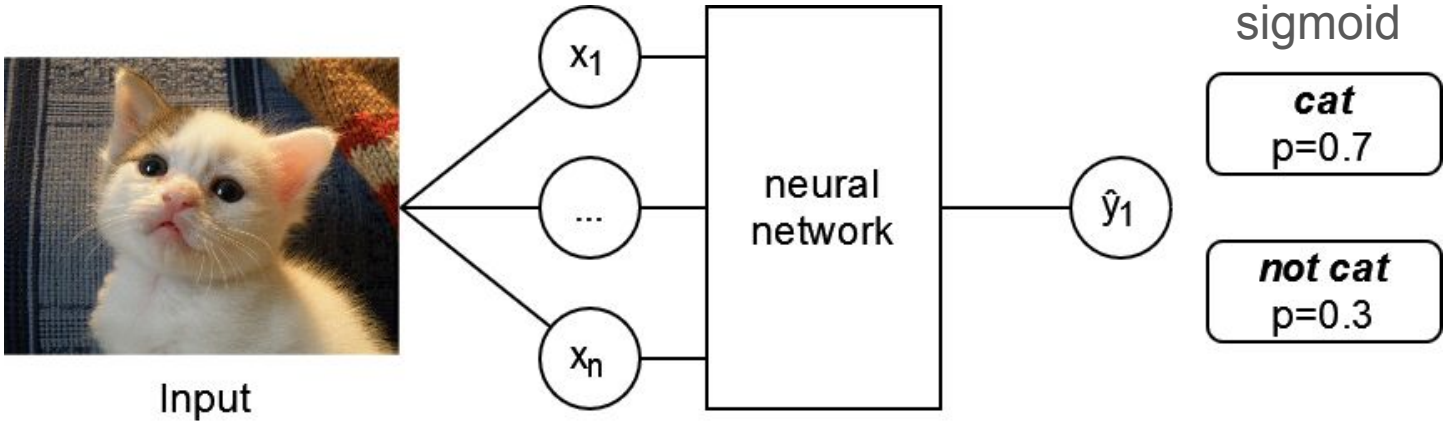
	height	weight
0	27.490803	3.110798
1	39.014286	5.251406
2	34.639877	7.237675
3	31.973169	6.393349
4	23.120373	6.839367

```
AgePredictor(
  (model): Sequential(
    (0): Linear(in_features=2, out_features=10, bias=True)
    (1): ReLU()
    (2): Linear(in_features=10, out_features=1, bias=True)
    (3): ReLU()
  )
)
```

	age
0	14.478074
1	22.054506
2	22.657122
3	21.835995
4	18.858849

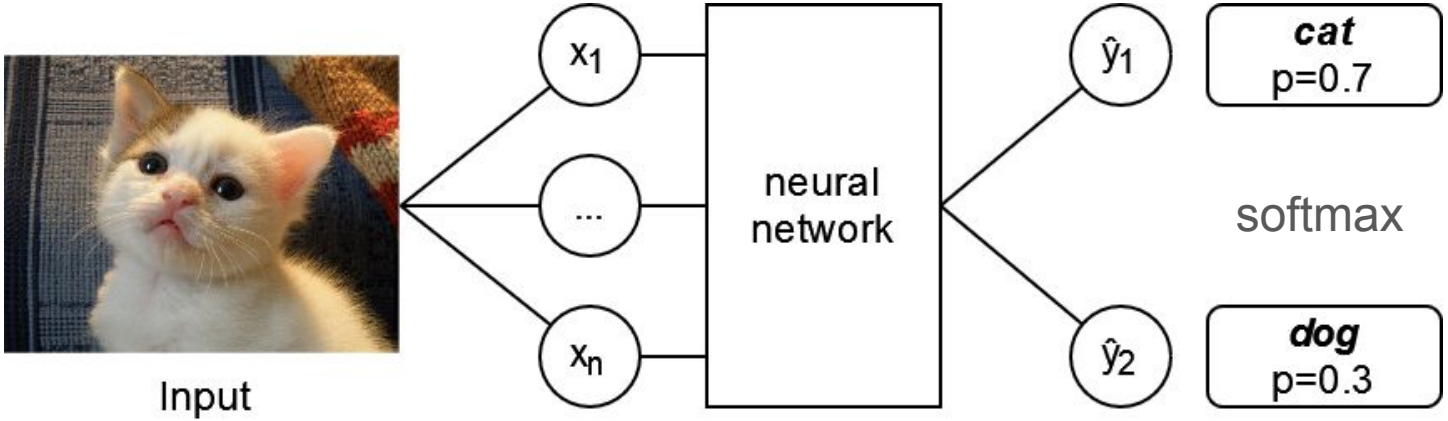
Q: How many units are in the hidden layer of the network?
How many in the input layer? How many in the output layer?
How many units could we place in the hidden layer?

Regression and classification - Intro to NN design



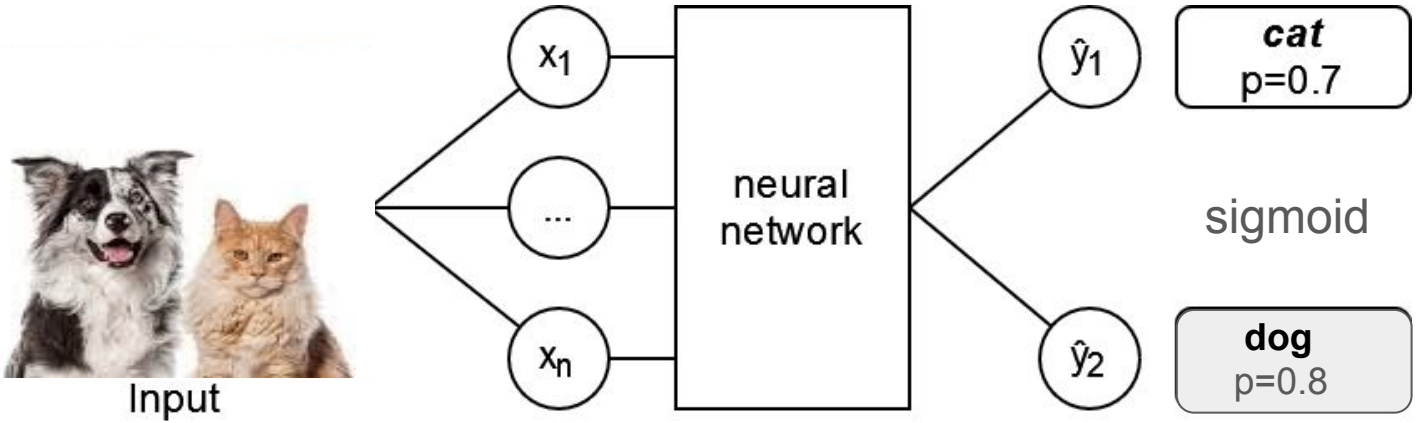
Classification

Regression and classification - Intro to NN design



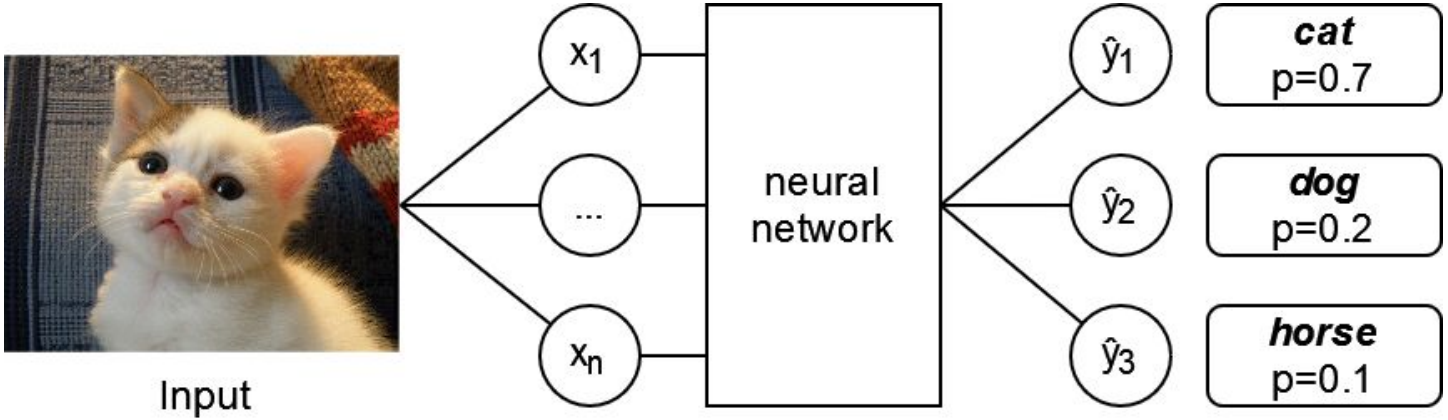
Classification

Regression and classification - Intro to NN design



Classification

Regression and classification - Intro to NN design



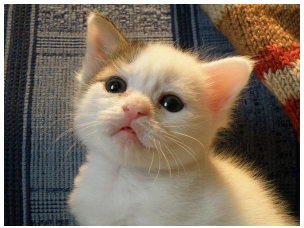
Classification

Cross entropy loss

$y = 1.0$ $\hat{y} = 0.75$

$$H(p,q) = -\frac{1}{n}\sum_{i=1}^{n} p(x_i)\log q(x_i)$$

Labels y_i map to $p(x_i)$, predictions $\hat{y}_i$ map to $q(x_i)$. Suppose that we show the network, only the following image (n=1).



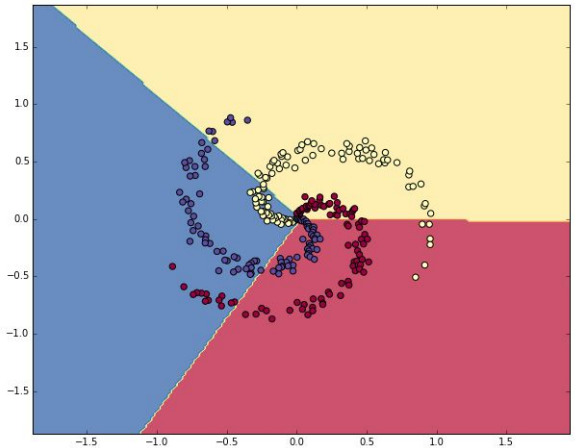
"one hot encoding" 🔥 = only one true label

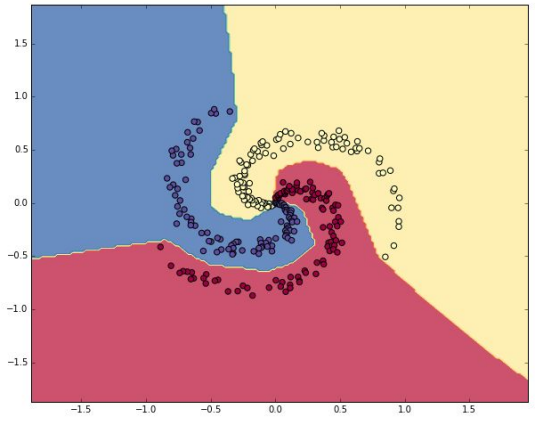
The label for the image is encoded as y = [1, 0, 0] (i = 0 = cat, i = 1 = dog, i = 2 = horse)

- First the network outputs **ŷ = [0.75, 0.25, 0.0]**
- Then the network outputs **ŷ = [0.99, 0.01, 0.0]** **(if trained correctly)**

‘Deep learning’ is the art of making squiggly functions

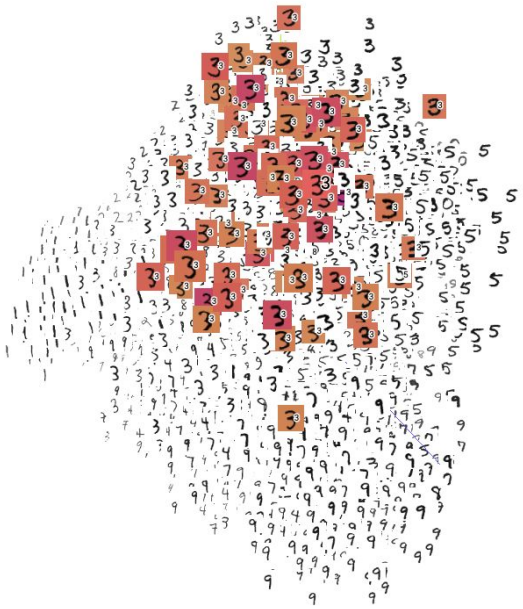
- Squiggly function = ‘sophisticated decision boundary’
- More weights and non linear activations ~ more squiggly boundaries





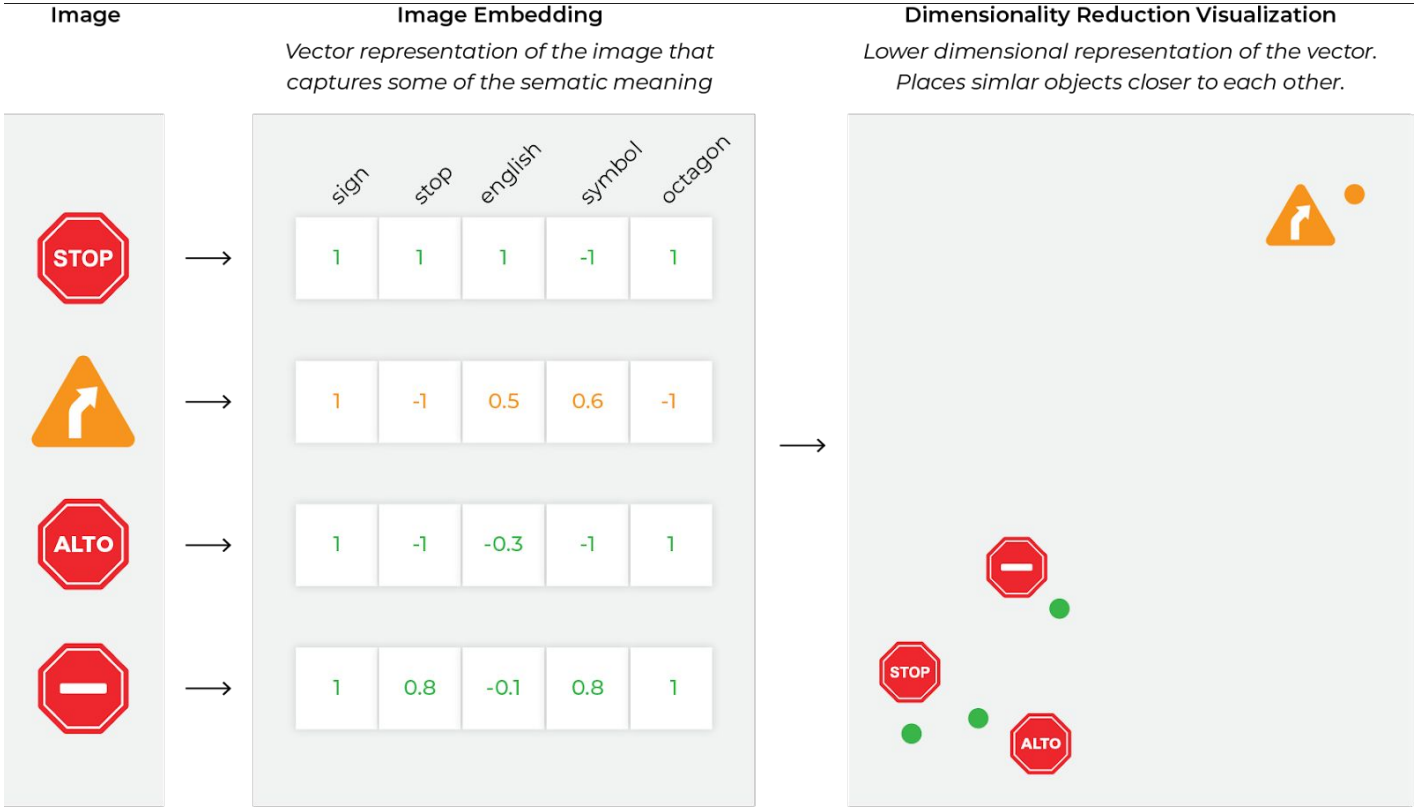
Quick intro: Embeddings

When we train a neural network, we make it **embed** inputs that are semantically similar, close to each other



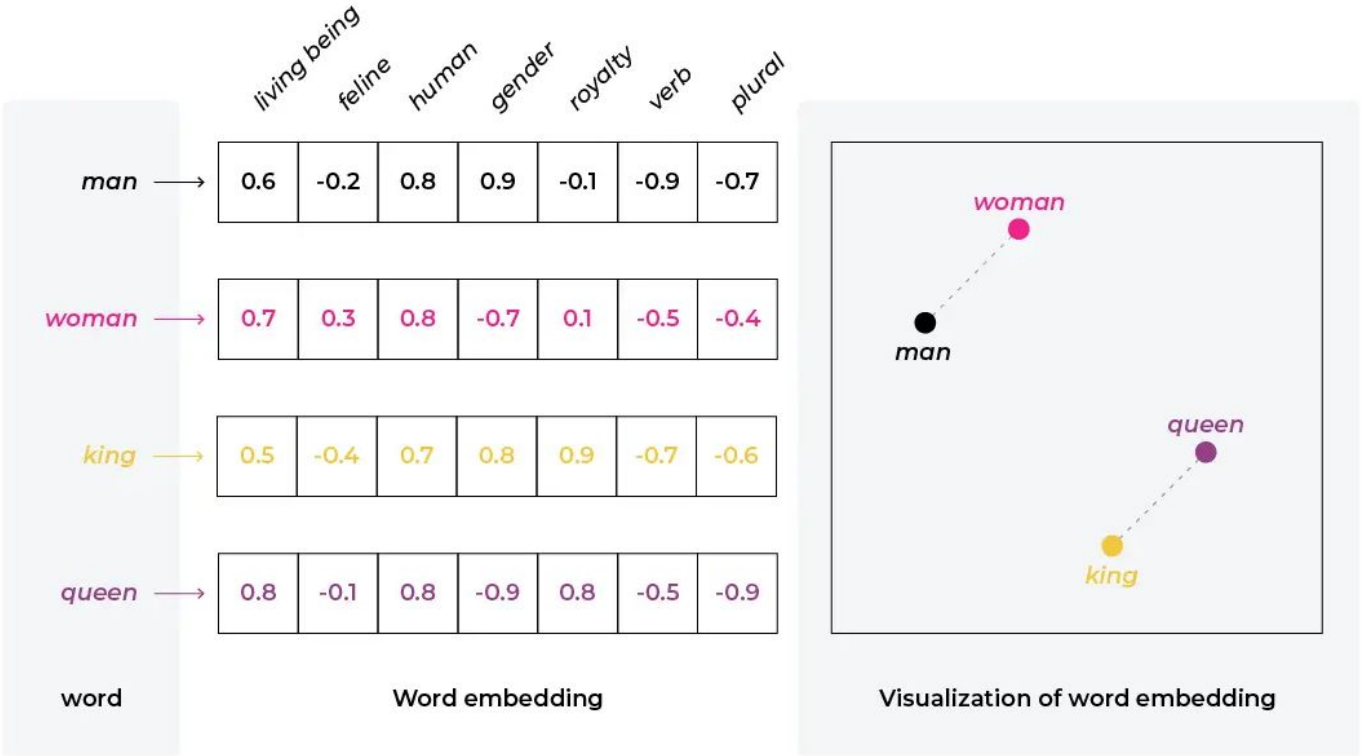
Source of image (try exploring interactively)

Quick intro: Embeddings



Source: [Embeddings: Meaning, Examples and How To Compute]

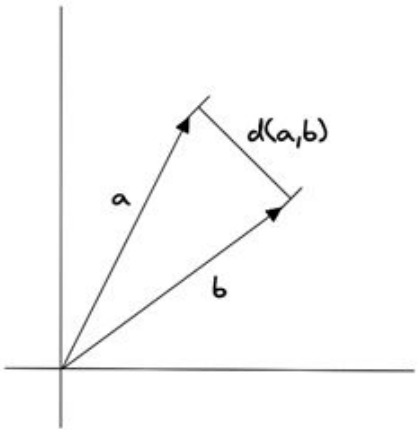
Quick intro: Embeddings



Source: [Embeddings: Meaning, Examples and How To Compute]

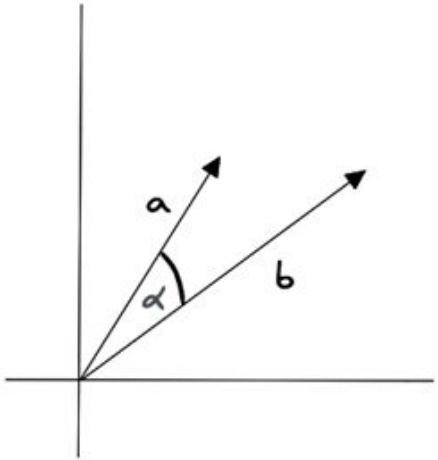
Quick intro: Embeddings

Euclidean distance



Distance between ends of vectors

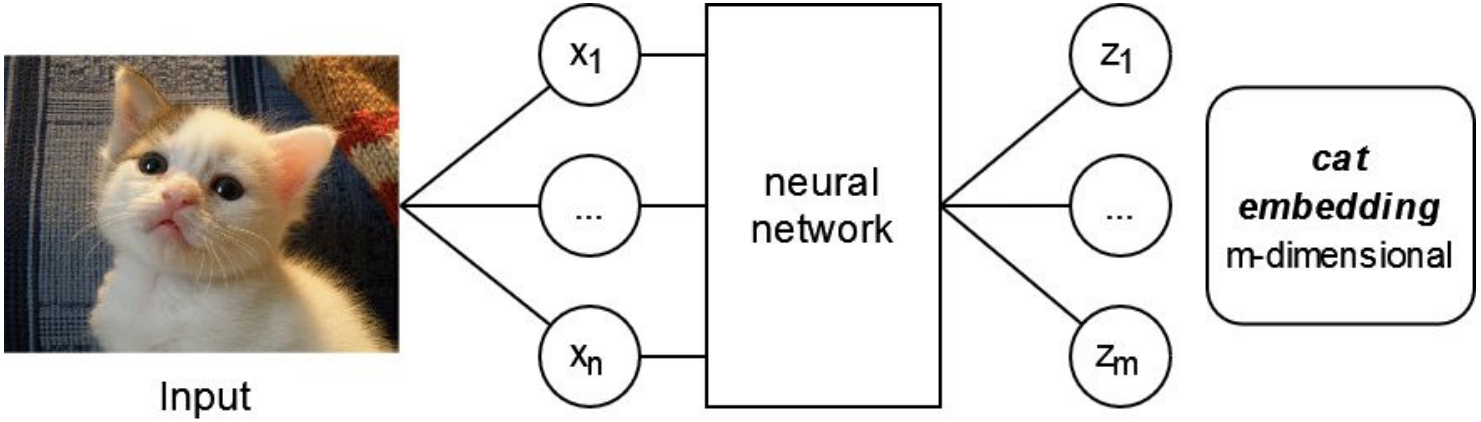
Cosine similarity



Cosine of angle θ between vectors

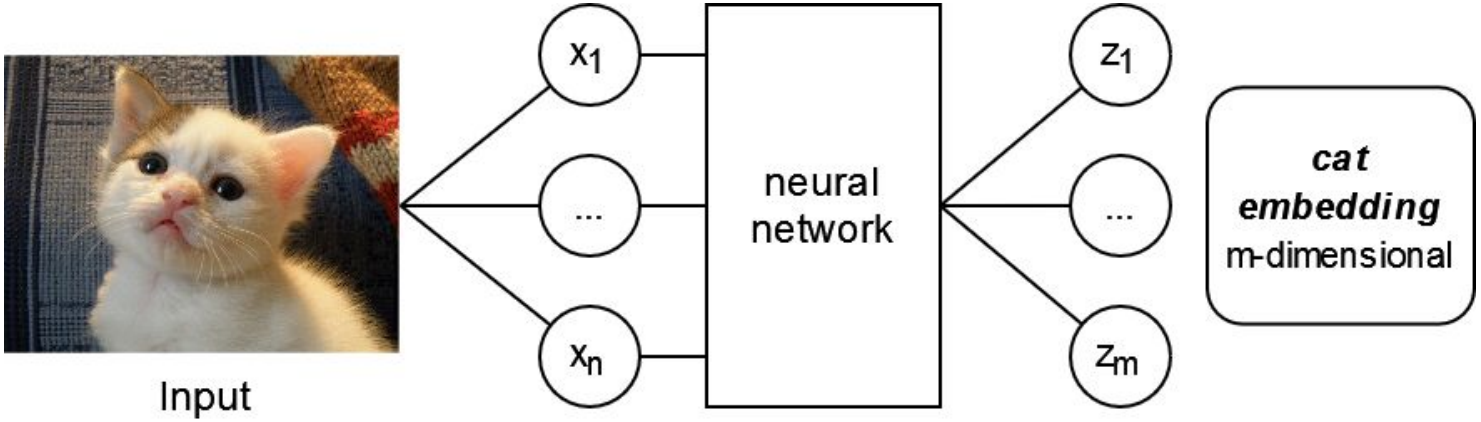
Source: openHPI Embeddings MOOC

Regression and classification - Intro to NN design



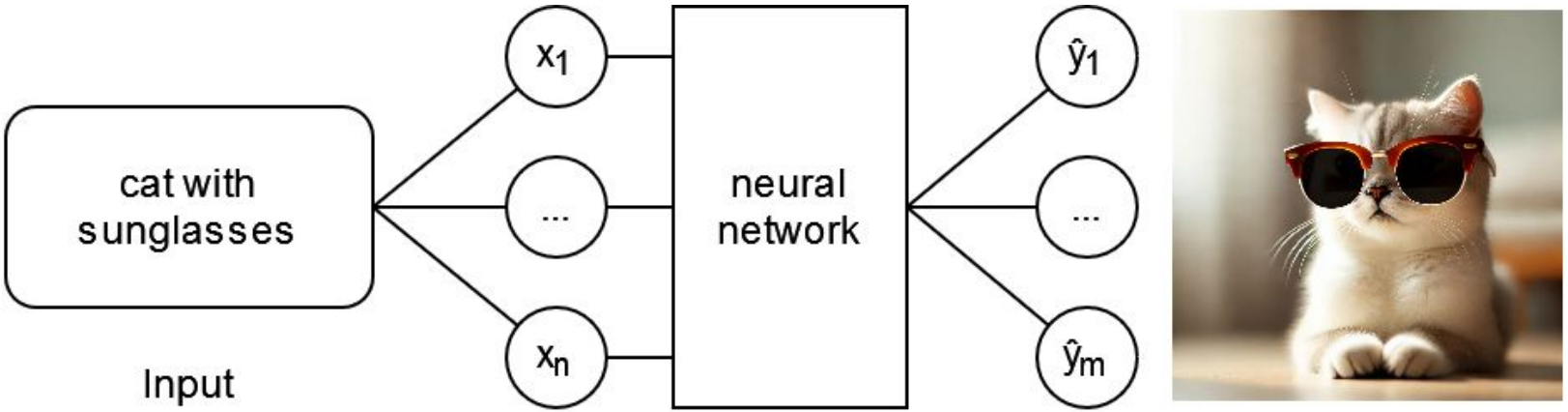
Q: Is creating embeddings a classification or regression task?

Regression and classification - Intro to NN design



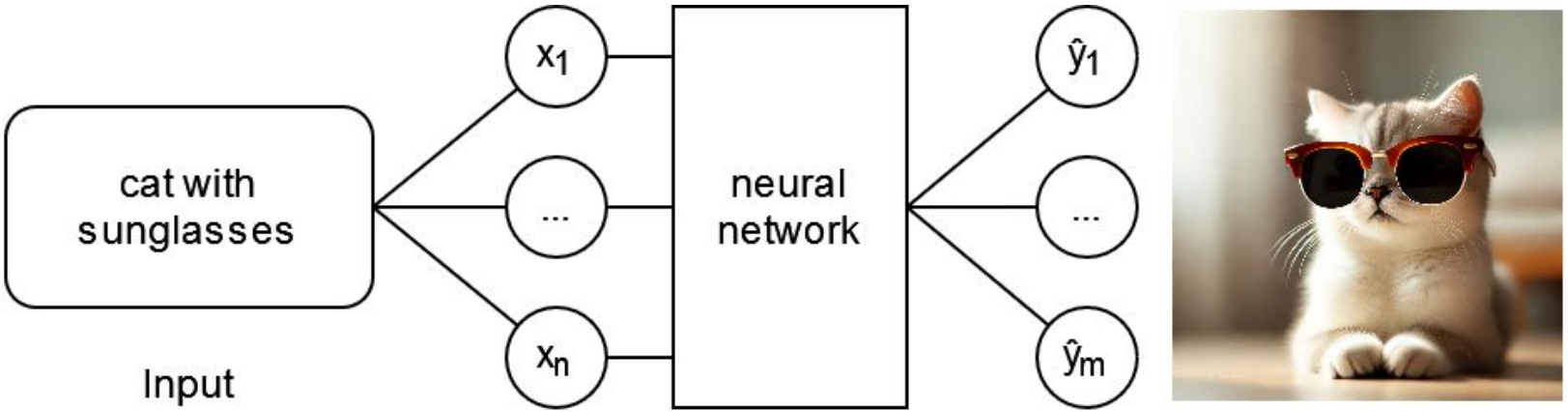
A: It's an artifact automatically created in both approaches.

Regression and classification - Intro to NN design



Q: Is generating an image a classification or regression task?

Regression and classification - Intro to NN design

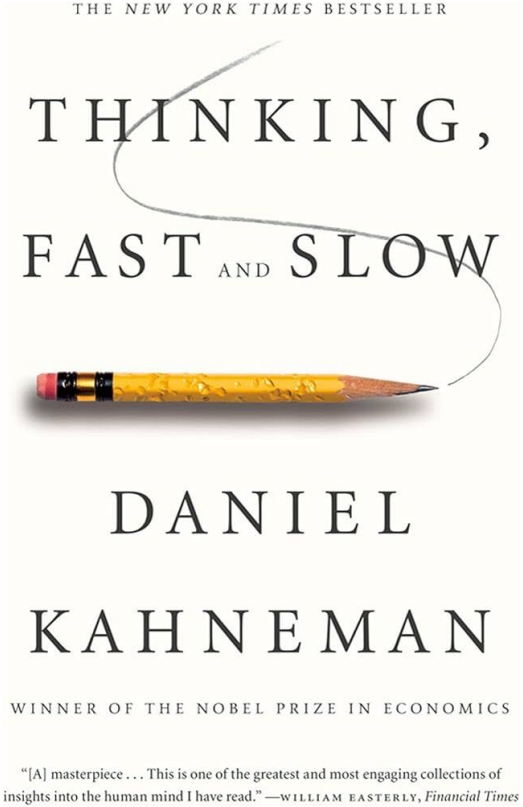


A: It's a *generative* problem and can be either of them or even a combination of regression and classification.

Applications of deep learning

- Computer vision: image classification, object detection, segmentation, image description, image generation, satellite and drone image analysis
- Natural Language Processing: translation, DNA sequence prediction, text generation
- Audio understanding, audio generation, lip reading
- Playing games, finding optimal strategies (when paired with reinforcement learning and search algorithms)
- Any system that requires the finding of **significant correlations** or patterns/”intuitions” in the data, i.e. the ‘thinking fast’ systems referred by Daniel Kahneman in his famous book.

Applications of deep learning



- System I guides quick judgments
- System II monitors and corrects when needed

Applications of deep learning

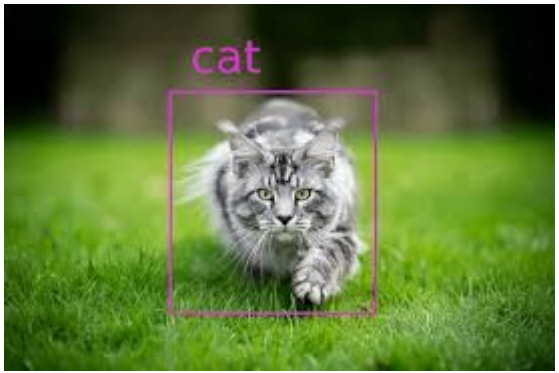


Image source

- System I - Deep Learning
- System II - Deep Learning + Search and Planning

RESEARCH

AI achieves silver-medal standard solving International Mathematical Olympiad problems

25 JULY 2024

AlphaProof and AlphaGeometry teams

Share

Source

Why do we use deep learning?

- Deep neural networks are ***universal function approximators***
  - *This means that given the right architecture and training, they can **compute everything that's computable**, just like a Universal Turing Machine*
  - *In practice this means that deep networks are **very good** at finding very sophisticated decision boundaries (aka "curve fitting")*
  - *Finding the function means the finding **architecture and or training** of the neural network*
  - *It's **fair** to call deep neural networks "**linear regression on steroids**"*
  - *Deep neural networks pick hidden **correlations** in the data, but most architectures don't pick on **causality** (current research topic)*

Should we always try first the latest and newest architecture?

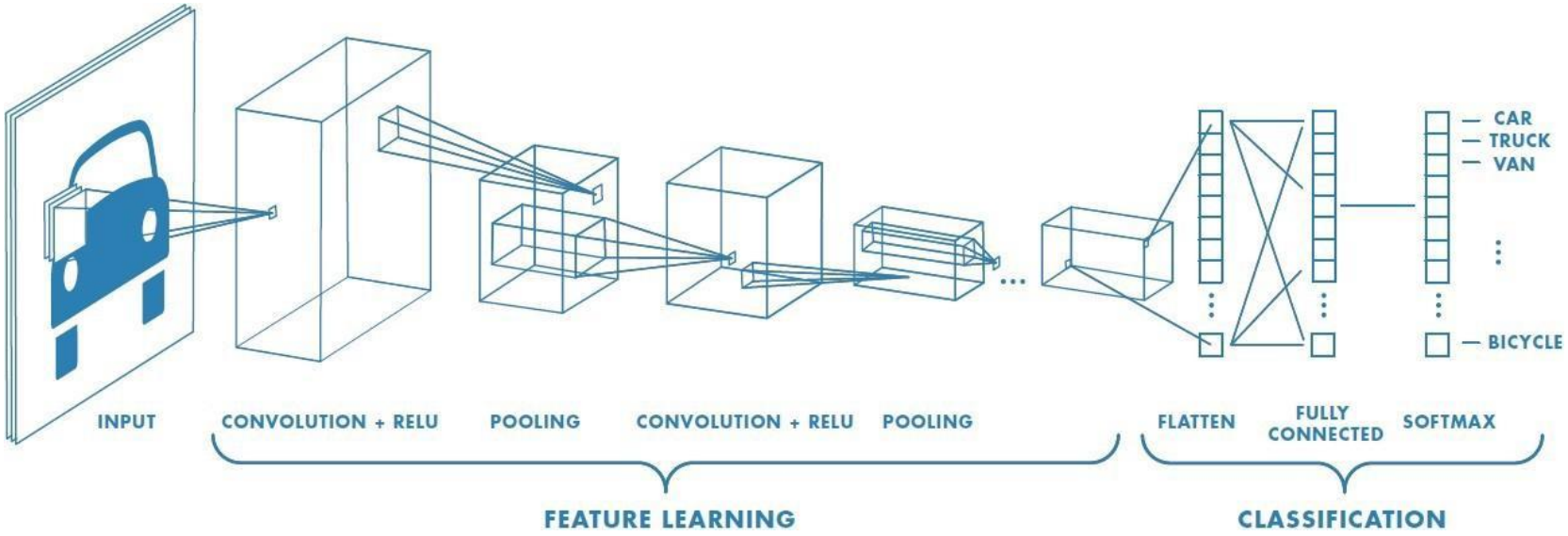
Inductive biases

Different architectures (*ways to ‘stir the pile’*) correspond to:

- Different assumptions on what input affects the output
- Different goals encoded on the loss function (e.g. regression vs classification)
- Generation vs discrimination

Tradeoffs between *amount of computation vs acceptable performance* are dominant when considering different architectures

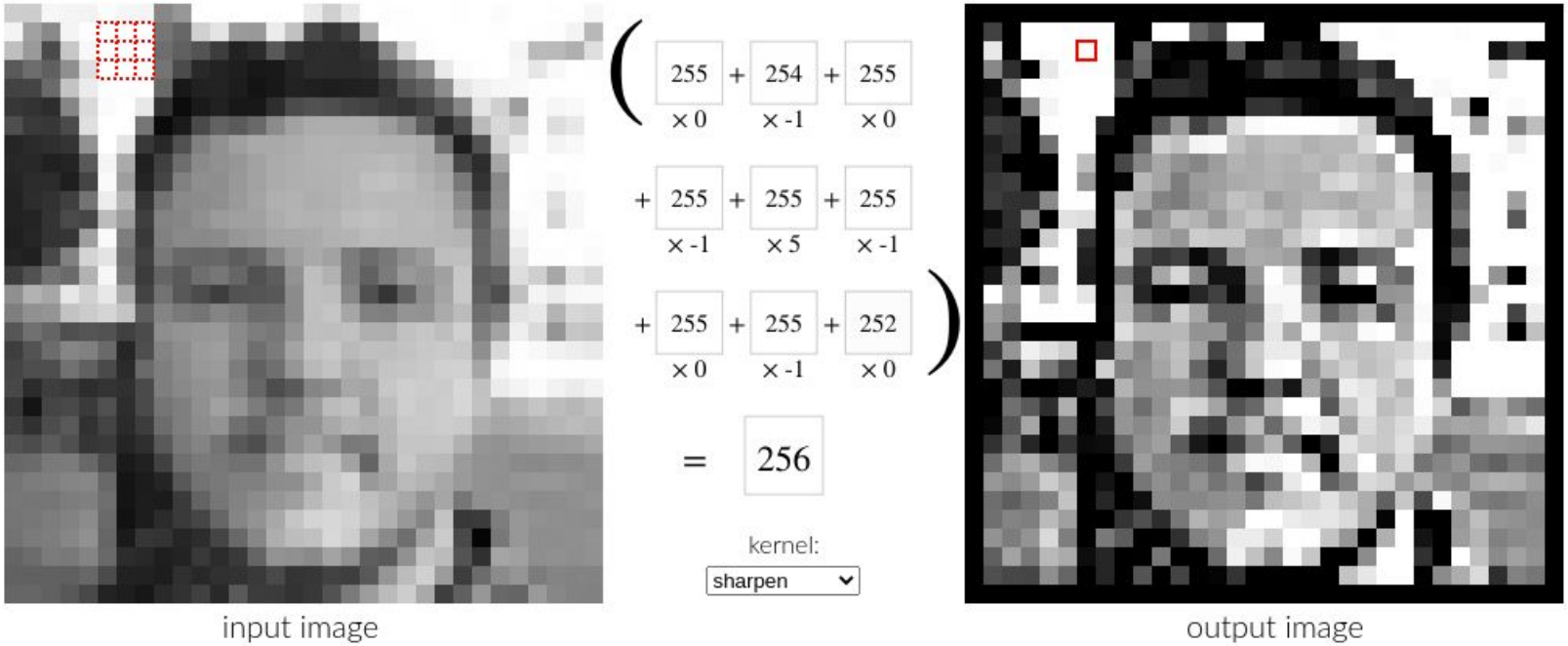
Convolutional neural networks



Q: Where is the multilayer perceptron in this diagram?

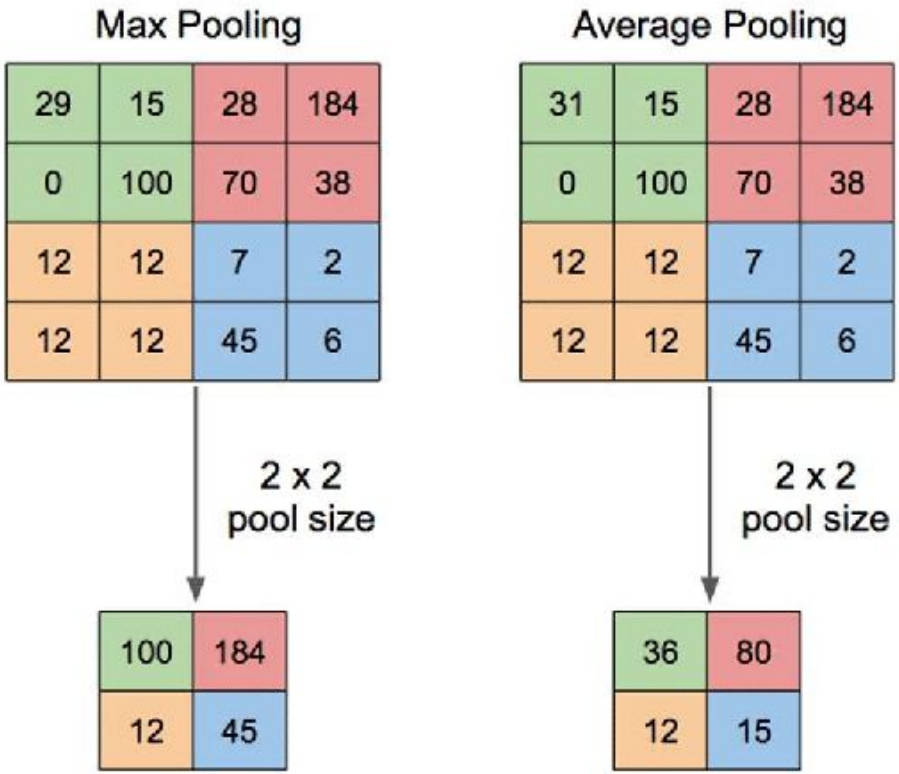
Image source

What is a convolution filter?

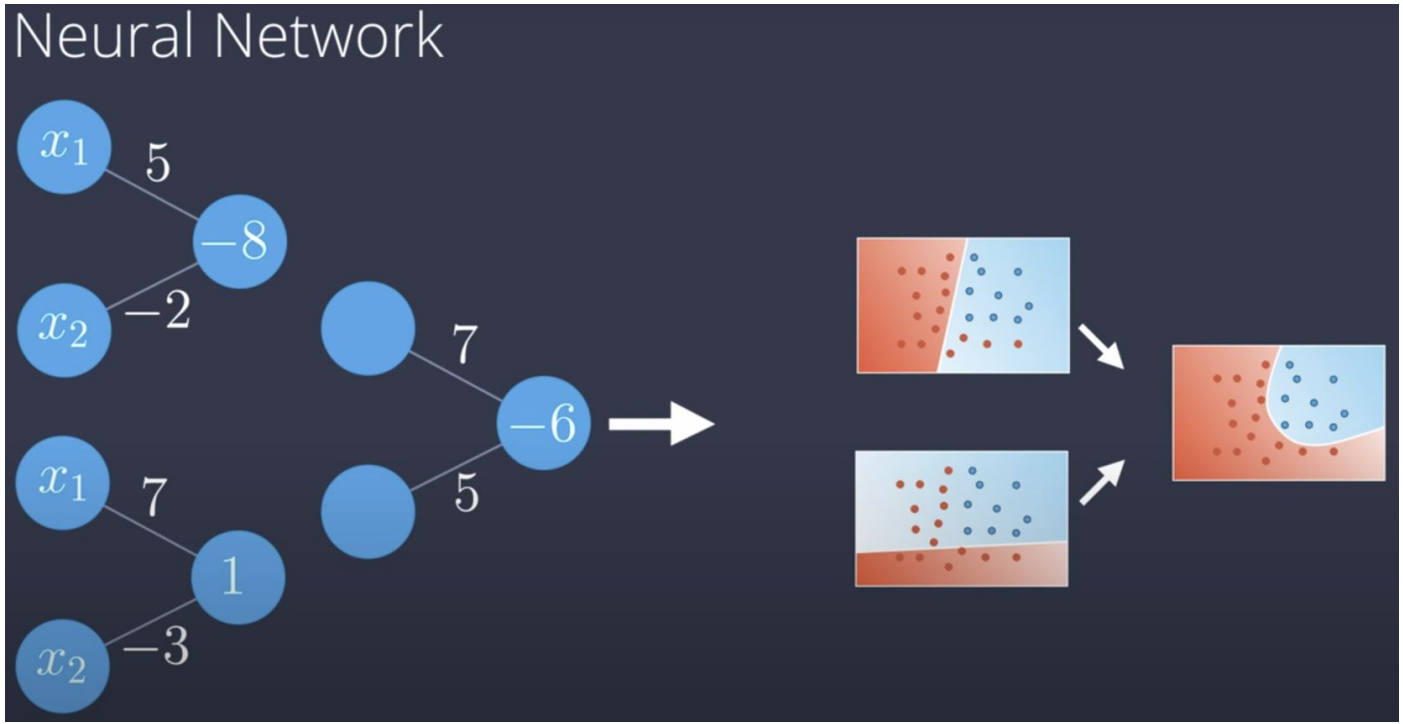


https://setosa.io/ev/image-kernels

Pooling

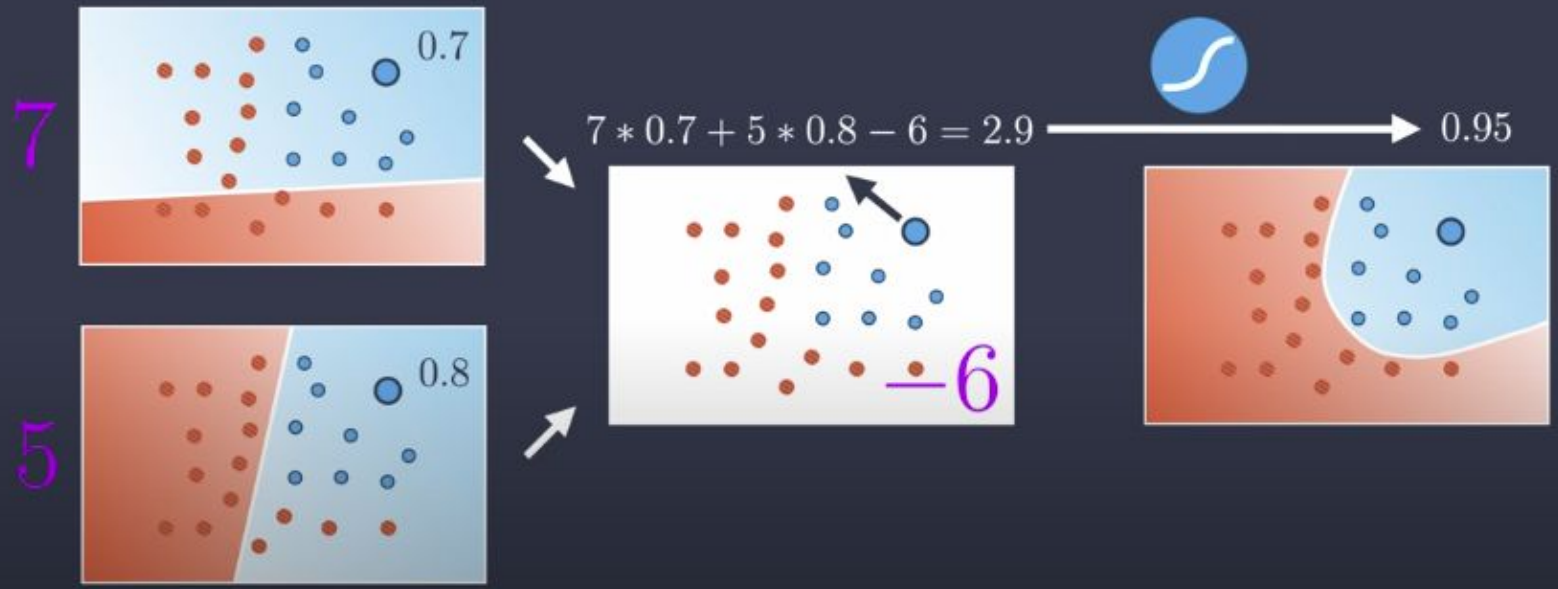

Max Pooling
29 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
100 184
12 45
Average Pooling
31 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
36 80
12 15

Learning non-linearities

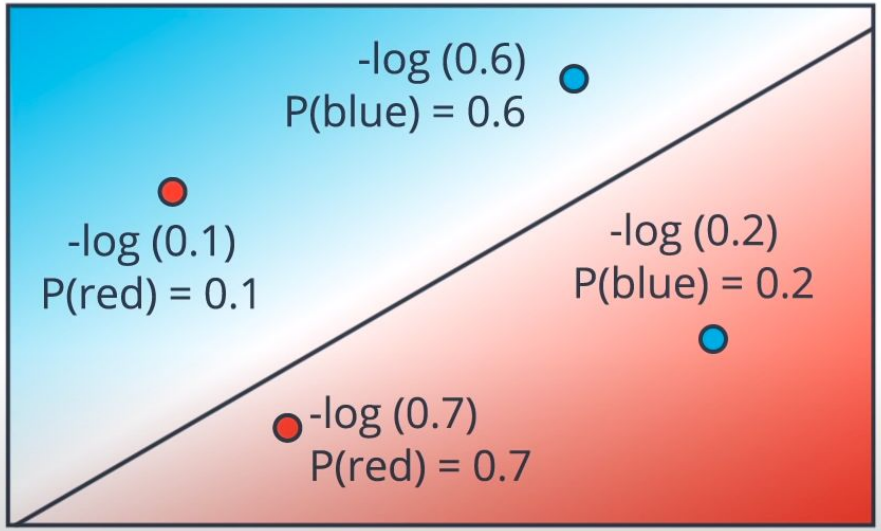


How neural networks learn non-linearities

Neural Network



Error Function



$-\log(0.6) - \log(0.2) - \log(0.1) - \log(0.7) = 4.8$

0.51 1.61 2.3 0.36

If $y = 1$

$P(\text{blue}) = \hat{y}$

$\text{Error} = -\ln(\hat{y})$

If $y = 0$

$P(\text{red}) = 1 - P(\text{blue}) = 1 - \hat{y}$

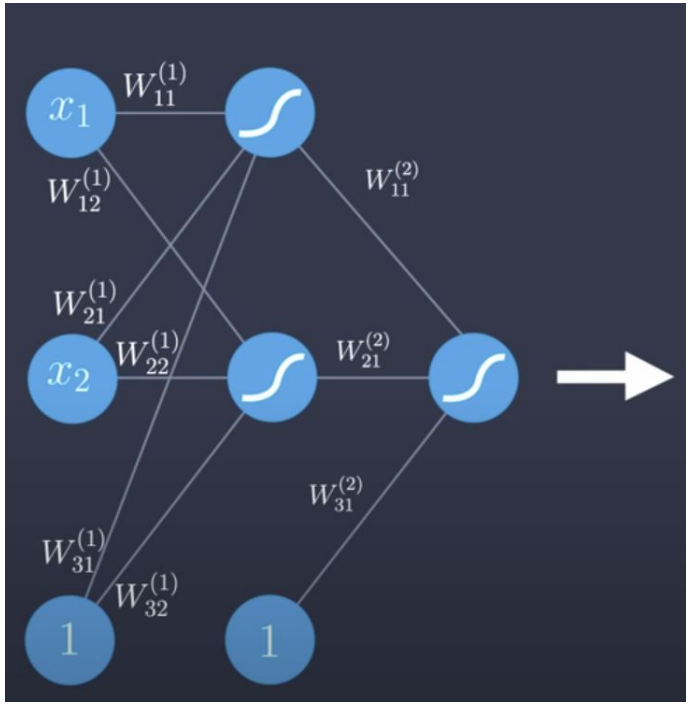
$\text{Error} = -\ln(1 - \hat{y})$

$$\text{Error} = -(1-y)(\ln(1-\hat{y})) - y\ln(\hat{y})$$

$$\text{Error Function} = -\frac{1}{m}\sum_{i=1}^{m}(1-y_i)(\ln(1-\hat{y}_i)) + y_i\ln(\hat{y}_i)$$

Q: Which of these points have higher cross entropy?

The feedforward pass

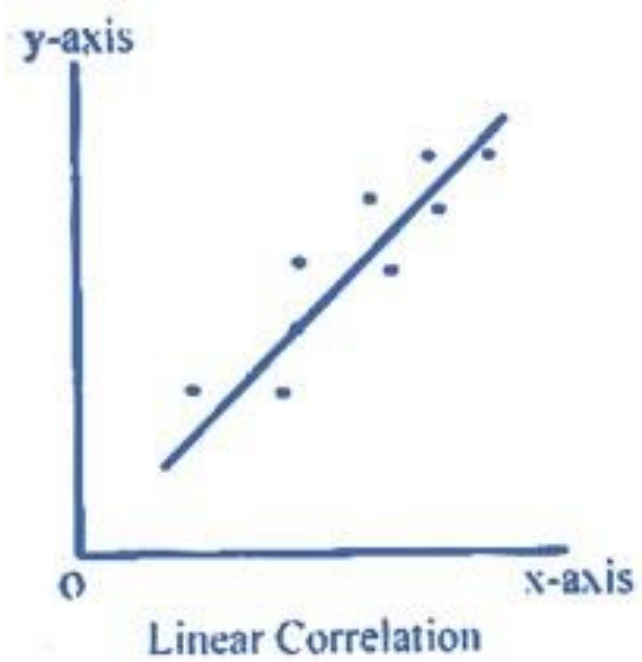


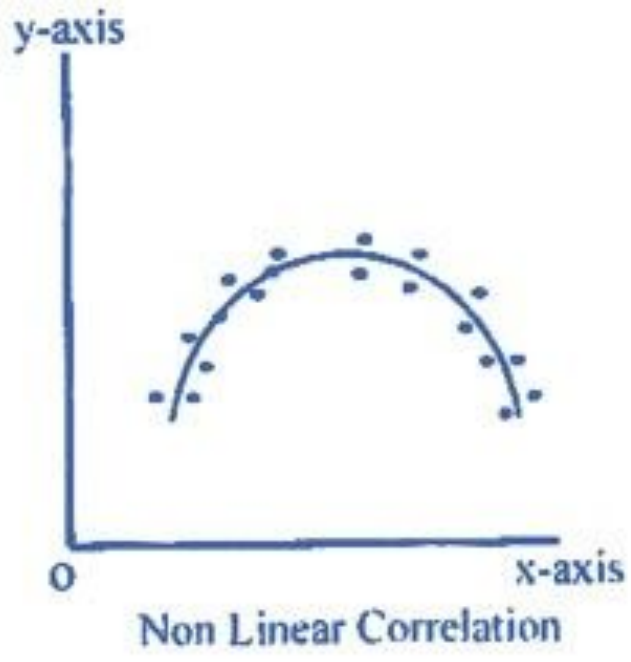
$$\hat{y} = \sigma \begin{pmatrix} W_{11}^{(2)} \\ W_{21}^{(2)} \\ W_{31}^{(2)} \end{pmatrix} \sigma \begin{pmatrix} W_{11}^{(1)} & W_{12}^{(1)} \\ W_{21}^{(1)} & W_{22}^{(1)} \\ W_{31}^{(1)} & W_{32}^{(1)} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix}$$

$$\hat{y} = \sigma \circ W^{(2)} \circ \sigma \circ W^{(1)}(x)$$

[Understanding the feedforward pass]

Learning non-linearities

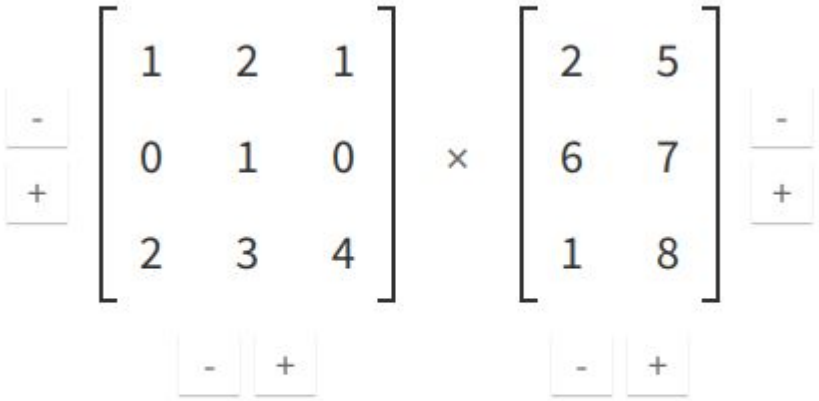




How neural networks learn non-linearities

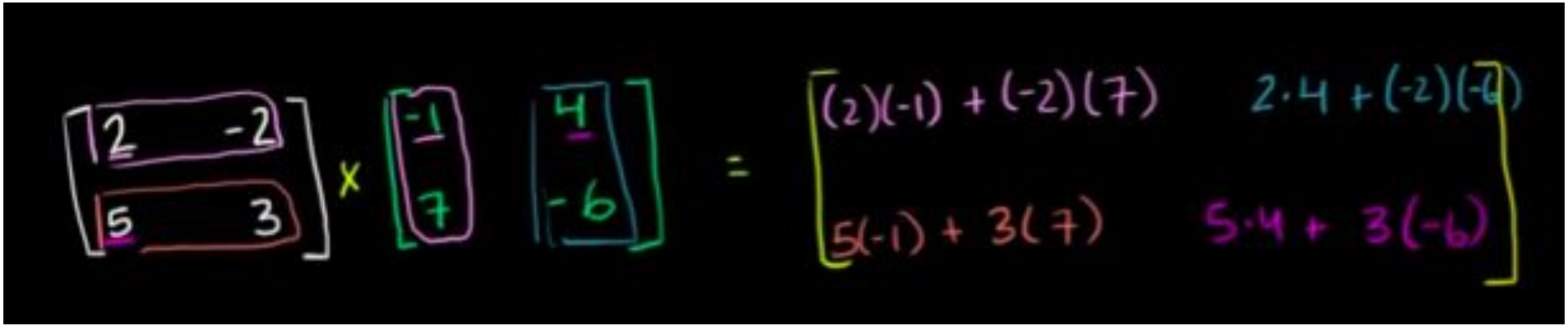
- Read: scalars, vectors, matrices, and tensors
  - Understanding rank terminology

Matrix multiplication



http://matrixmultiplication.xyz/

Matrix multiplication



Khan Academy Tutorial

Pay attention to the size of the input

Anthony Wang @aytwang · 15h

Replying to @the_antlr_guy

Wow, this is extraordinary. I can't count the number of hours I've spent in the debugger combing through every variable that could have gone wrong using print(x.shape) statements 😂

1 16

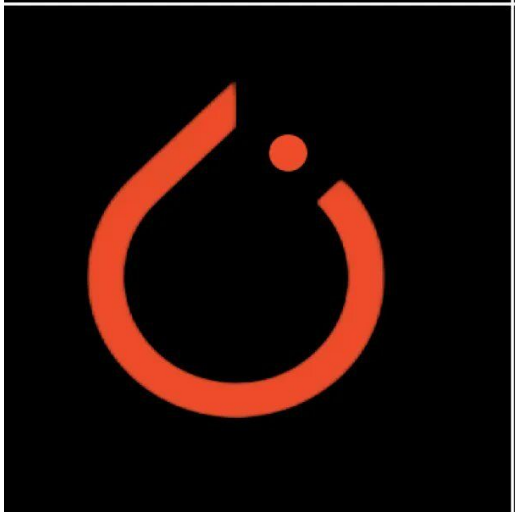
Terence Parr @the_antlr_guy · 15h

haha! Same here! Seems like print(x.shape) is the most common deep learning programming construct there is!

2 6 17

Upgrade from 1.x to 2.0:

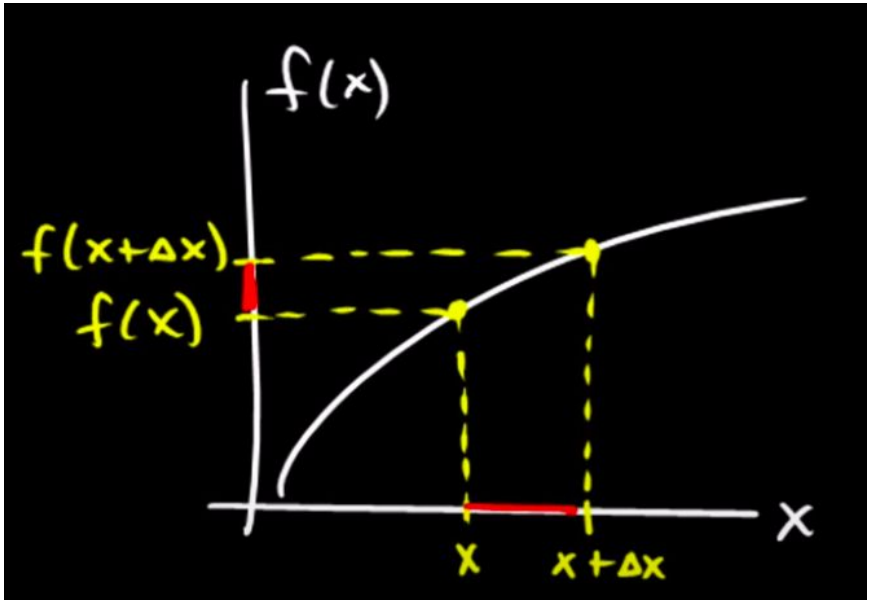
Nothing works
Documentation missing.
What happened to tflite
CUDA hell
Dafuq is gradient_tape
Where's .contrib



Upgrade from 1.x to 2.0:

Everything works
Live Q&A before release.
Clear deprecations
.compile() ❤️
Actually open source
The best is yet to come

Numerical calculation of derivatives



Numerical derivatives

Numerical and analytic derivatives in Python

Backpropagation

Finding the derivative of the error [edit]

Calculating the partial derivative of the error with respect to a weight w_{ij} is done using the chain rule twice:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net}_j}\frac{\partial \mathrm{net}_j}{\partial w_{ij}} \qquad \textbf{(Eq. 1)}$$

In the last factor of the right-hand side of the above, only one term in the sum net_j depends on w_{ij}, so that

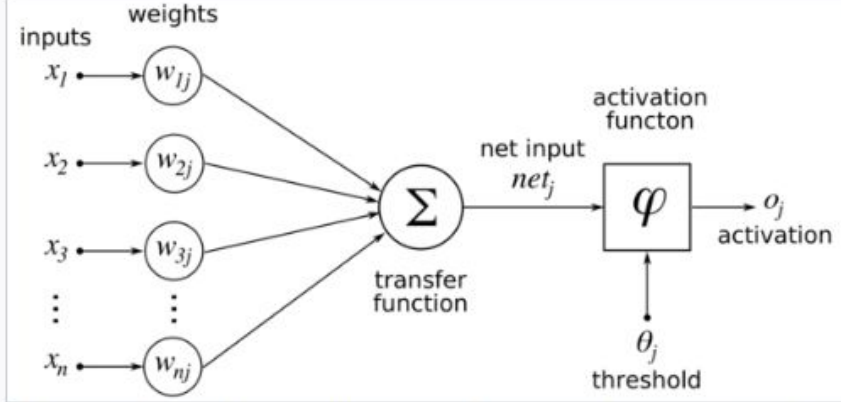


Diagram of an artificial neural network to illustrate the notation used here.

https://en.wikipedia.org/wiki/Backpropagation

The chain rule

Computing the derivative of a composite function:

$$f(x) = u(v(x)) \qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

$$f(x) = u(v(w(x))) \qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

Gradient computation graphs

Backpropagation: a simple example

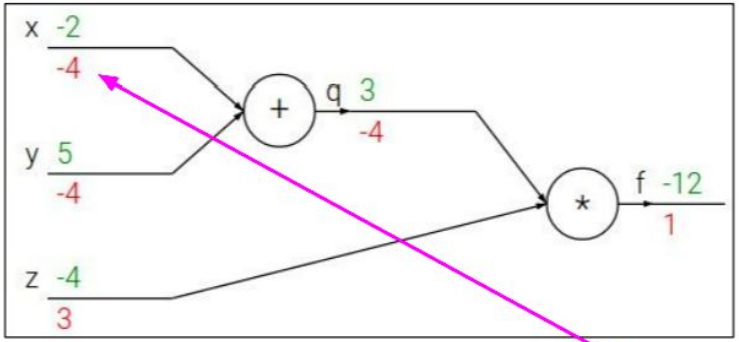
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

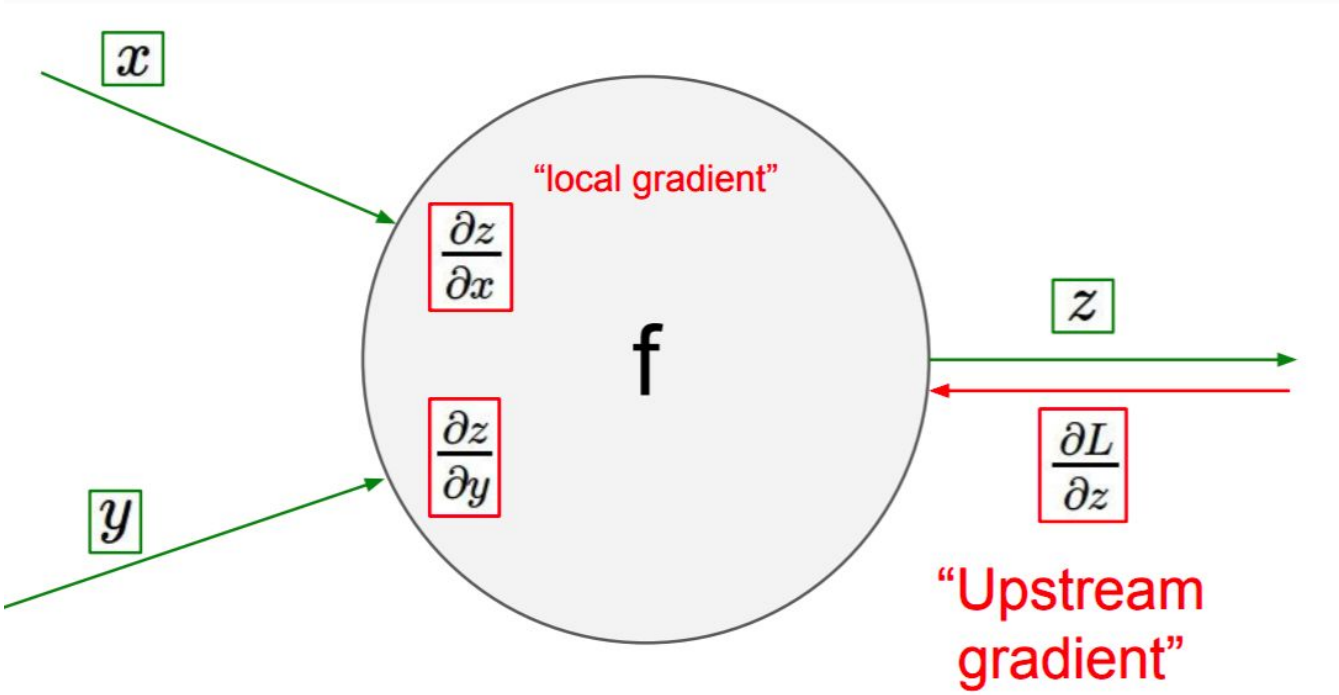
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture04.pdf

Gradient computation graphs

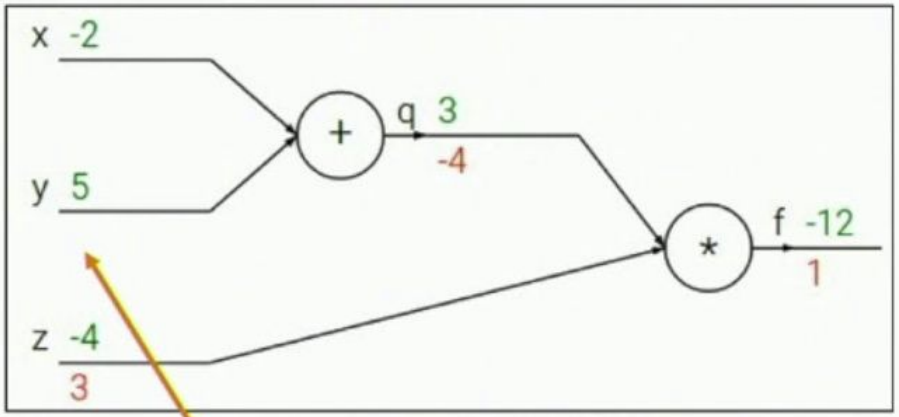


http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture04.pdf

Backpropagation: Simple Example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4



1. Forward pass: Compute outputs

$$q = x + y \qquad f = qz$$

2. Backward pass: Compute derivatives

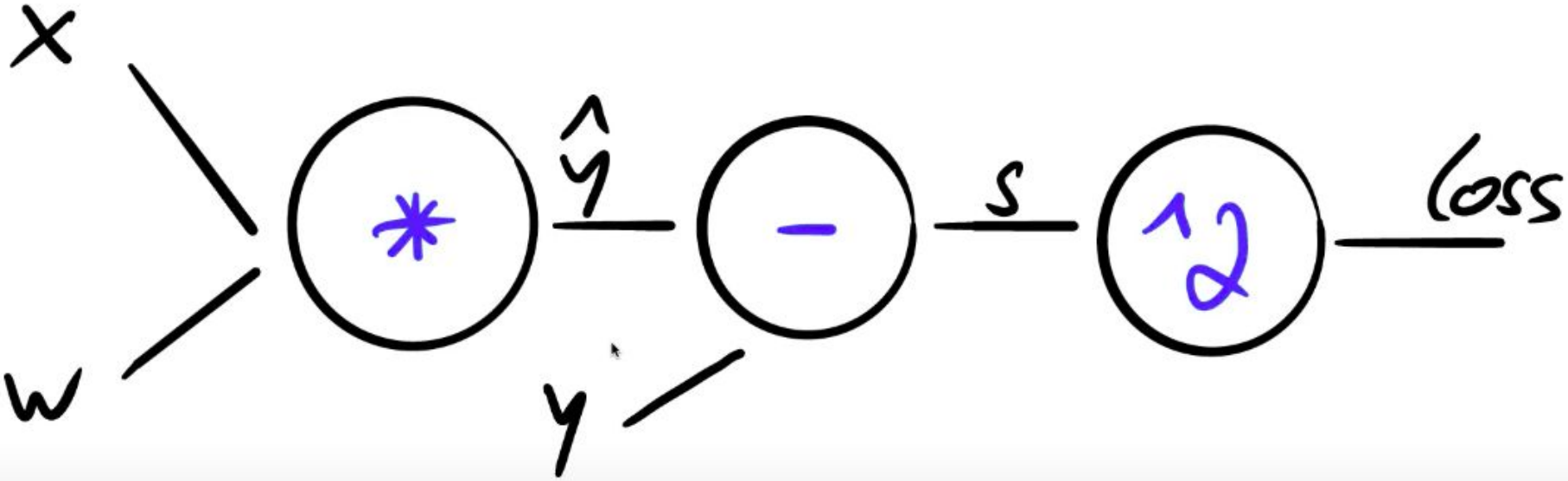
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Chain Rule

$$\frac{\partial f}{\partial y} = \frac{\partial q}{\partial y}\frac{\partial f}{\partial q}$$

Justin Johnson's lecture

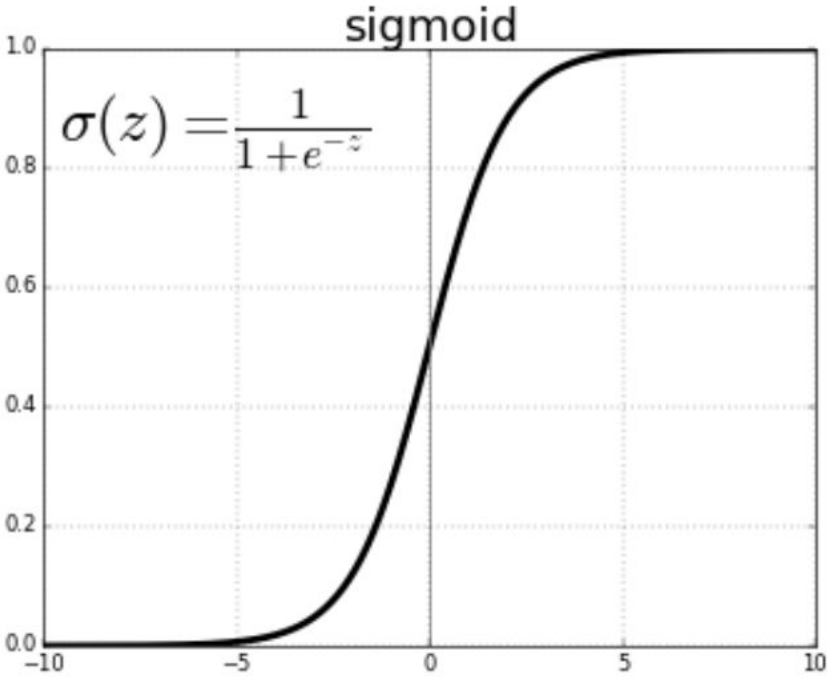
$\hat{y} = w \cdot x$ $\quad loss = (\hat{y} - y)^2 = (wx - y)^2$



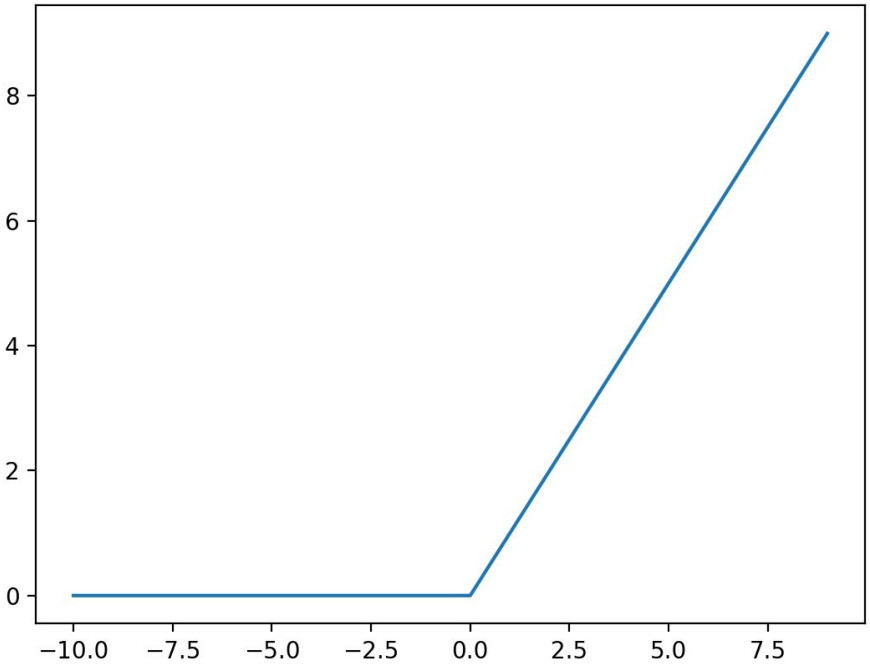
Backpropagation in PyTorch

- Watch this (clearest, most practical explanation on backprop ever)
  - Notebook
  - Solution
- Review questions
  - What is backpropagation doing?
  - What needs to happen before we call **backward()** in PyTorch?
  - What is the gradient computation graph?
  - Why is **grad_fn** changing with every operation done to something involving the weights?
  - Is the rectifier linear unit a linear or a non-linear function?
  - Is a torch tensor always run on a GPU?

The sigmoid activation function



The rectifier linear unit (ReLU) activation function



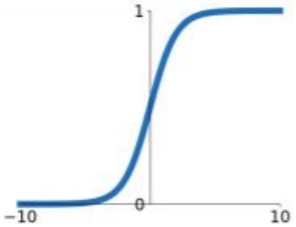
$$f(x) = x^+ = \max(0, x)$$

Activation functions

ReLU is a good default choice for most problems

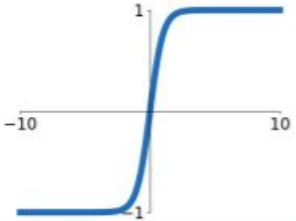
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



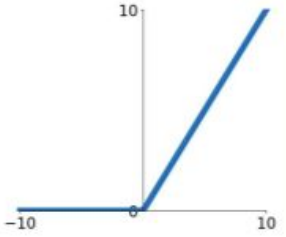
tanh

$$\tanh(x)$$



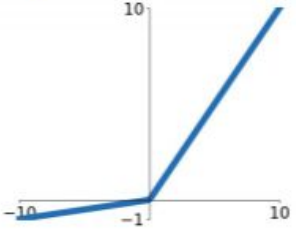
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

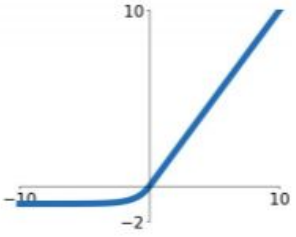


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

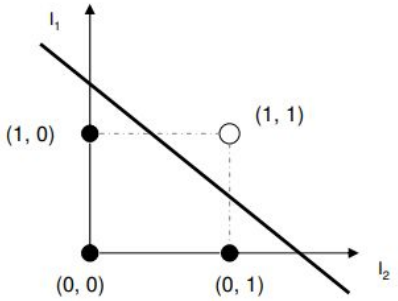


http://cs231n.stanford.edu/slides/2019/cs231n_2019_lecture04.pdf

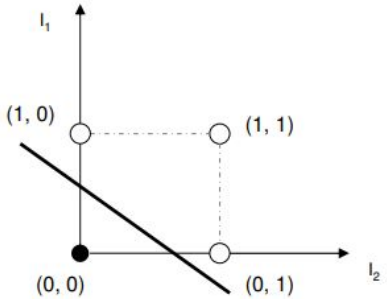
Doing logic with a neural network

- Experiments, Solution notebook

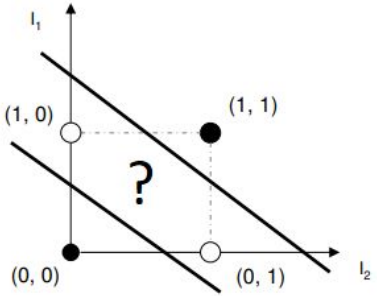
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



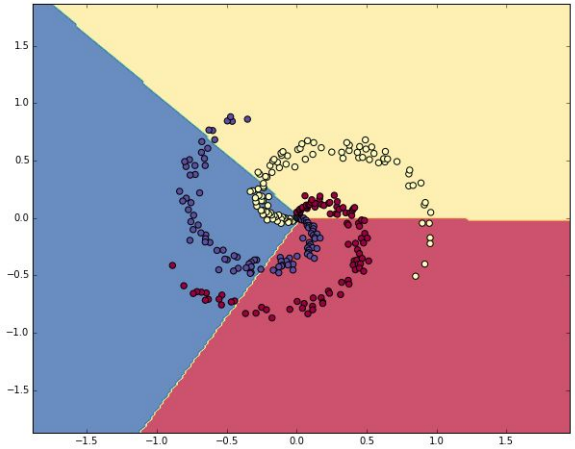
OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

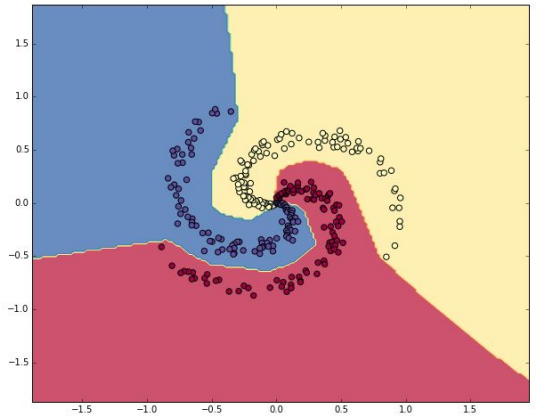


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



Learning non-linearities





https://cs231n.github.io/neural-networks-case-study

Experiment notebook

Q: What happens with large learning rates? What happens with large coefficients for l2 regularization?

What is a GPU?

Most deep learning libraries require the use of NVIDIA graphical processing units with CUDA capabilities

Why use a GPU for deep learning?

How is a CPU different from a GPU?

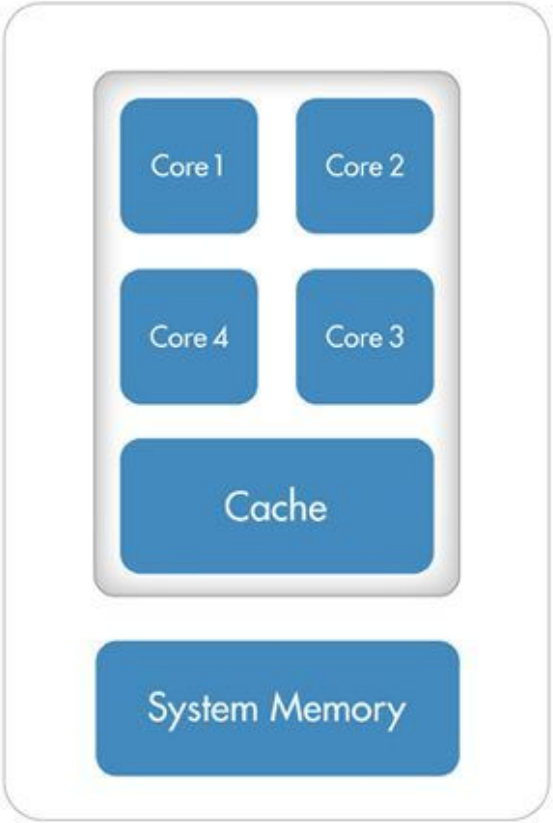
- More cores!

Why are CPUs less effective for deep learning?

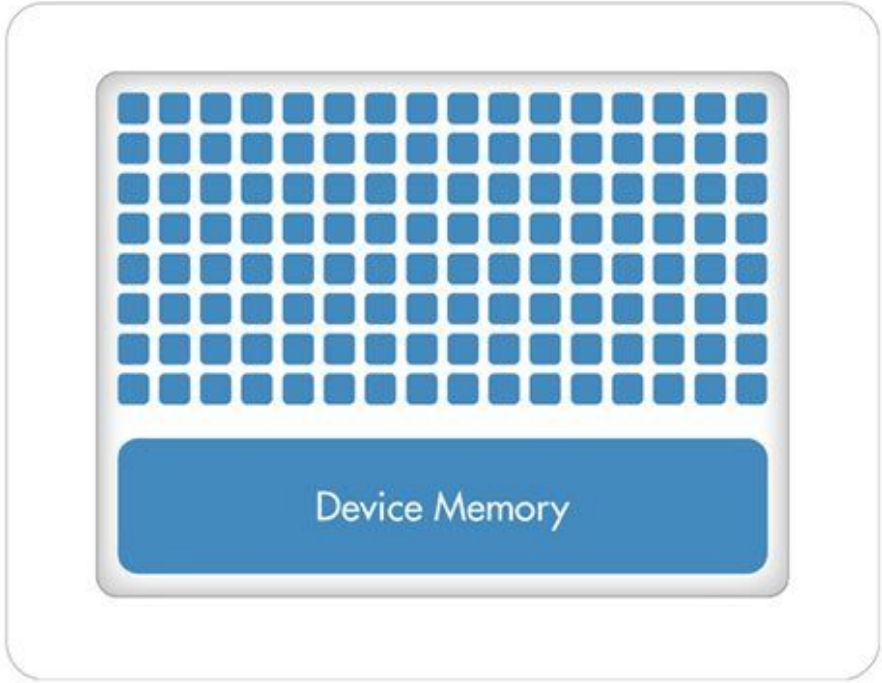
- Less cores :(

Why use a GPU for deep learning?

CPU (Multiple Cores)



GPU (Hundreds of Cores)



Choose the right loss function for the task

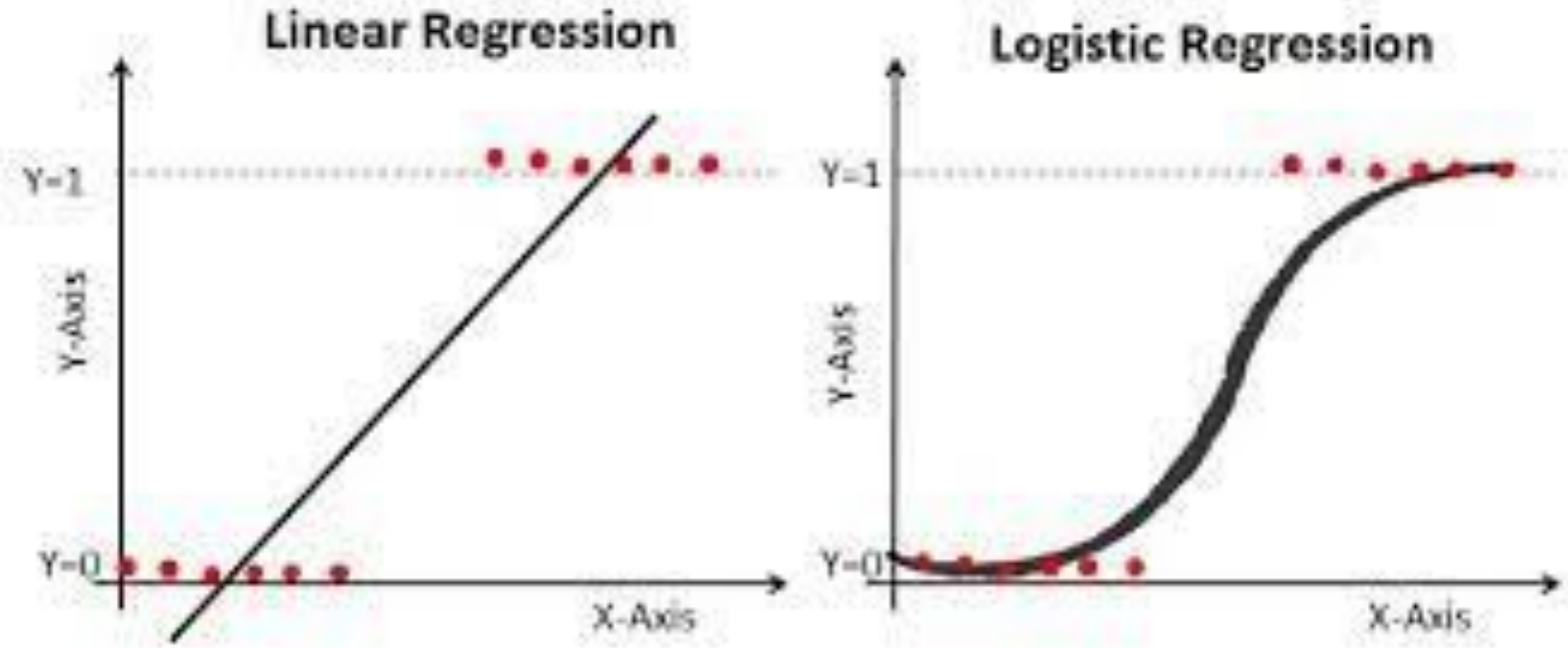
$$\mathrm{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \tilde{y}_i)^2$$

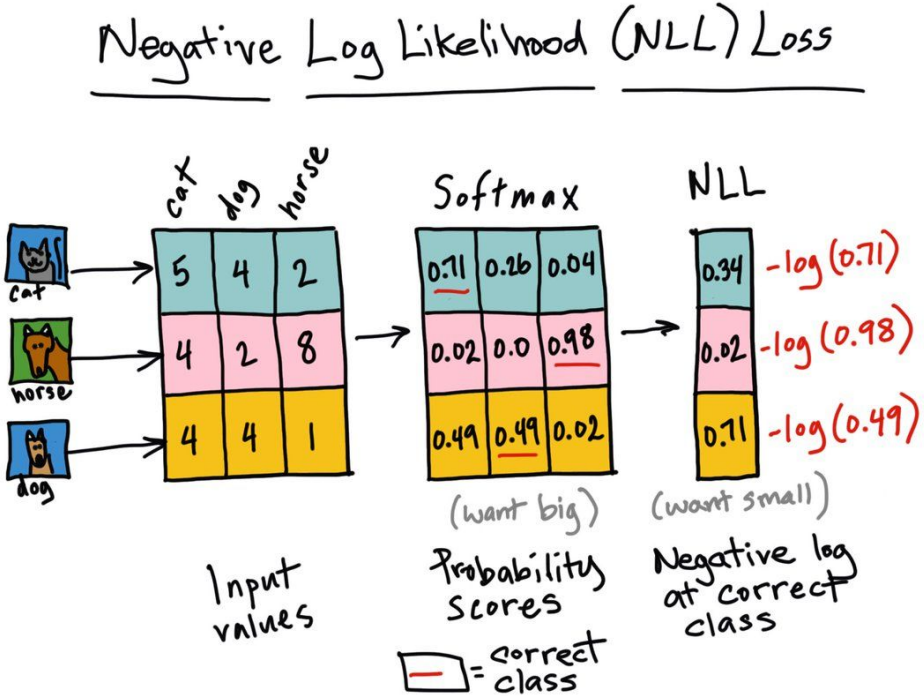
Mean Squared Error is a loss function for regression

Choose the right loss function for the task

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

Cross entropy is a function for classification, used interchangeably with negative log-likelihood, here we see it in its binary case

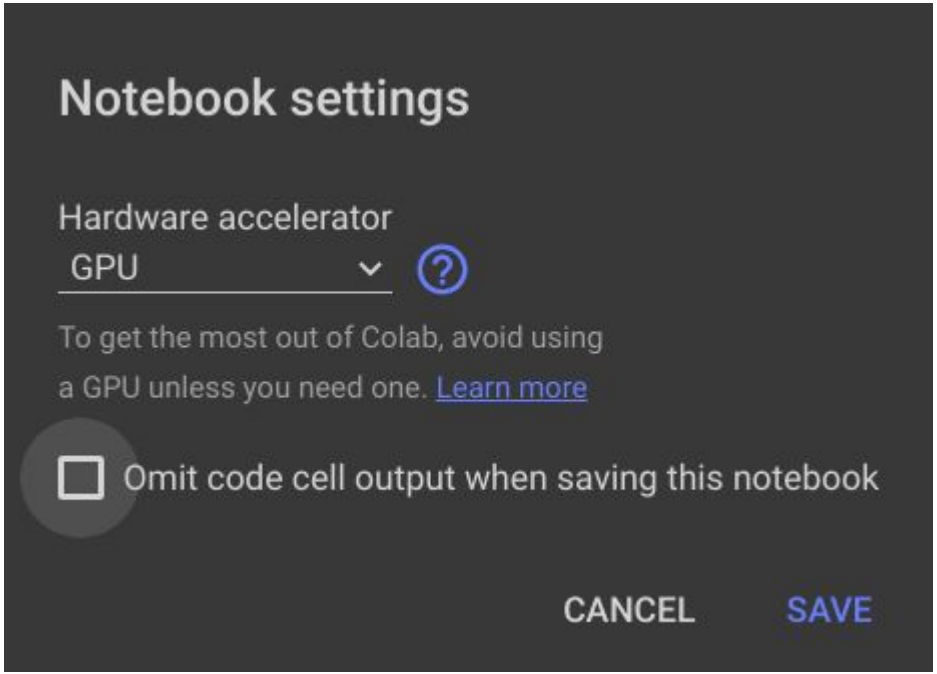

Linear Regression
Y=1
Y-Axis
Y=0
X-Axis
Logistic Regression
Y=1
Y-Axis
Y=0
X-Axis

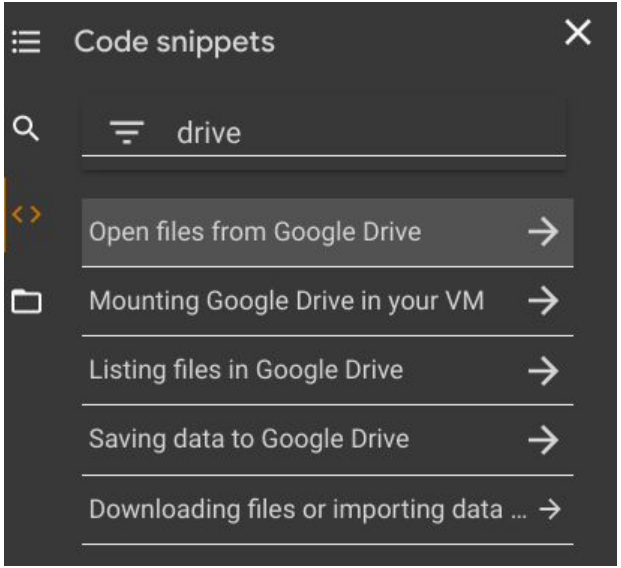


[Exercise](#)
[Solution](#)

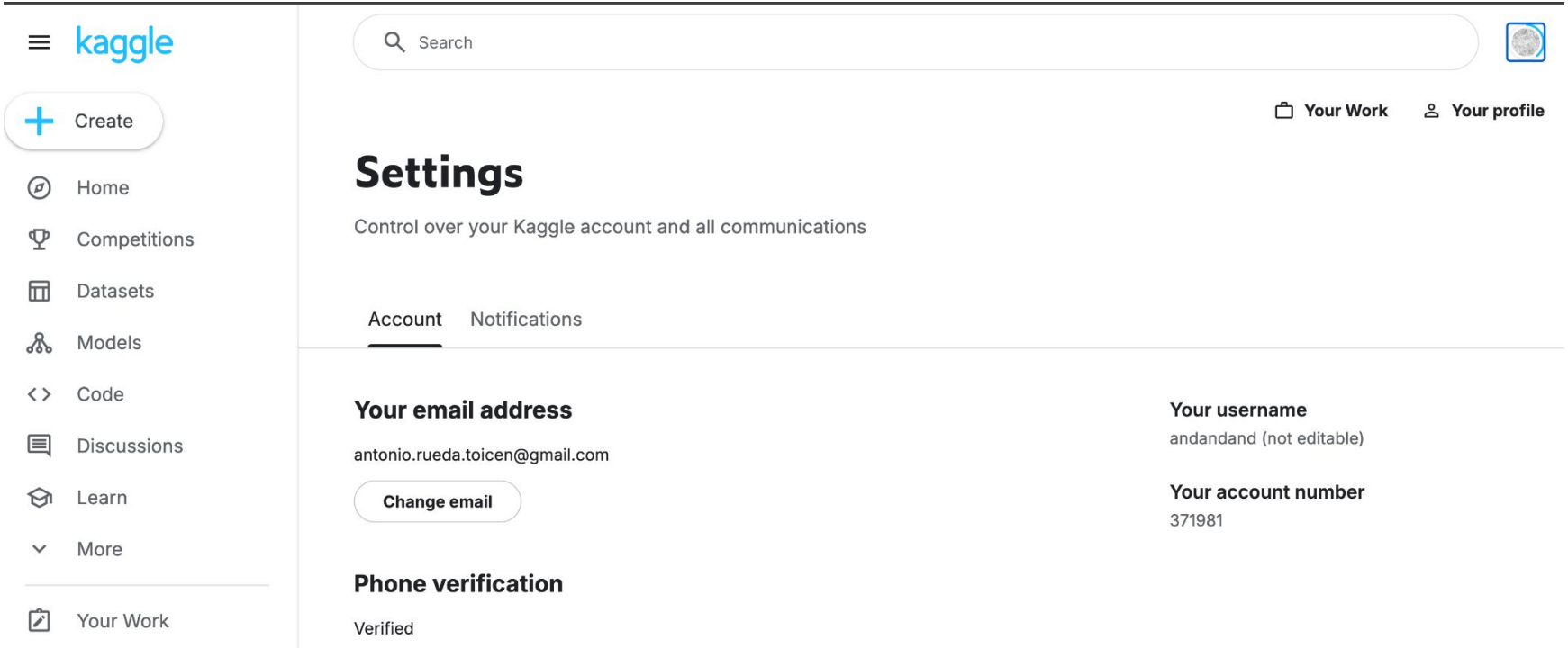
Refer to [this tutorial](#) and [this discussion](#)

Setting up Google Colab



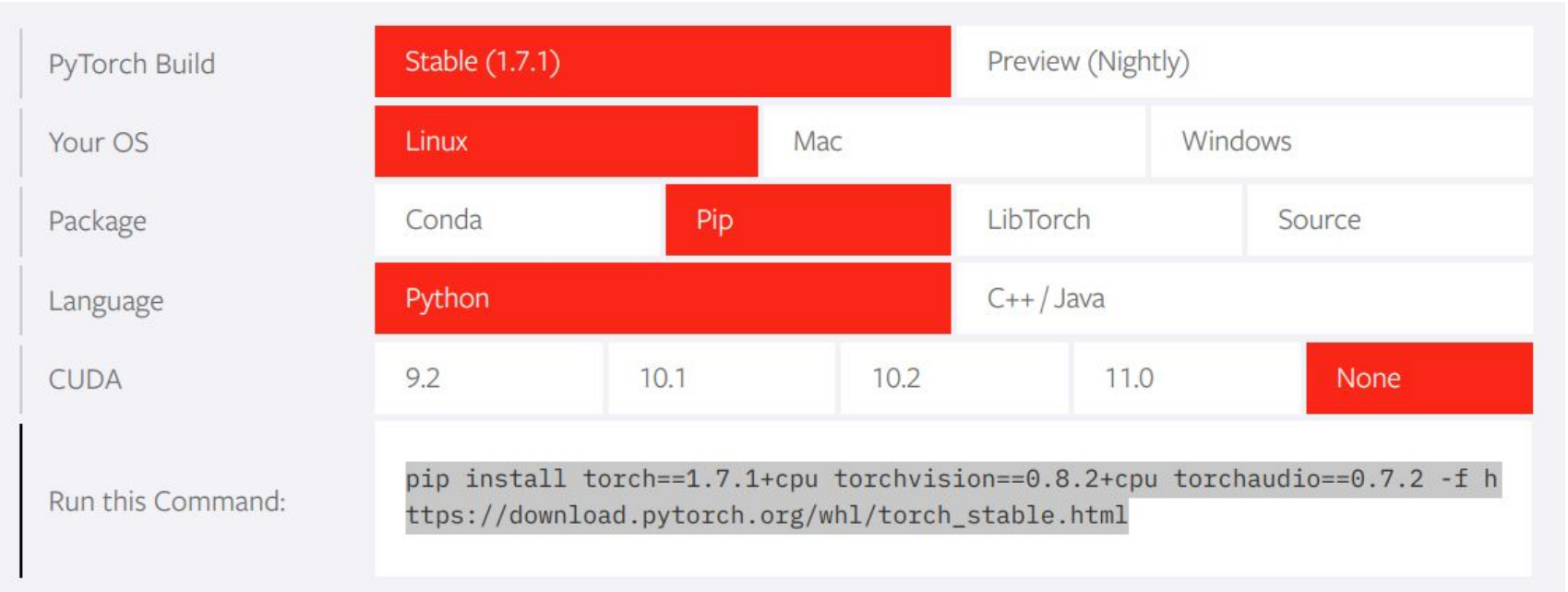


Setting up Kaggle



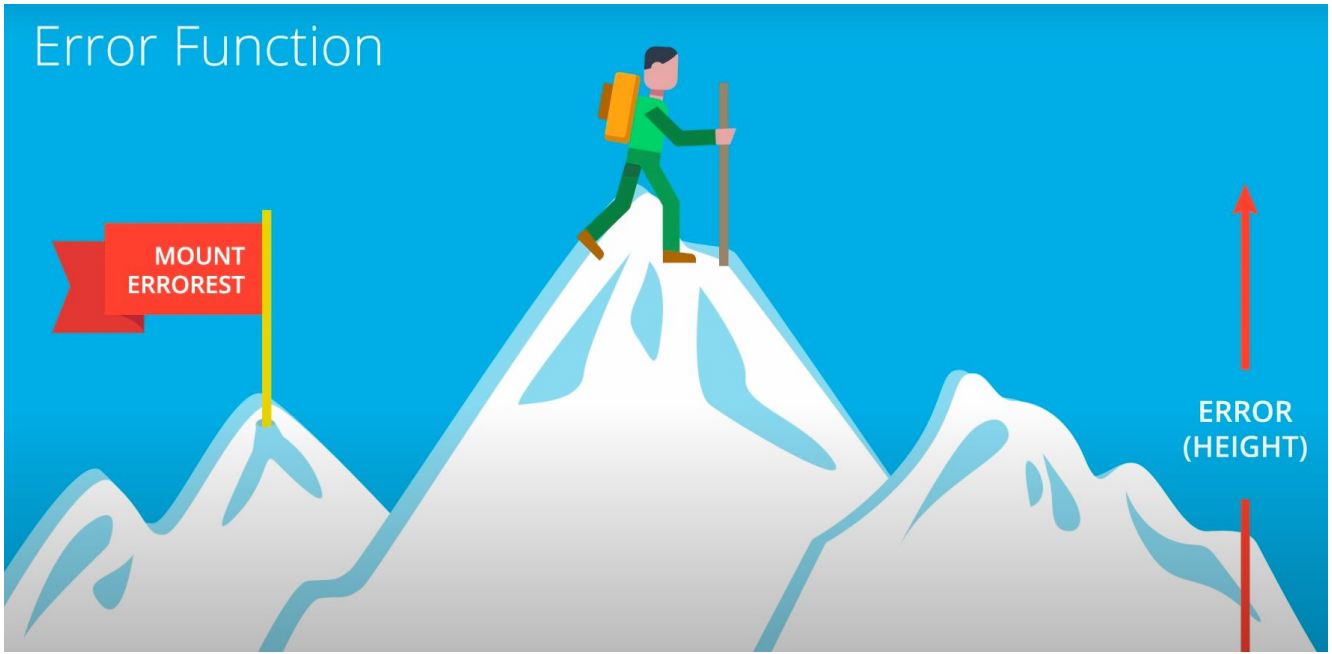
Phone verification grants access to accelerators (GPUs and TPUs)

Installing PyTorch



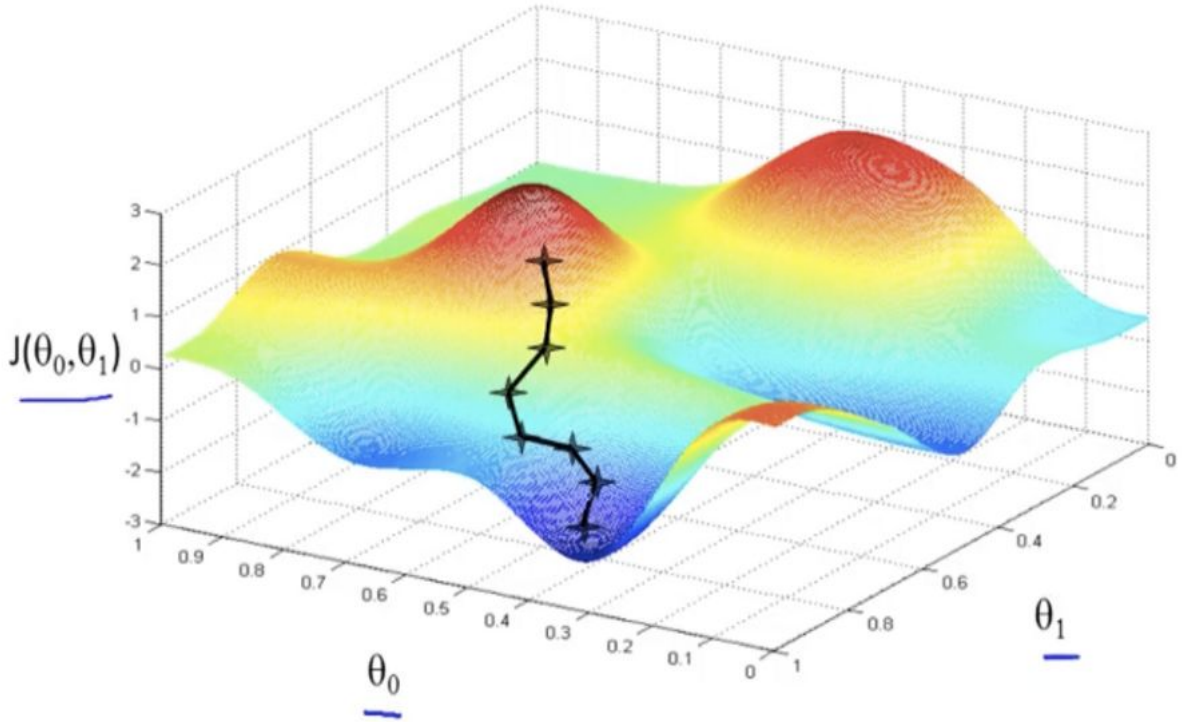
https://pytorch.org/

Gradient descent

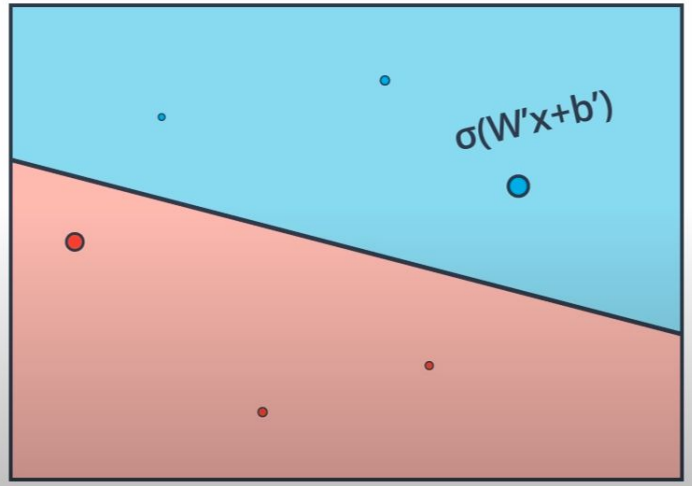


https://github.com/fastai/fastbook/blob/master/04_mnist_basics.ipynb

Stochastic gradient descent



Gradient Descent Algorithm



1. Start with random weights:

   $w_1, \ldots, w_n, b$

2. For every point $(x_1, \ldots, x_n)$:

   2.1. For i =1 . . . n

      2.1.1. Update $w_i' \longleftarrow w_i - \alpha(\hat{y}-y)x_i$

      2.1.2. Update $b' \longleftarrow b - \alpha(\hat{y}-y)$

3. Repeat until error is small

Implementing gradient descent

Scalar to Scalar

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If x changes by a small amount, how much will y change?

Vector to Scalar

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

Derivative is **Gradient**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x}\right)_n = \frac{\partial y}{\partial x_n}$$

For each element of x, if it changes by a small amount then how much will y change?

Vector to Vector

$$x \in \mathbb{R}^N, y \in \mathbb{R}^M$$

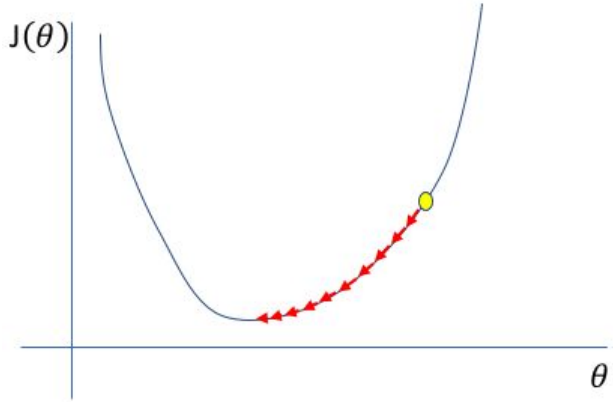
Derivative is **Jacobian**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^{N \times M} \quad \left(\frac{\partial y}{\partial x}\right)_{n,m} = \frac{\partial y_m}{\partial x_n}$$

For each element of x, if it changes by a small amount then how much will each element of y change?

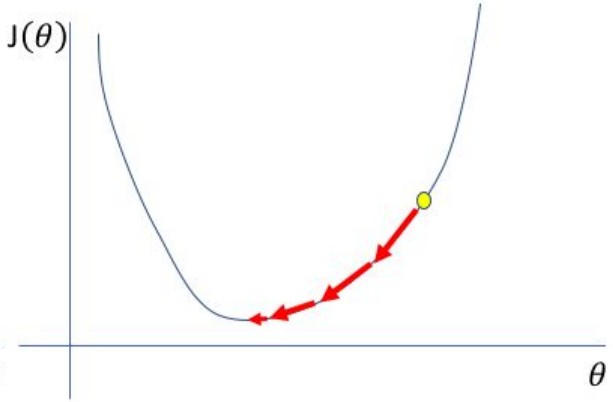
The importance of the learning rate

Too low



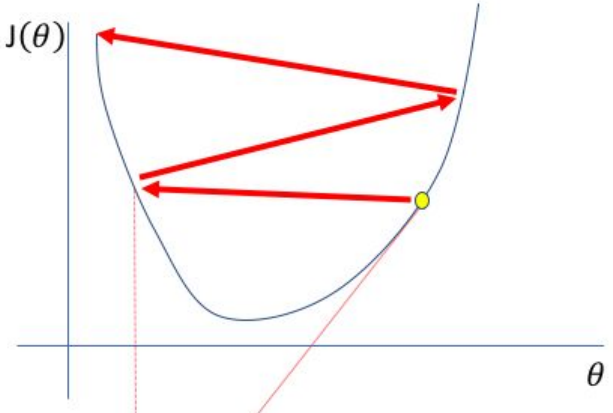
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high

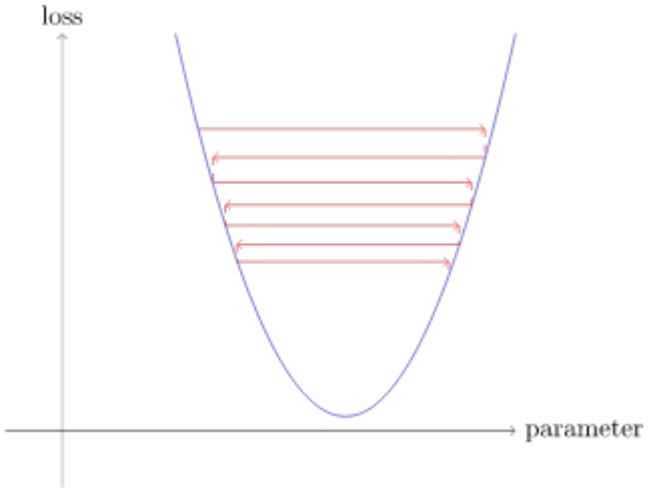


Too large of a learning rate causes drastic updates which lead to divergent behaviors

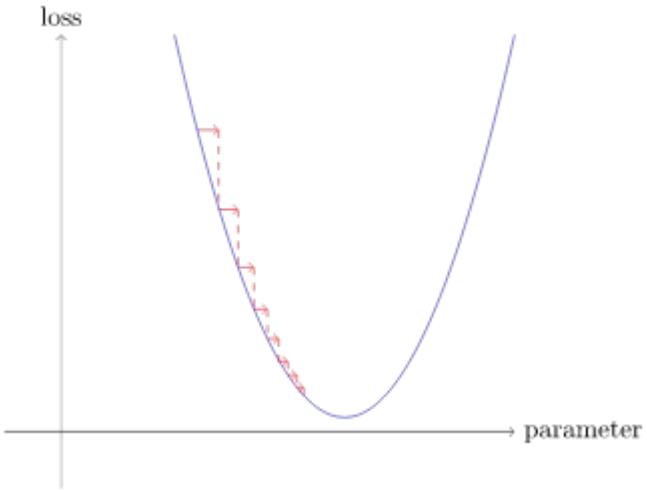
The importance of the learning rate

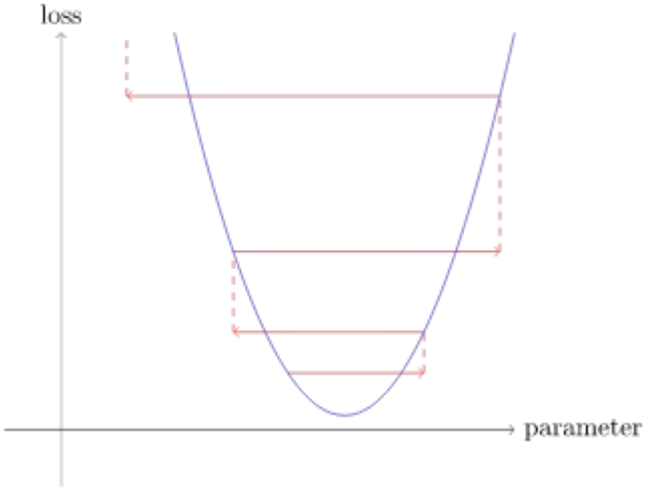
$$\Delta w_{ij} = (\eta * \frac{\partial E}{\partial w_{ij}})$$

Choosing a learning rate



Choosing a learning rate





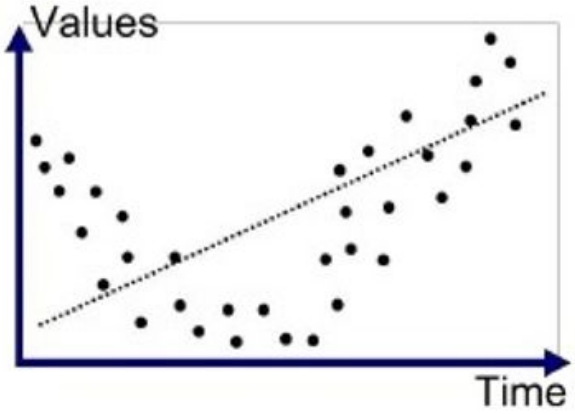
Weight decay aka lx-regularization

L1 ERROR FUNCTION $= -\frac{1}{m}\sum_{i=1}^{m}(1-y_i)ln(1-\hat{y}_i)+y_i ln(\hat{y}_i)+\lambda(|w_1|+...+|w_n|)$

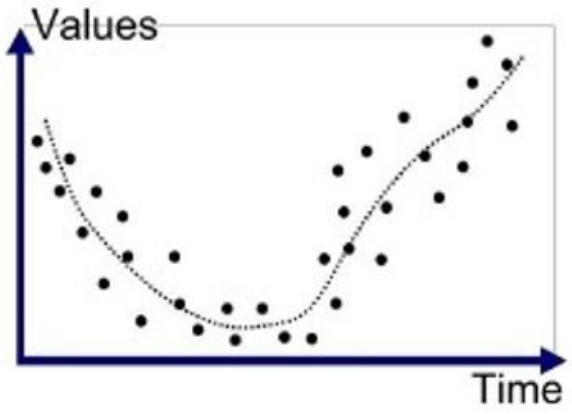
L2 ERROR FUNCTION $= -\frac{1}{m}\sum_{i=1}^{m}(1-y_i)ln(1-\hat{y}_i)+y_i ln(\hat{y}_i)+\lambda({w_1}^2+...+{w_n}^2)$

L1 and L2 regularization

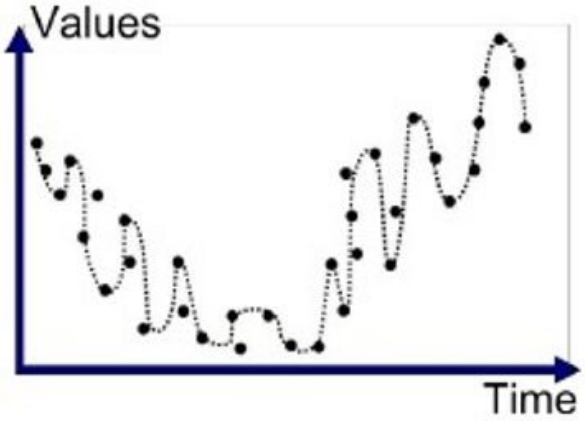
The bias vs variance tradeoff



Underfitted

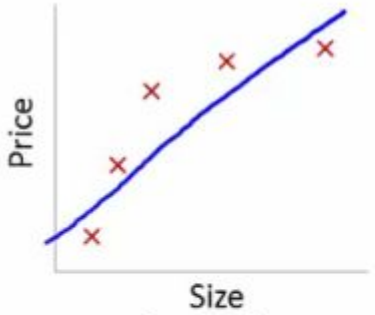


Good Fit/Robust



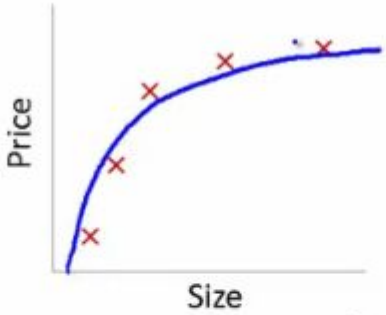
Overfitted

The bias vs variance tradeoff



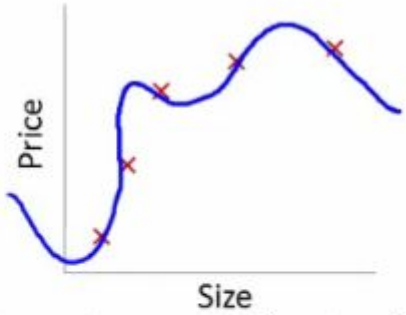
$\theta_0 + \theta_1 x$

High bias
(underfit)



$\theta_0 + \theta_1 x + \theta_2 x^2$

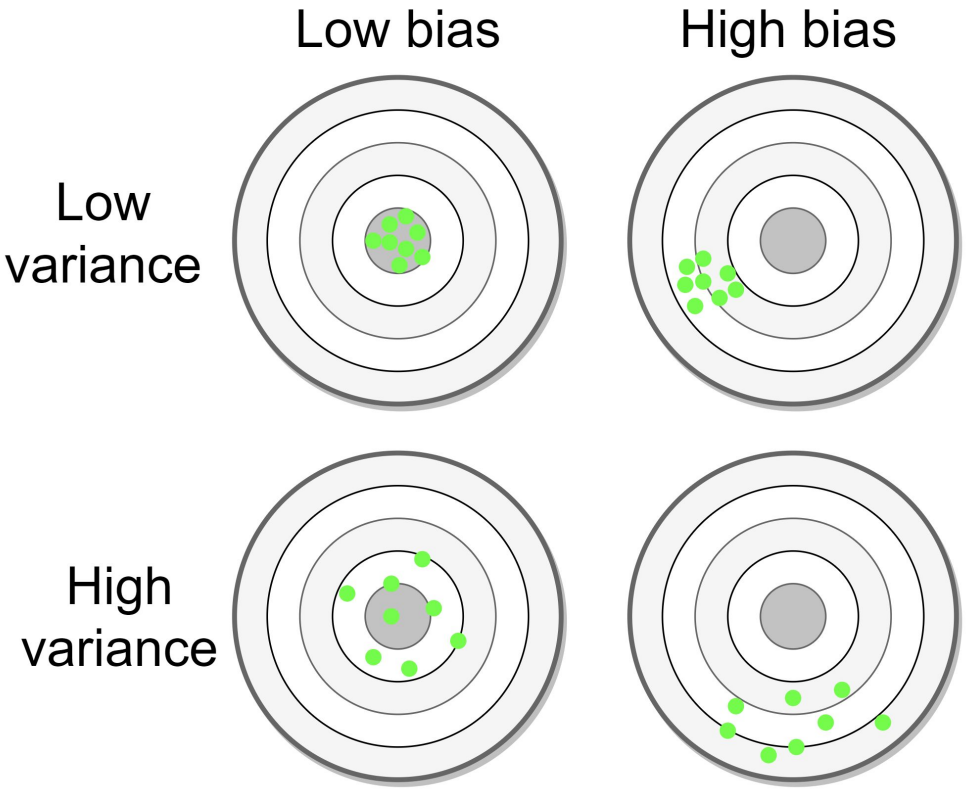
"Just right"



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

High variance
(overfit)

The bias vs variance tradeoff



The Softmax function

The softmax function takes as input a vector *z* of *K* real numbers, and normalizes it into a probability distribution consisting of *K* probabilities proportional to the exponentials of the input numbers. That is, prior to applying softmax, some vector components could be negative, or greater than one; and might not sum to 1; but after applying softmax, each component will be in the interval $(0, 1)$, and the components will add up to 1, so that they can be interpreted as probabilities. Furthermore, the larger input components will correspond to larger probabilities.

The standard (unit) softmax function $\sigma : \mathbb{R}^K \to \mathbb{R}^K$ is defined by the formula

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \text{ for } i = 1, \dots, K \text{ and } \mathbf{z} = (z_1, \dots, z_K) \in \mathbb{R}^K$$

https://en.wikipedia.org/wiki/Softmax_function

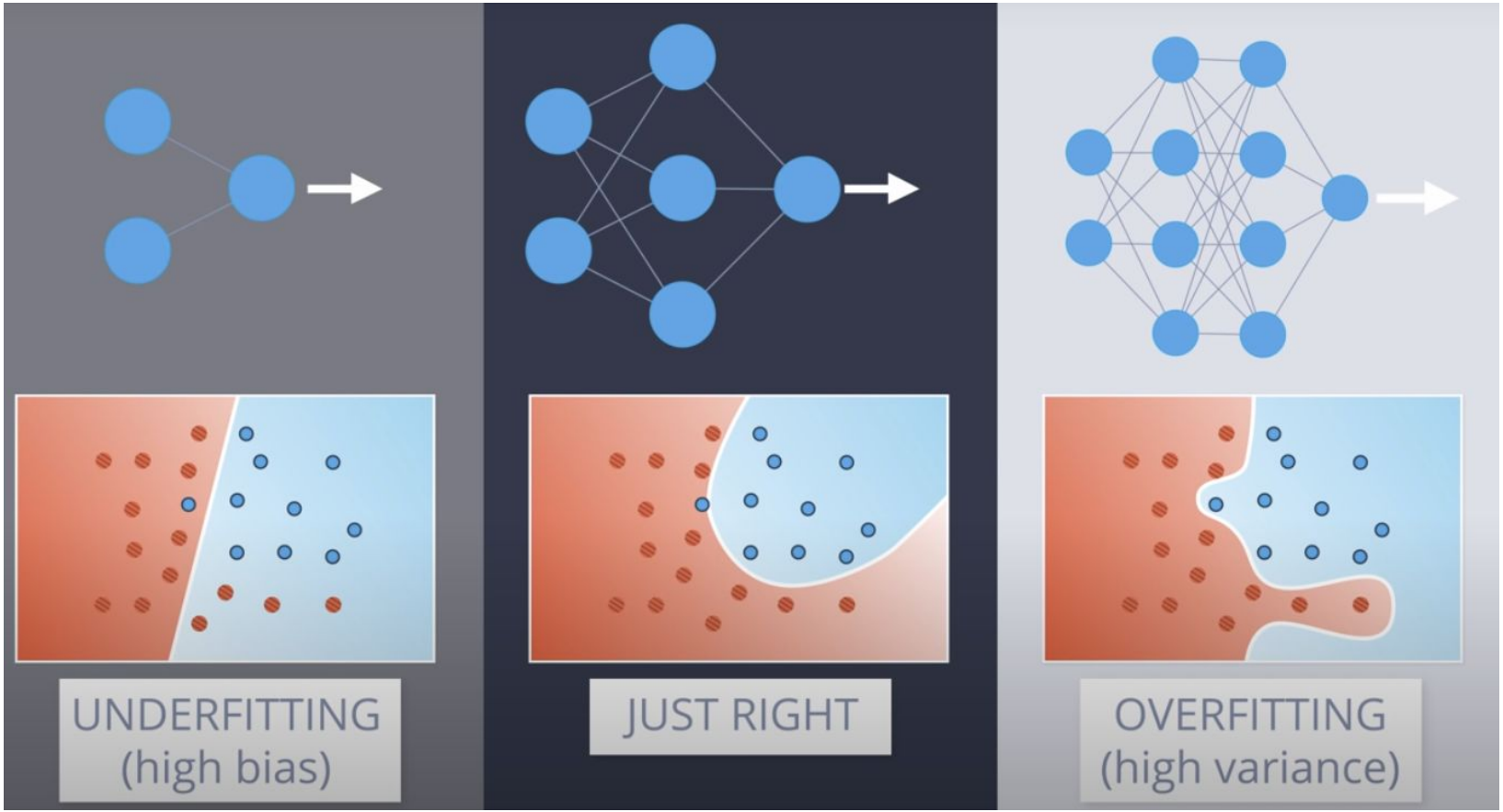
The softmax activation function in Python

Performance metrics vs loss functions

A model performance metric measures the quality of predictions in the validation or test sets. Accuracy, precision, recall, specificity, fbeta, f1, and ROC are performance metrics, not loss functions

The default classification error metric in fastai is error_rate = 1 - accuracy

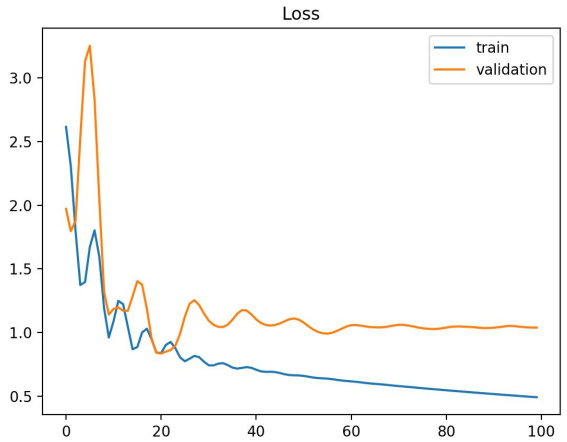
The loss function is used to adjust the weights of the model (through its gradient). The error metric is a report of performance and it can be opposite to the loss function

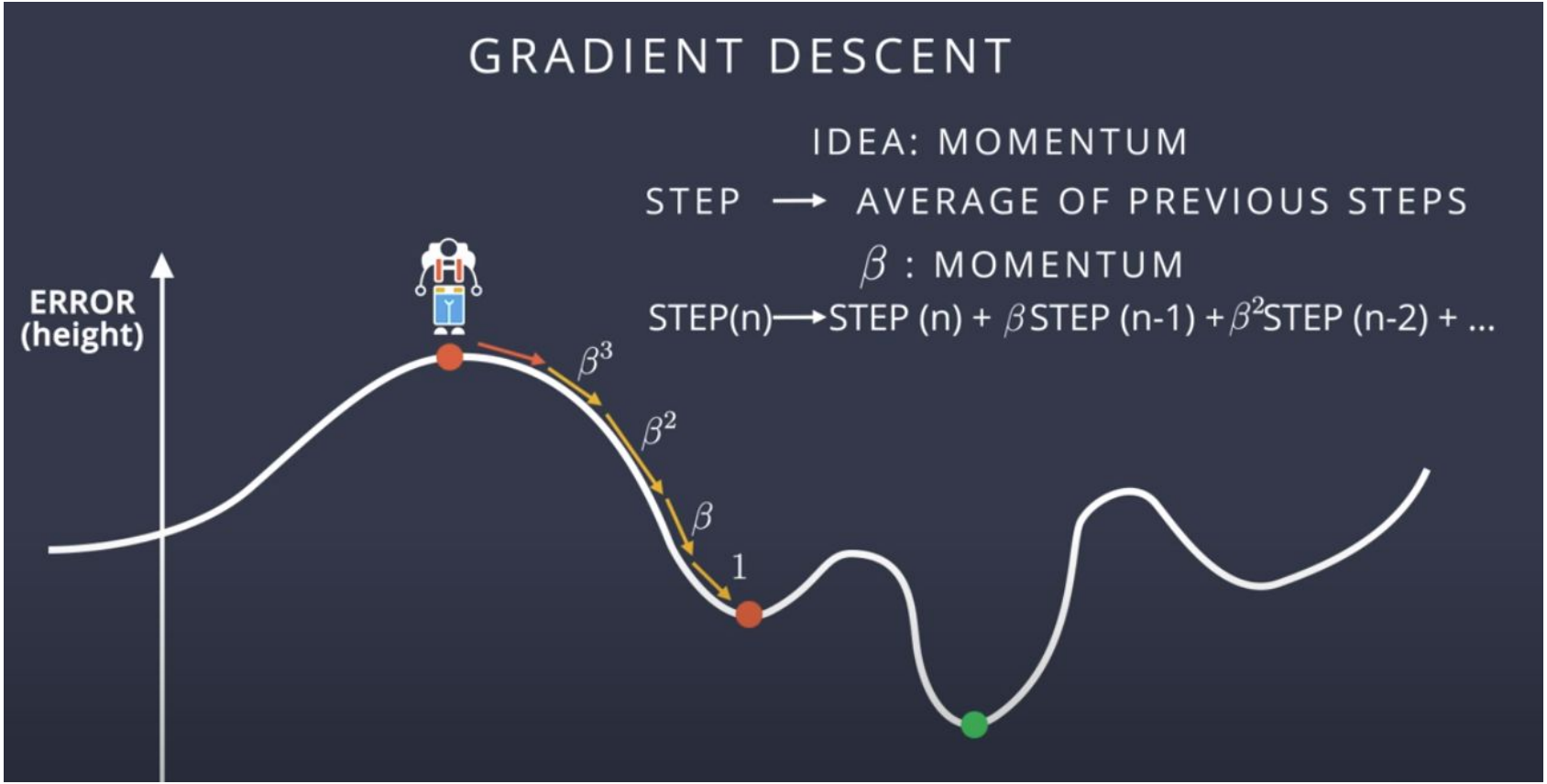


Understanding the bias vs variance tradeoff in neural network architectures

How long should we train the network?

- Until it overfits! (Training loss goes down while validation loss goes up)
- This general principle applies to **all** hyperparameters related to model complexity





The intuition of momentum

Exploring momentum's beta (notebook)

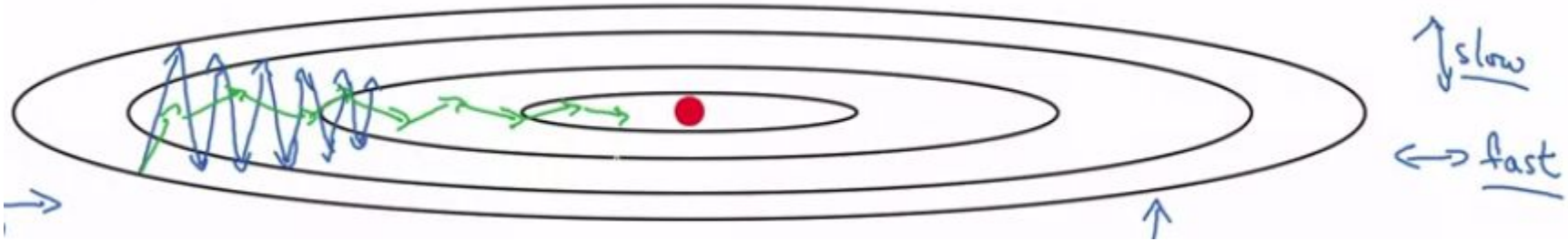
Understanding momentum

$$\Delta w_{ij} = (\eta * \frac{\partial E}{\partial w_{ij}}) + (\gamma * \Delta w_{ij}^{t-1})$$

momentum factor

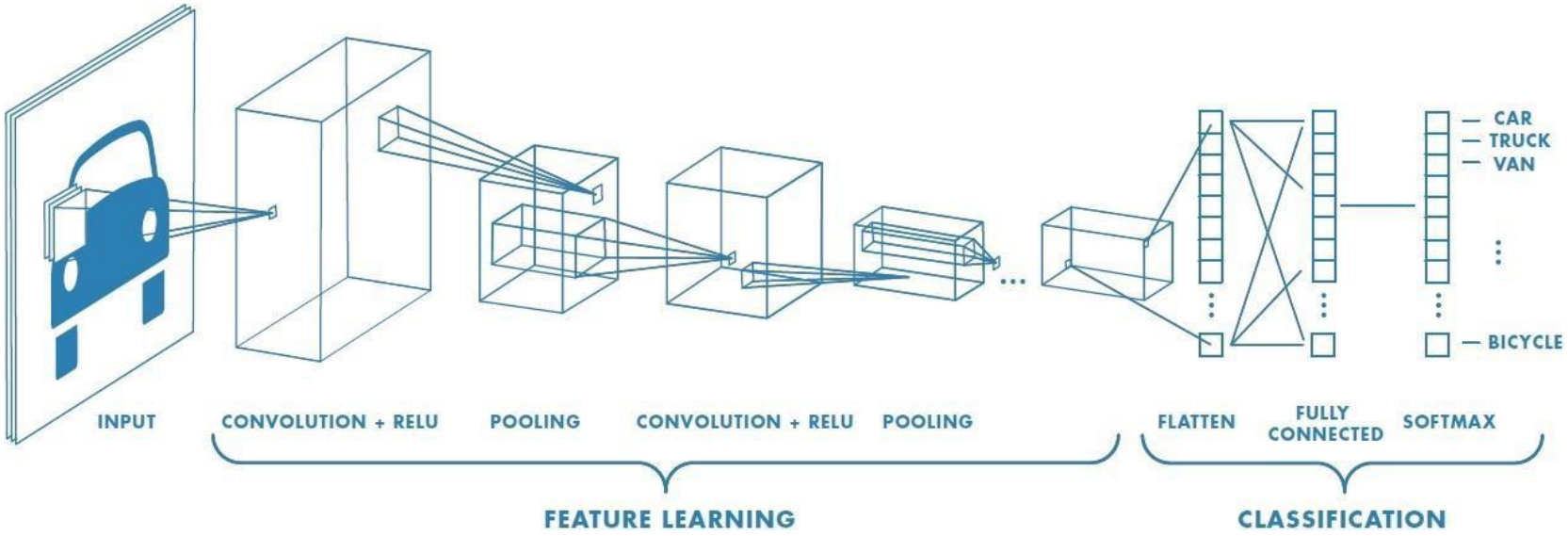
weight increment, previous iteration

The ADAM (Adaptive Moments) optimizer



Exponentially moving averages on both the mean and variance (“moments”) of the gradient history are used to smoothen the trajectory of the gradient descent.
We track the gradient history and use it to modify the learning rate. We use bigger learning rates with gradients that have less variance (the opposite is also true).
This optimization algorithm trains faster than SGD in many settings, but results vary across different datasets.

Convolutional Neural Networks



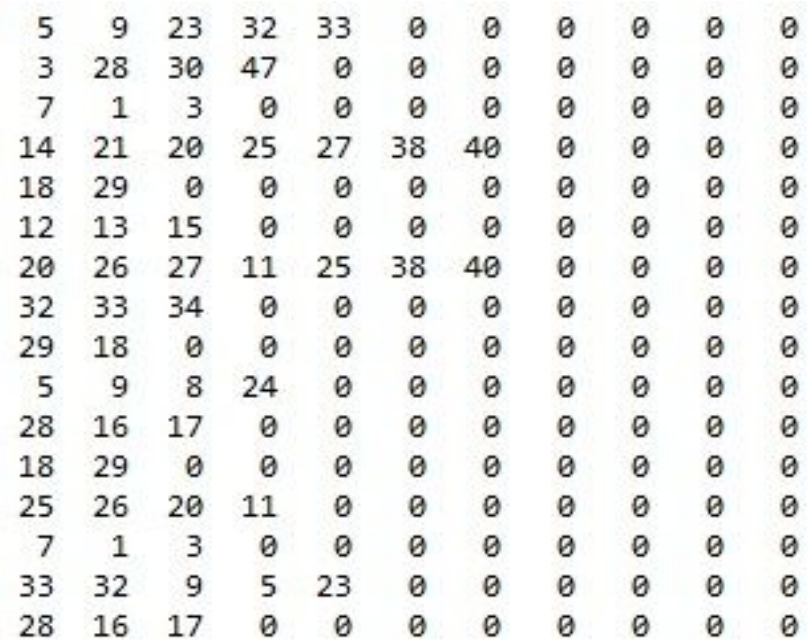
[CNNs vs multilayer perceptrons]

Image Classification

what we see

- Outdoor
- Building
- Snow

Image Classification



5	9	23	32	33	0	0	0	0	0	0
3	28	30	47	0	0	0	0	0	0	0
7	1	3	0	0	0	0	0	0	0	0
14	21	20	25	27	38	40	0	0	0	0
18	29	0	0	0	0	0	0	0	0	0
12	13	15	0	0	0	0	0	0	0	0
20	26	27	11	25	38	40	0	0	0	0
32	33	34	0	0	0	0	0	0	0	0
29	18	0	0	0	0	0	0	0	0	0
5	9	8	24	0	0	0	0	0	0	0
28	16	17	0	0	0	0	0	0	0	0
18	29	0	0	0	0	0	0	0	0	0
25	26	20	11	0	0	0	0	0	0	0
7	1	3	0	0	0	0	0	0	0	0
33	32	9	5	23	0	0	0	0	0	0
28	16	17	0	0	0	0	0	0	0	0

what the computer “sees”

- Outdoor
- Building
- Snow? *Do we care about snow?*
  - Enough of these images need to be shown to the algorithm

Defining image taxonomies

- Bedroom
- Terrace
- Desk
- Vegetation

- Do we have enough images with these things?

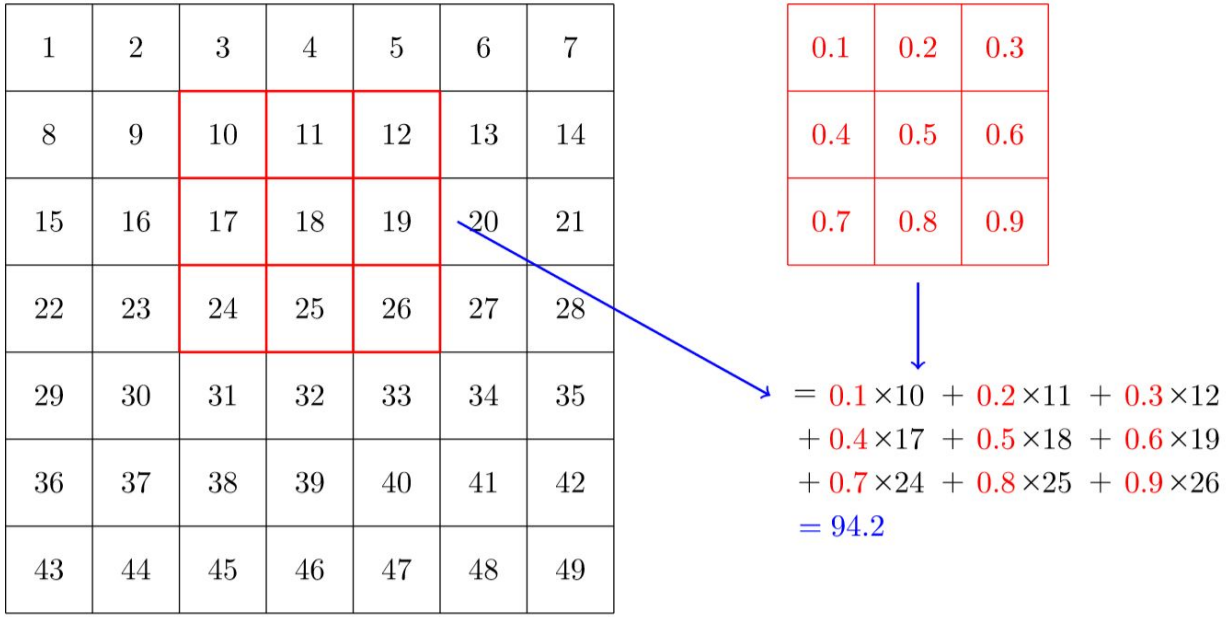
Labels for hard cases

- Should we have added ‘neon lights’ to our taxonomy?
- How many of these things we have?
- Should we invest effort on this?
- **Stacks of binary models** are appropriate for this problem

Hot Dog not a Hot Dog

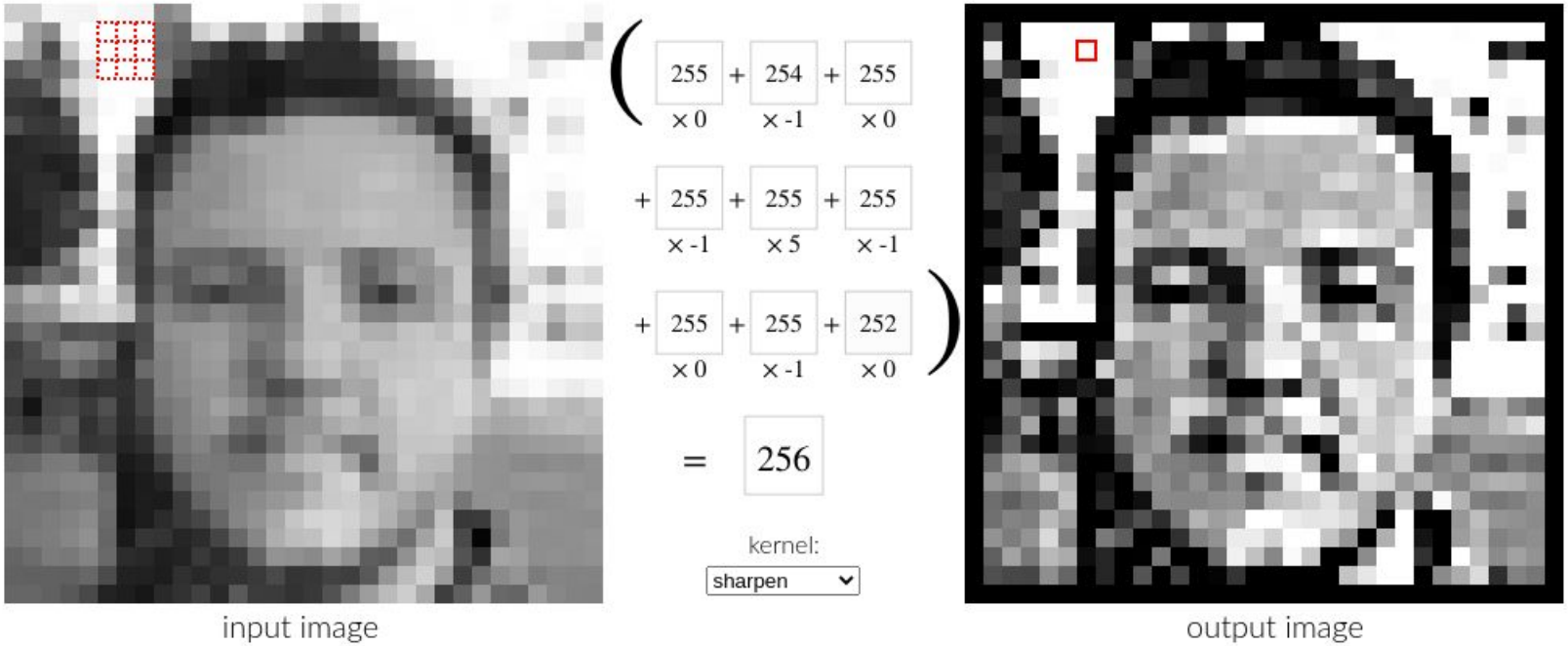
https://www.youtube.com/watch?v=vIci3C4JkL0

What is a convolution filter?



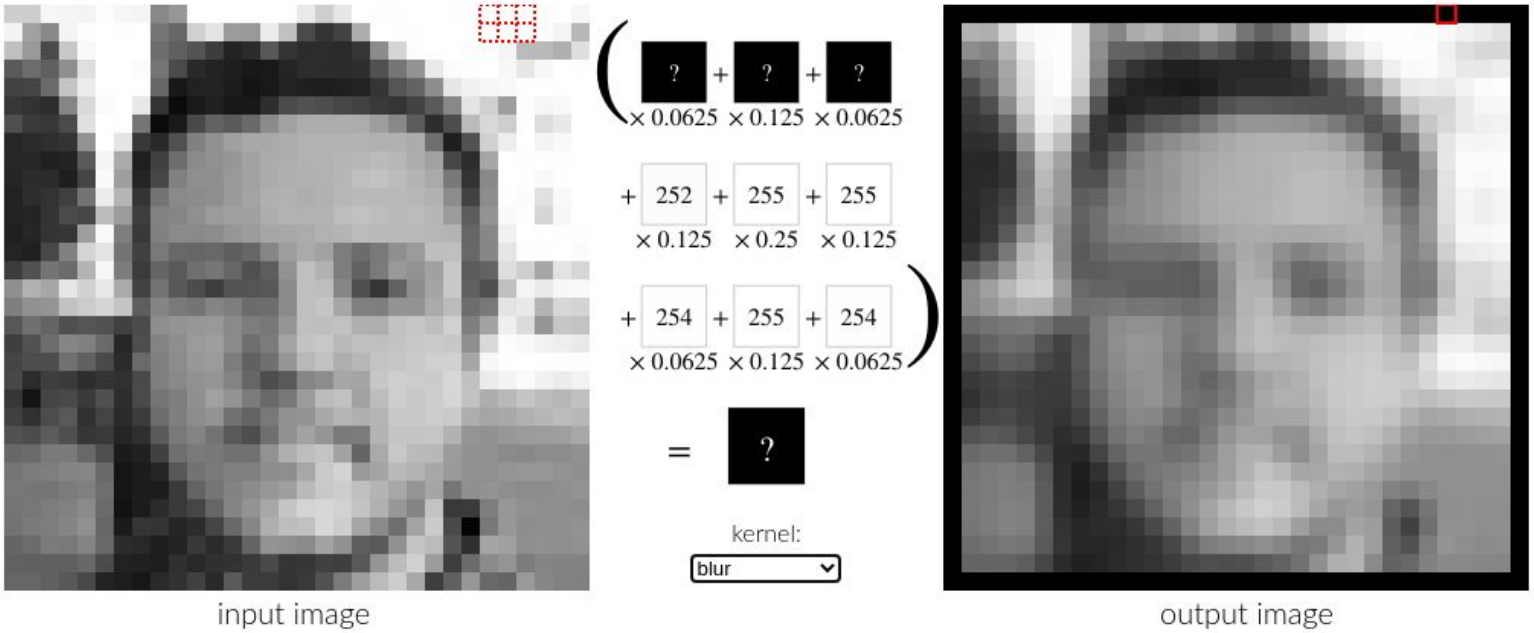
https://github.com/fastai/fastbook/blob/master/13_convolutions.ipynb

What is a convolution filter?



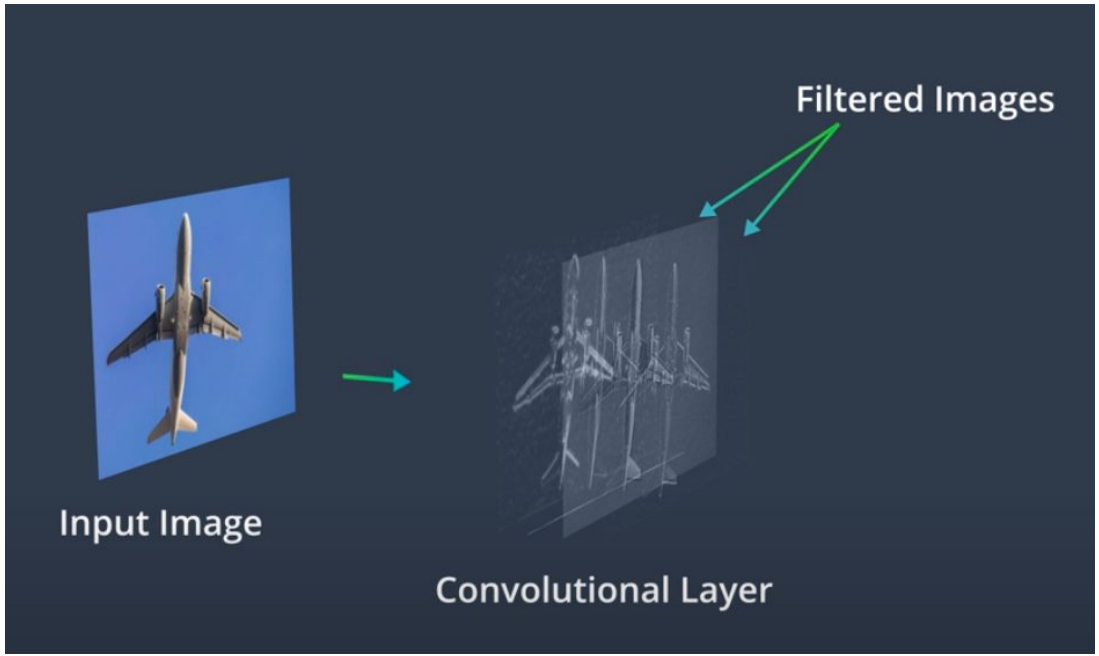
https://setosa.io/ev/image-kernels/

What is a convolution filter?



https://setosa.io/ev/image-kernels/

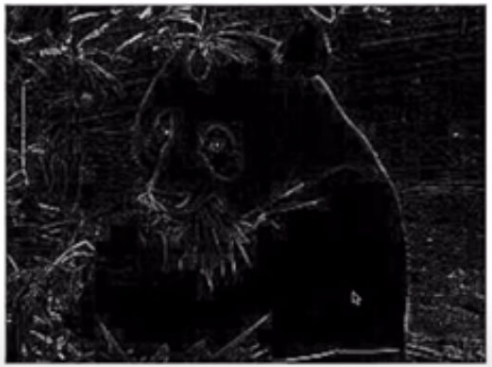
What is a convolution filter?



Understanding the output of convolutional layers

What is a convolution filter?

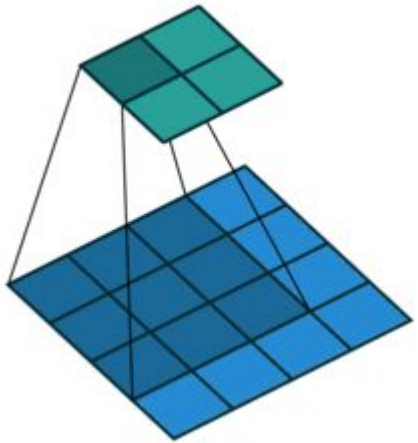
0	-1	0
-1	4	**-2**
0	-1	0



$$0 + -1 + 0 + -1 + 4 + \mathbf{-2} + 0 + -1 + 0 = \mathbf{-1}$$

Visualizing convolution kernels

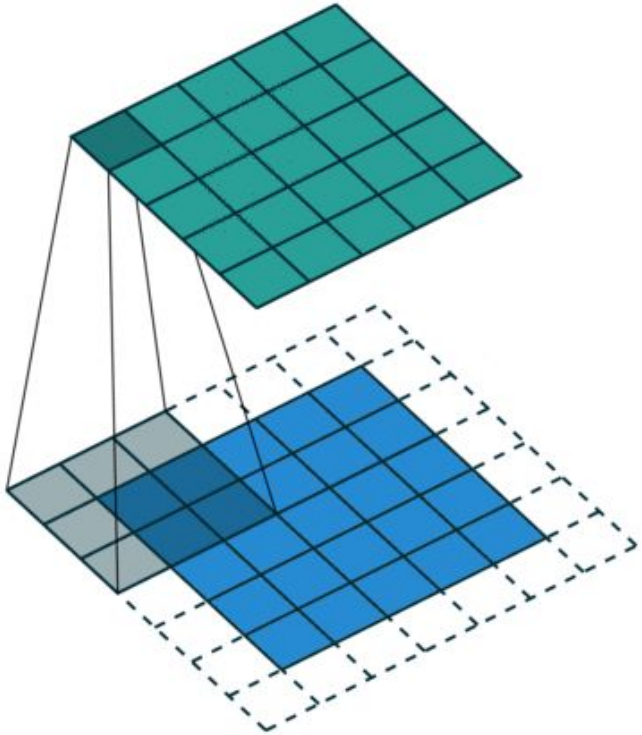
What is a convolution filter?



Convolution of 3x3 and stride = 1 without padding

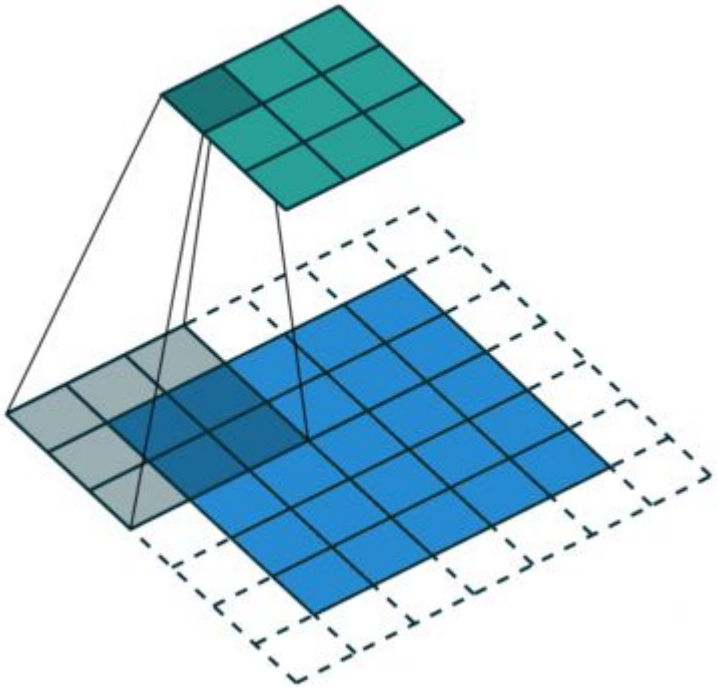
Effect: the output loses one pixel on each dimension

What is a convolution filter?



Convolution of 3x3 and stride = 1 padded with zeros
Effect: the output preserves original image size

What is a convolution filter?



Convolution of 3x3 and stride = 2 padded with zeros

Effect: the output is downsampled to about half its size

Activity: Conv2D in Pytorch (Colab notebook)

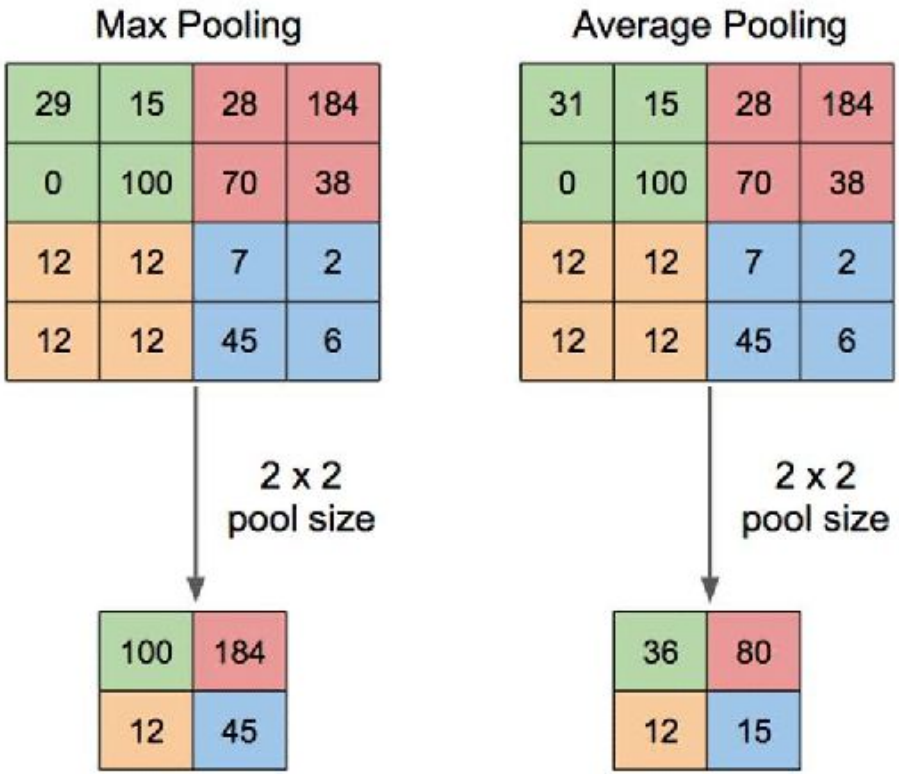
Experiment: finding edges with convolutions

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

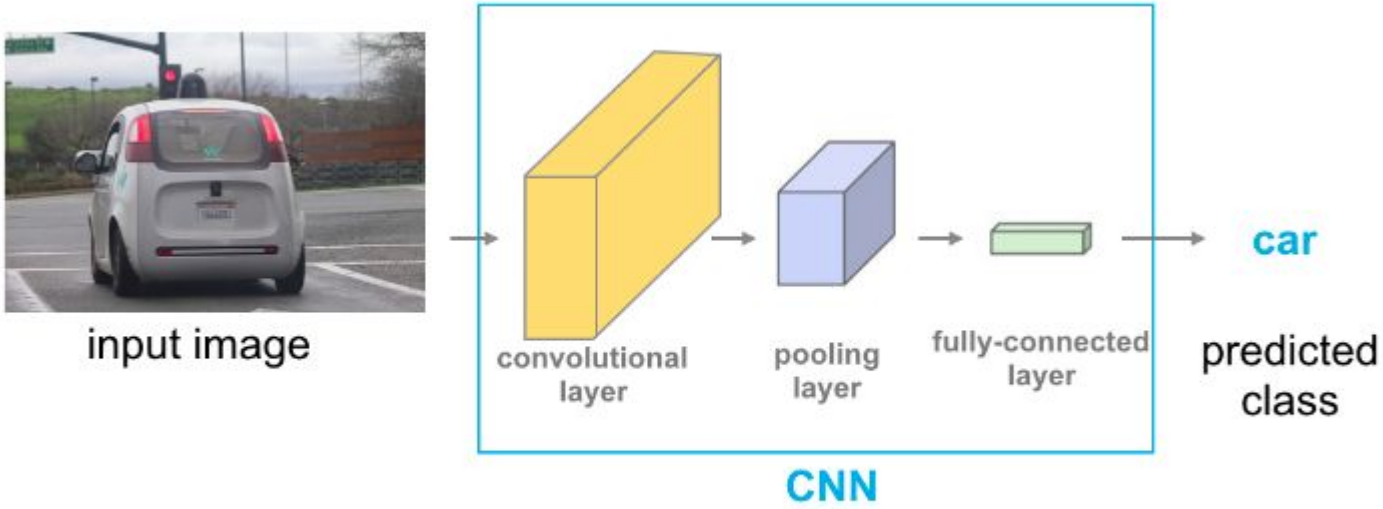
$$S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

Finding edges with convolution kernels

Pooling

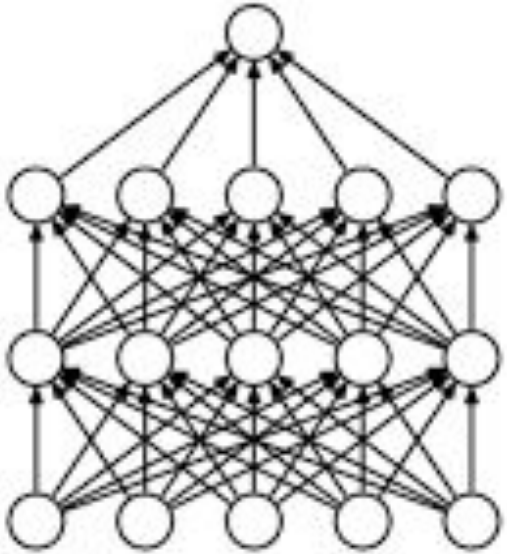

Max Pooling
29 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
100 184
12 45
Average Pooling
31 15 28 184
0 100 70 38
12 12 7 2
12 12 45 6
2 x 2
pool size
36 80
12 15

CNN architectures

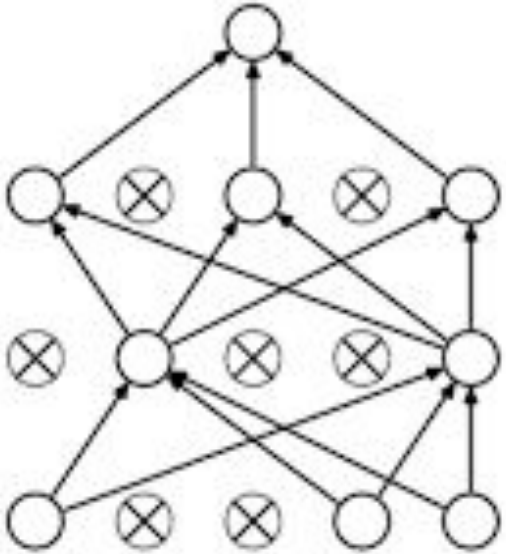


Layers in a CNN.

Dropout regularization

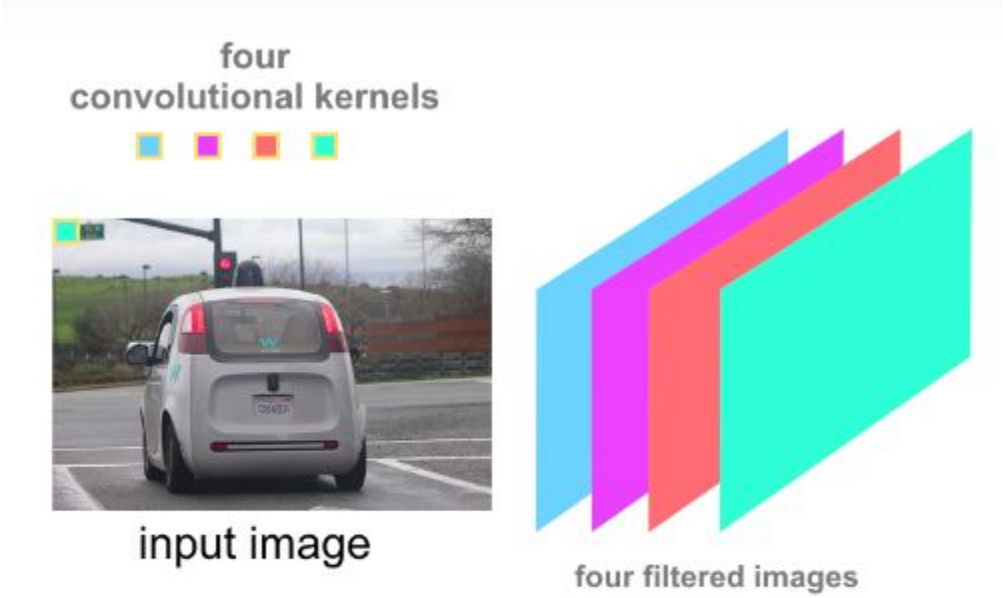


(a) Standard Neural Net



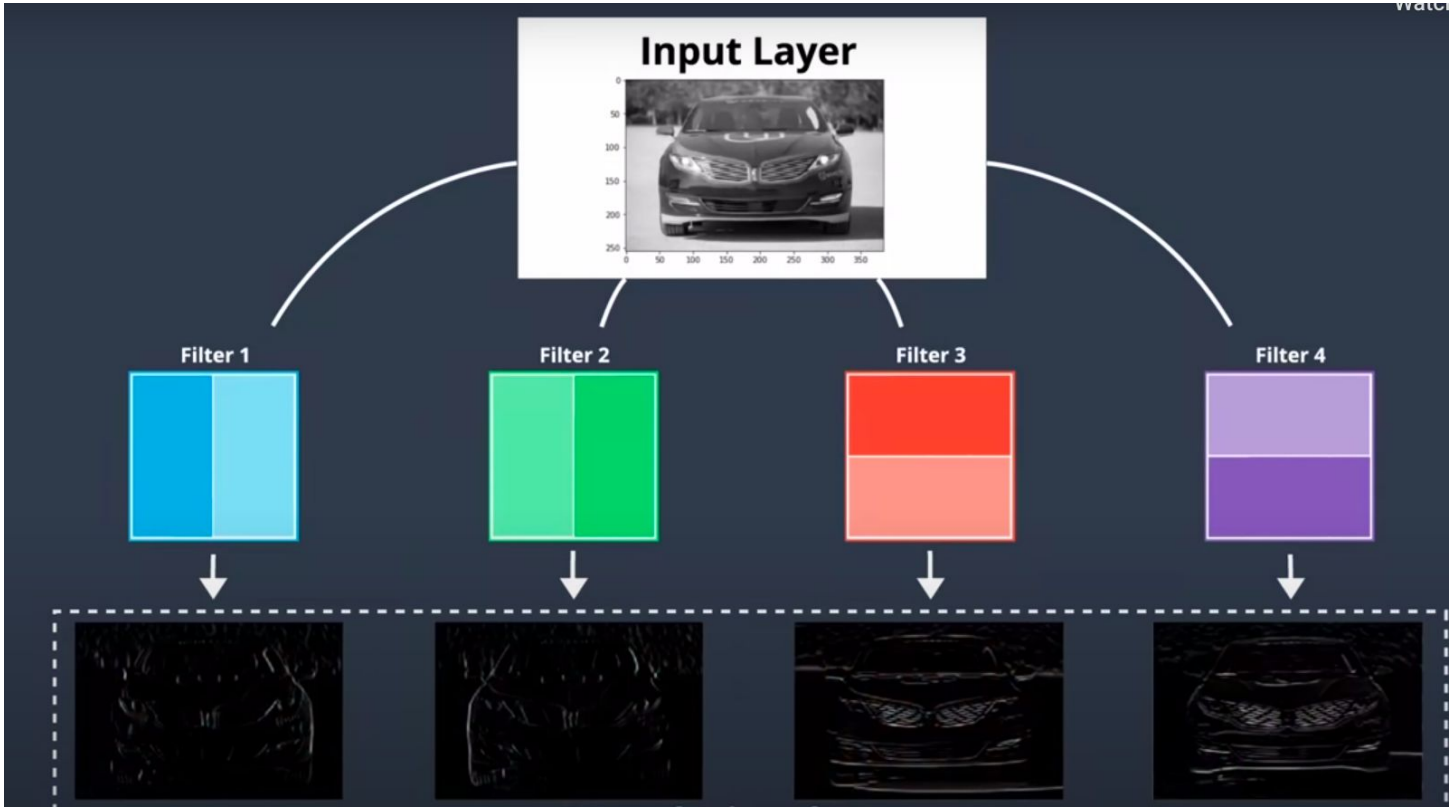
(b) After applying dropout.

CNN architectures

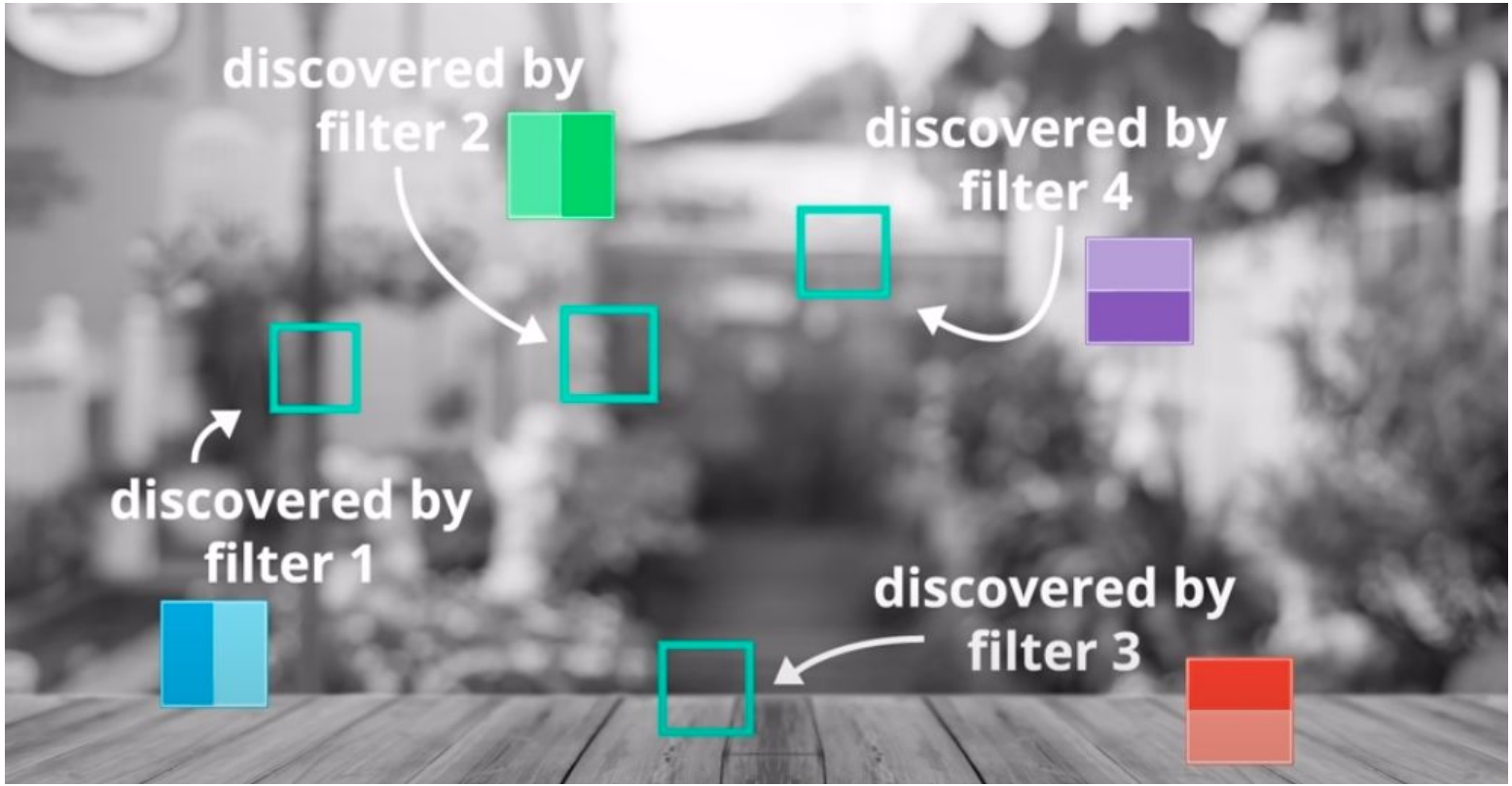


4 kernels = 4 filtered images.

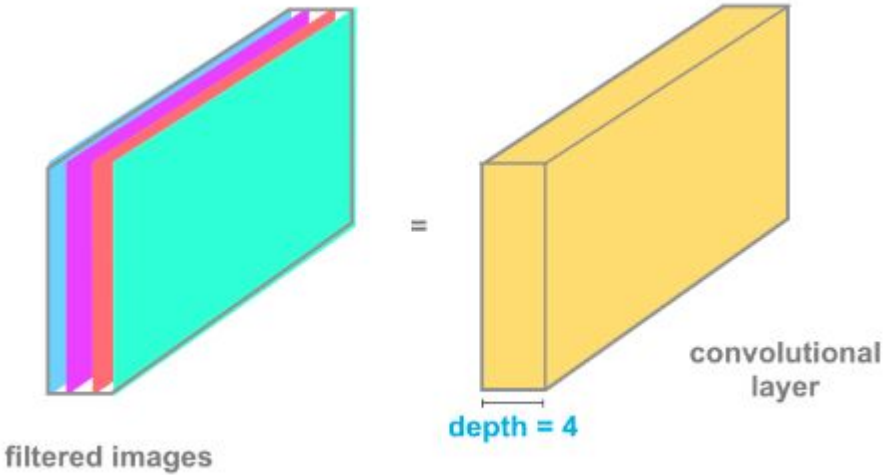
CNN architectures



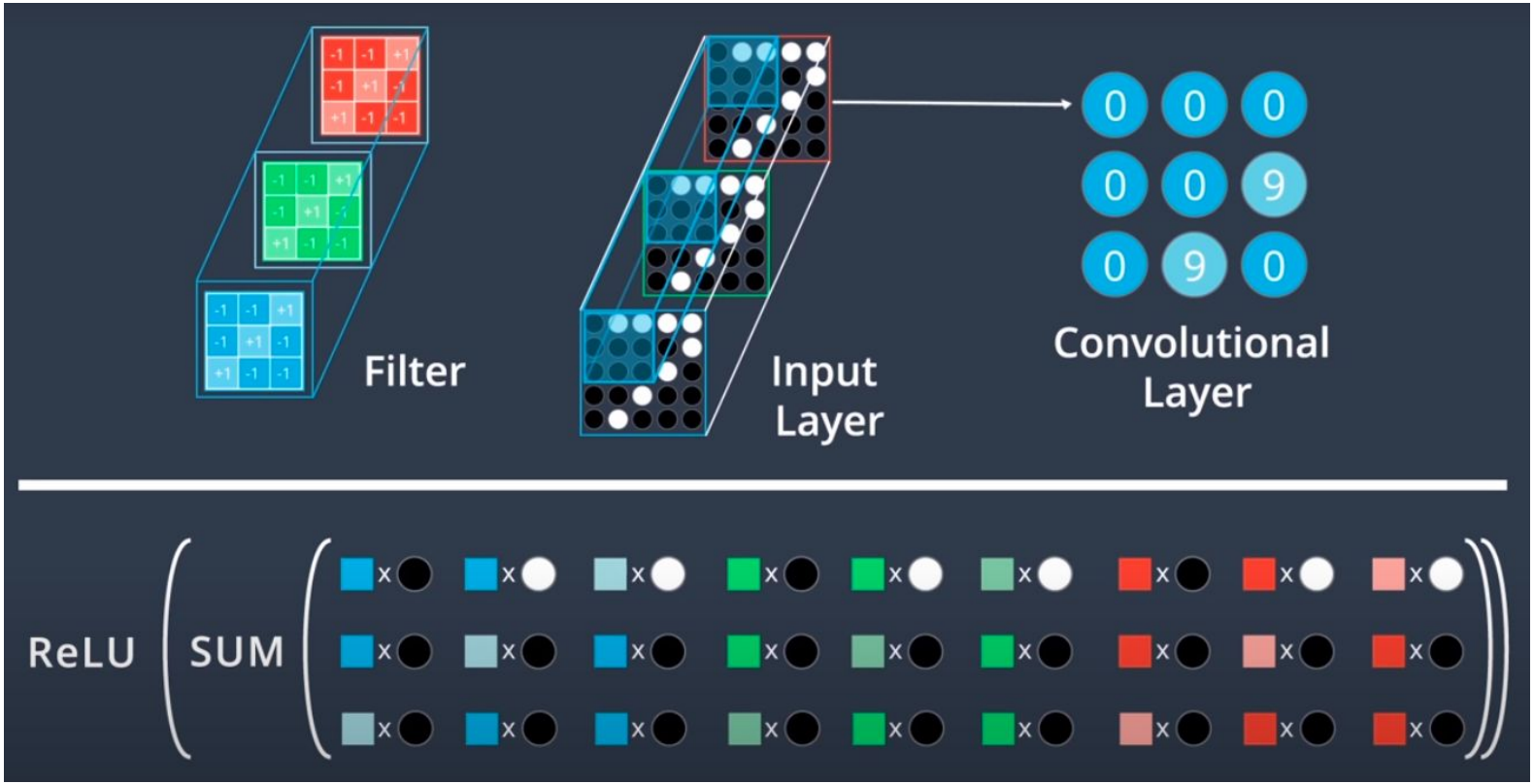
CNN architectures

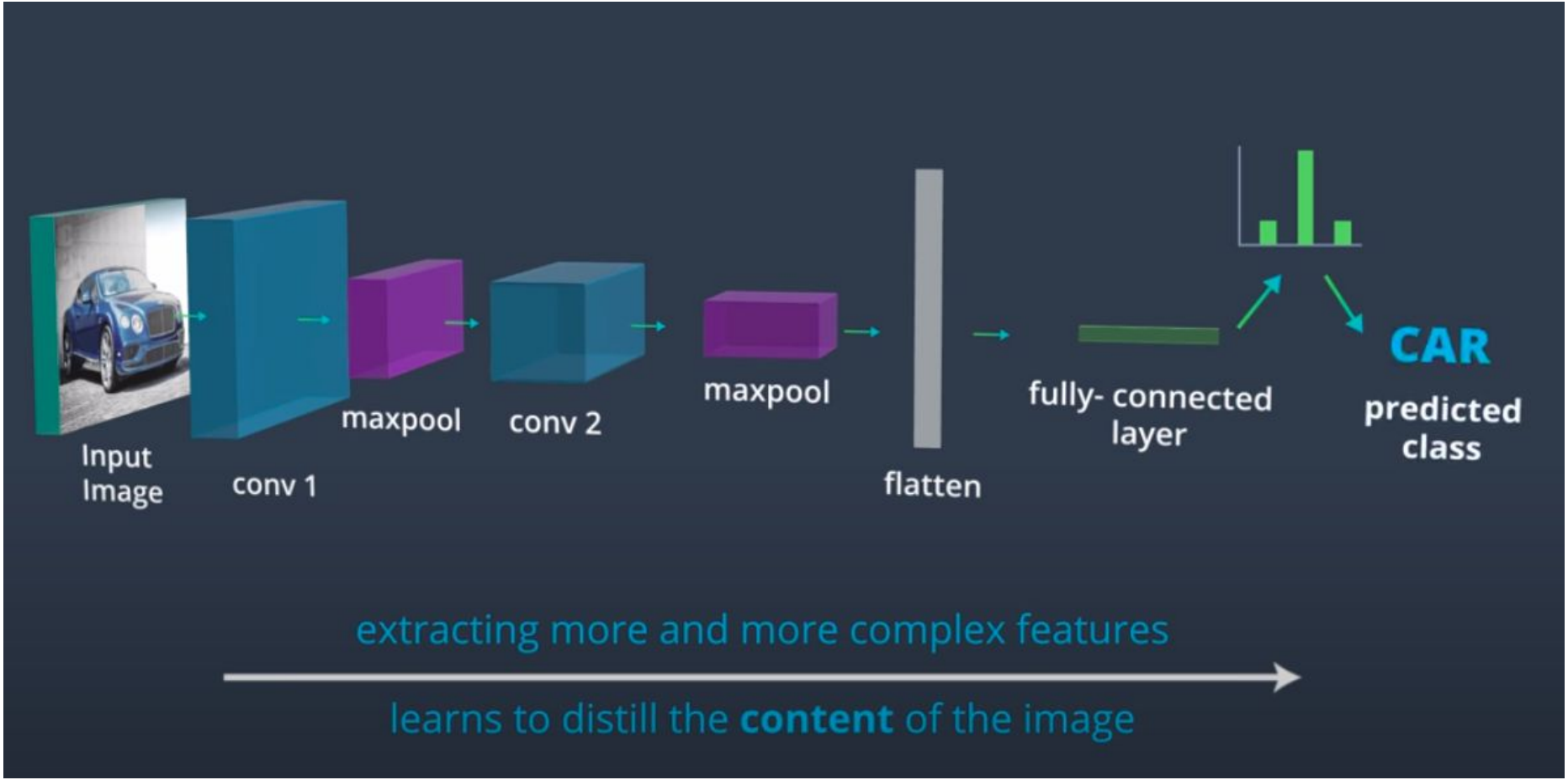


CNN architectures



CNN architectures




Input Image
conv 1
maxpool
conv 2
maxpool
flatten
fully- connected layer
CAR
predicted class
extracting more and more complex features
learns to distill the **content** of the image

Normalizing images

```
transforms = torch.nn.Sequential(
    transforms.CenterCrop(10),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225)),
)
scripted_transforms = torch.jit.script(transforms)
```

https://pytorch.org/docs/stable/torchvision/transforms.html#transforms-on-torch-tensor

Images need to be resized for the GPU

zeros

border

reflection

https://docs.fast.ai/vision.augment

Data augmentation

https://docs.fast.ai/vision.augment

Augmentations composition in PyTorch

```
train_transforms = transforms.Compose([transforms.RandomRotation(30),
                                       transforms.RandomResizedCrop(100),
                                       transforms.RandomHorizontalFlip(),
                                       transforms.ToTensor(),
                                       transforms.Normalize([0.5, 0.5, 0.5],
                                                            [0.5, 0.5, 0.5])])
```

https://sparrow.dev/torchvision-transforms/

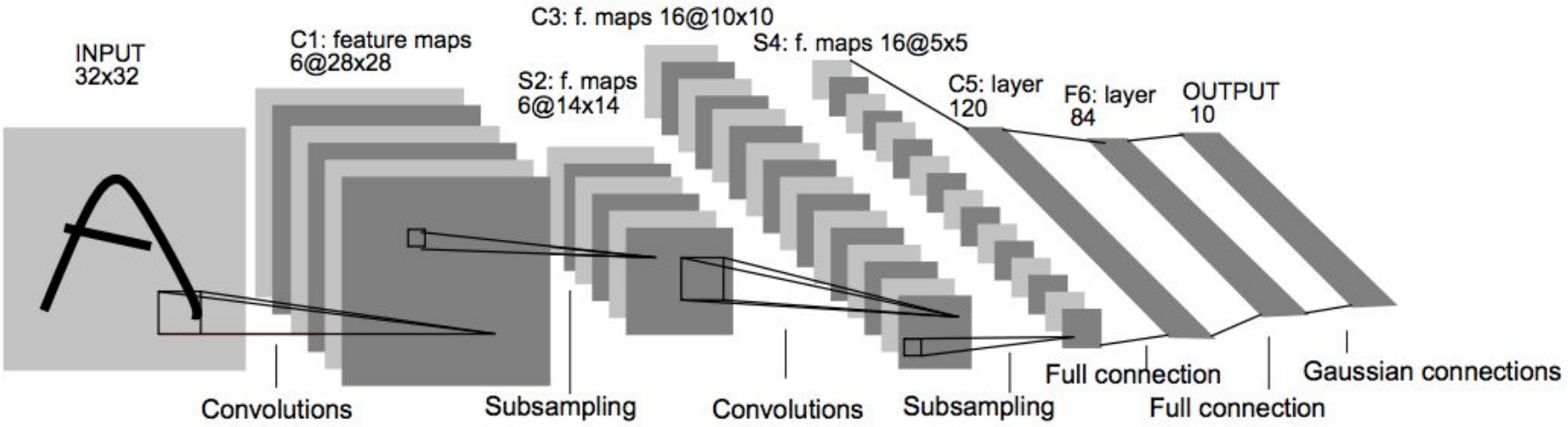
Practice: loading image data

- Notebook
- Solutions

Questions:

- Why are we using different means and standard deviations to normalize each image channel?
- How are we changing the dimensions of the input images before feeding them to the network?
- What is the effect of calling Resize before CenterCrop?

Understanding architecture diagrams: LeNet



Understanding architecture diagrams: AlexNet

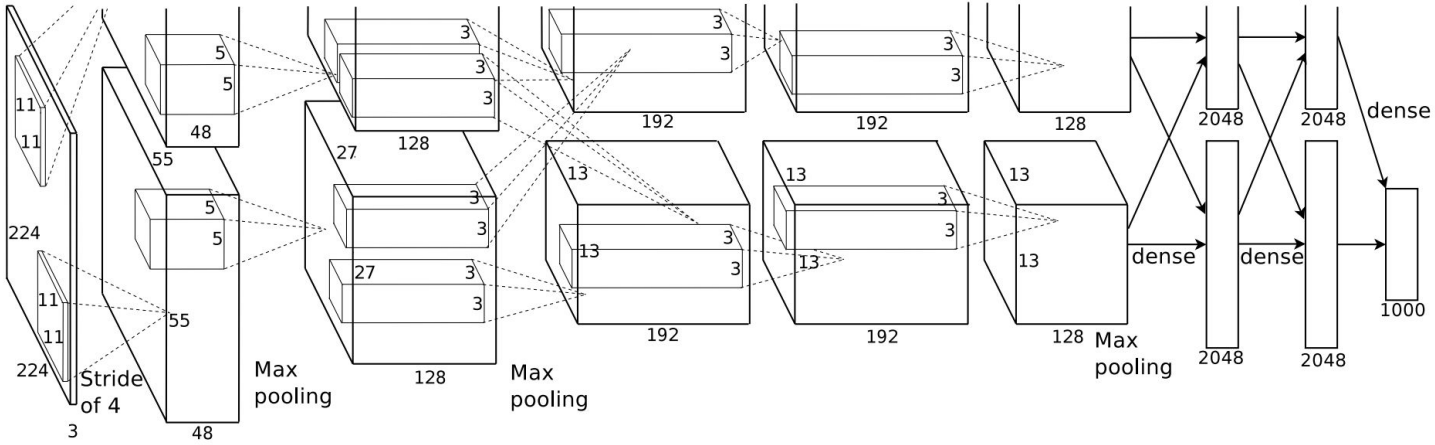


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Transfer learning

Using a pretrained model for a task different to what it was originally trained on is called *transfer learning*

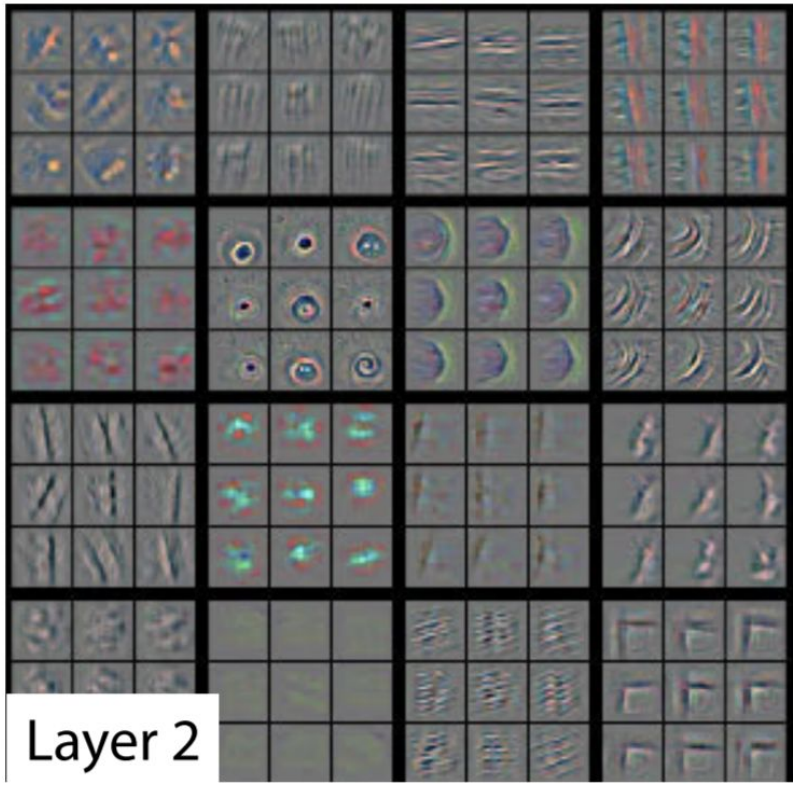
ImageNet is the single most important dataset in computer vision, it contains 1.3 million labeled images of a thousand categories and has been used to benchmark classification models during the last decade. Most popular models have been pretrained and published on ImageNet

What a pretrained network already knows

https://arxiv.org/pdf/1311.2901.pdf

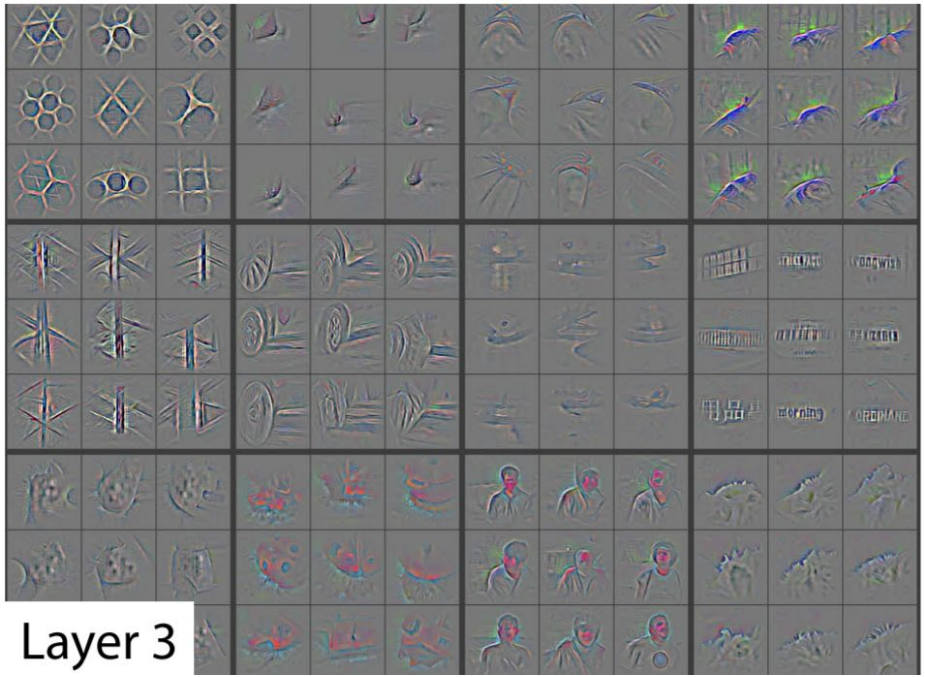
What a pretrained network already knows

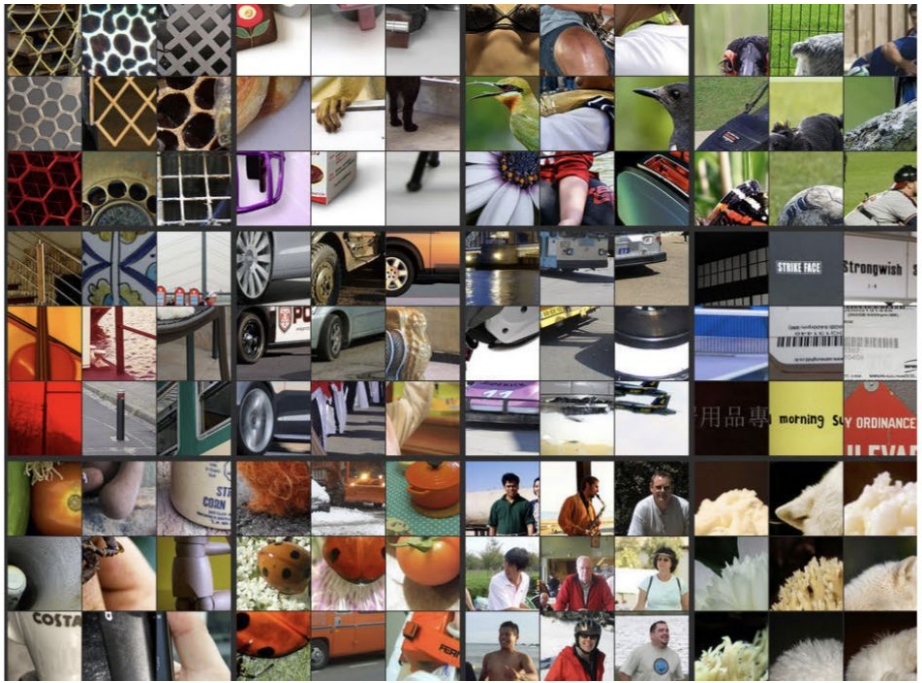
Layer 1



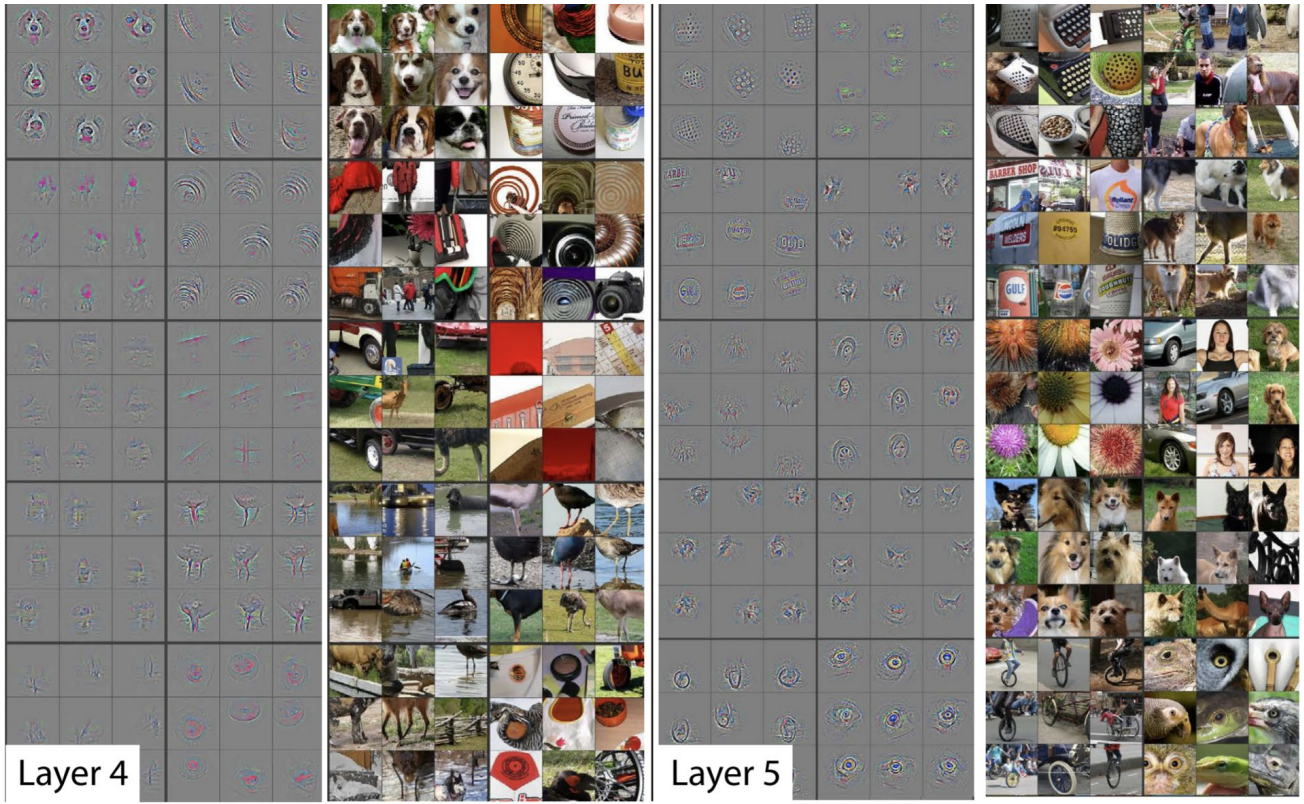
Layer 2

What a pretrained network already knows

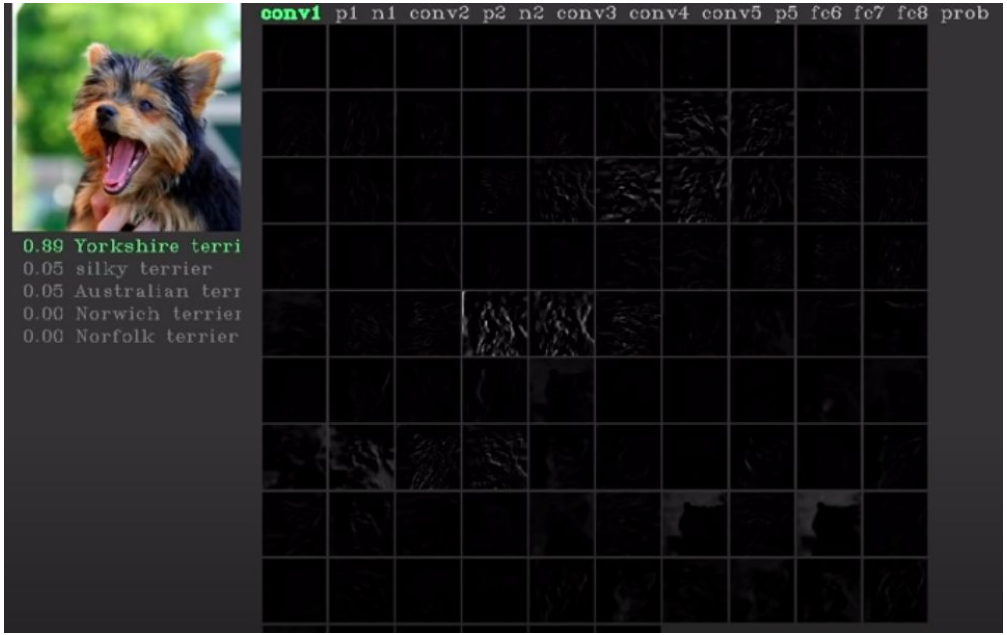




What a pretrained network already knows

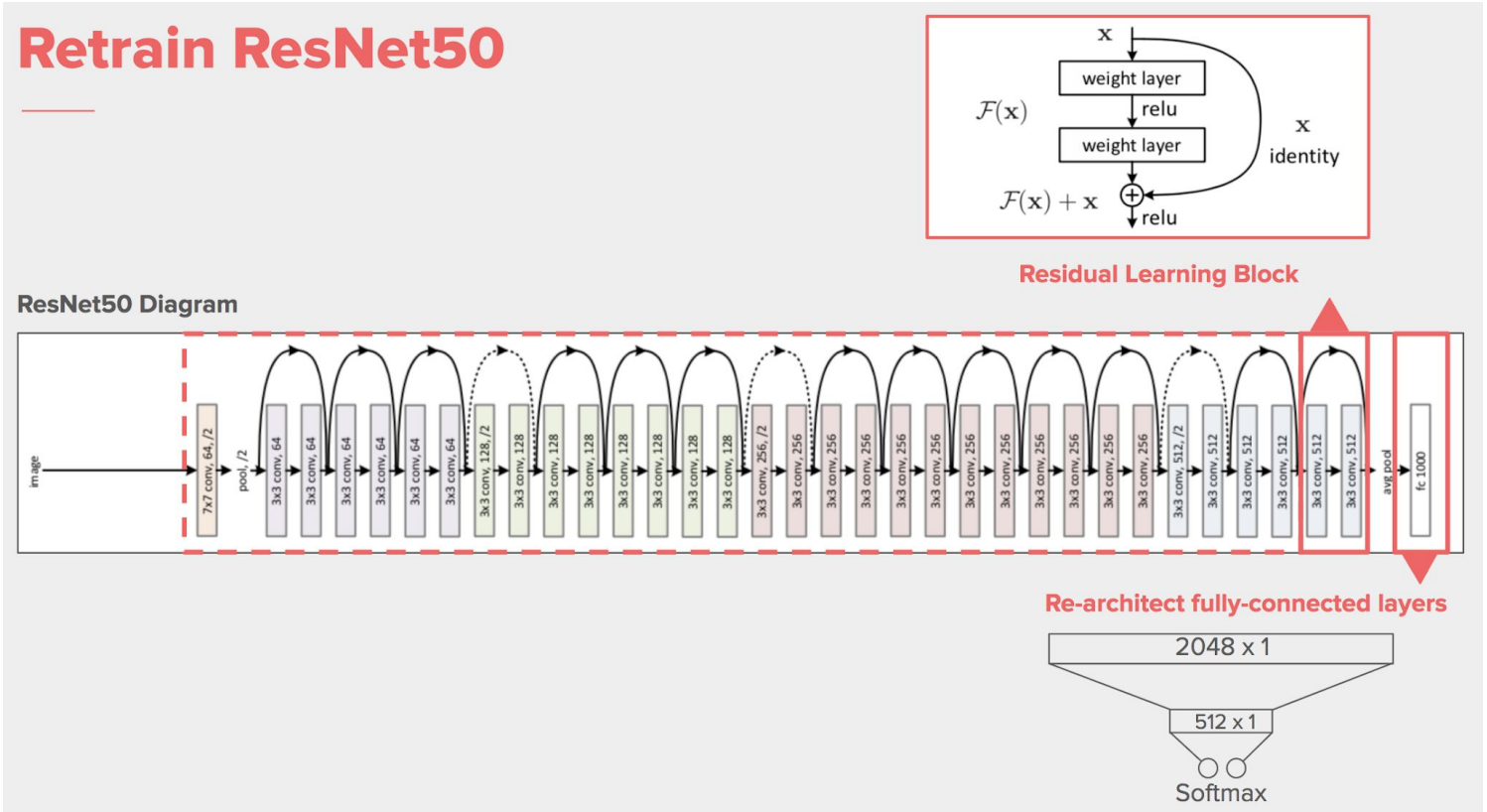


What a pretrained network already knows



[Visualizing the layers of an Alexnet trained on Imagenet]

Resnet


Retrain ResNet50
x
weight layer
relu
weight layer
$\mathcal{F}(\mathbf{x})$
x
identity
$\mathcal{F}(\mathbf{x}) + \mathbf{x}$
relu
Residual Learning Block
ResNet50 Diagram
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128
3x3 conv, 256
3x3 conv, 512
avg pool
fc 1000
Re-architect fully-connected layers
2048 x 1
512 x 1
Softmax

Densenet

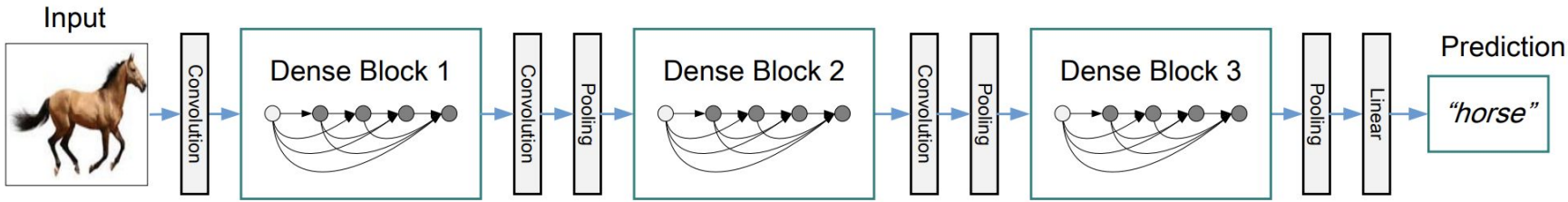
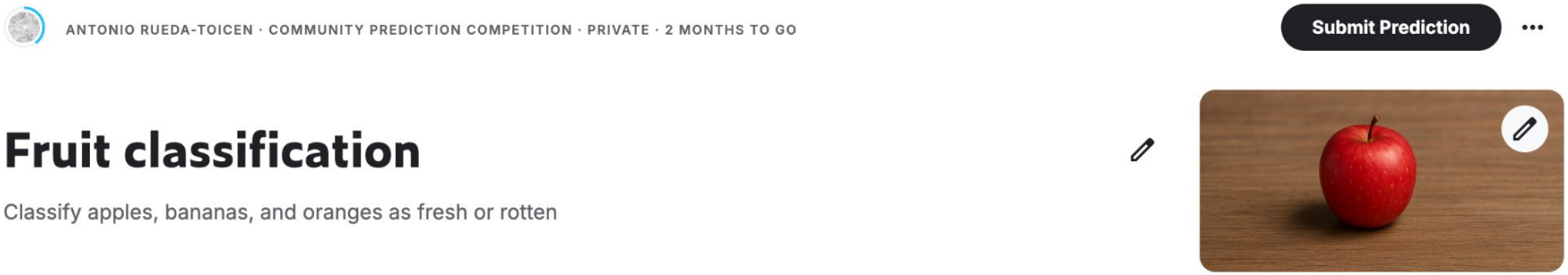


Figure 2: A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature-map sizes via convolution and pooling.

Building an image classifier



https://www.kaggle.com/t/b4dbb9add11c4da0962b837929799d52

Transformer: Block

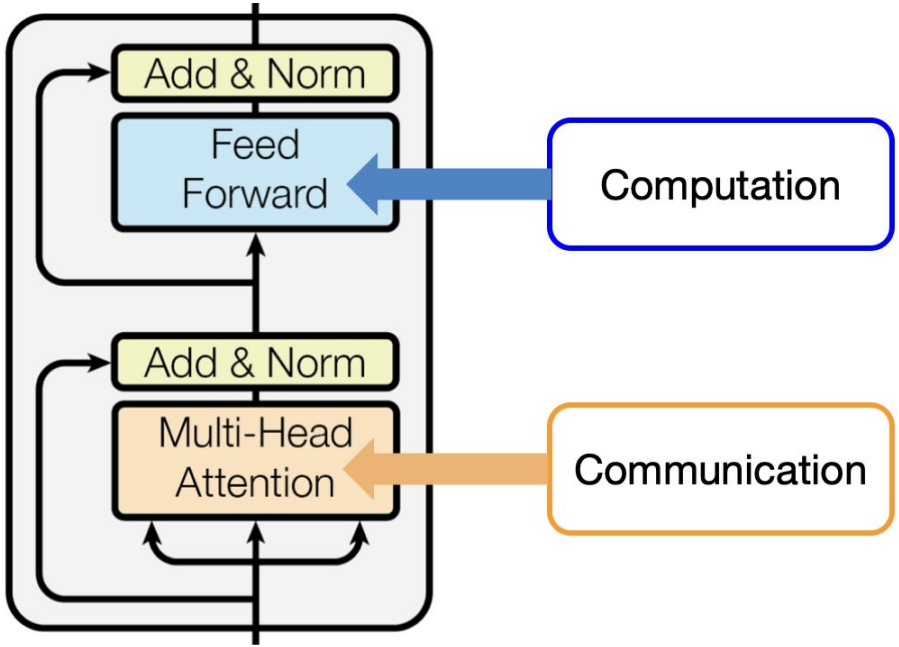


Image source

Transformer encoder: Self-attention

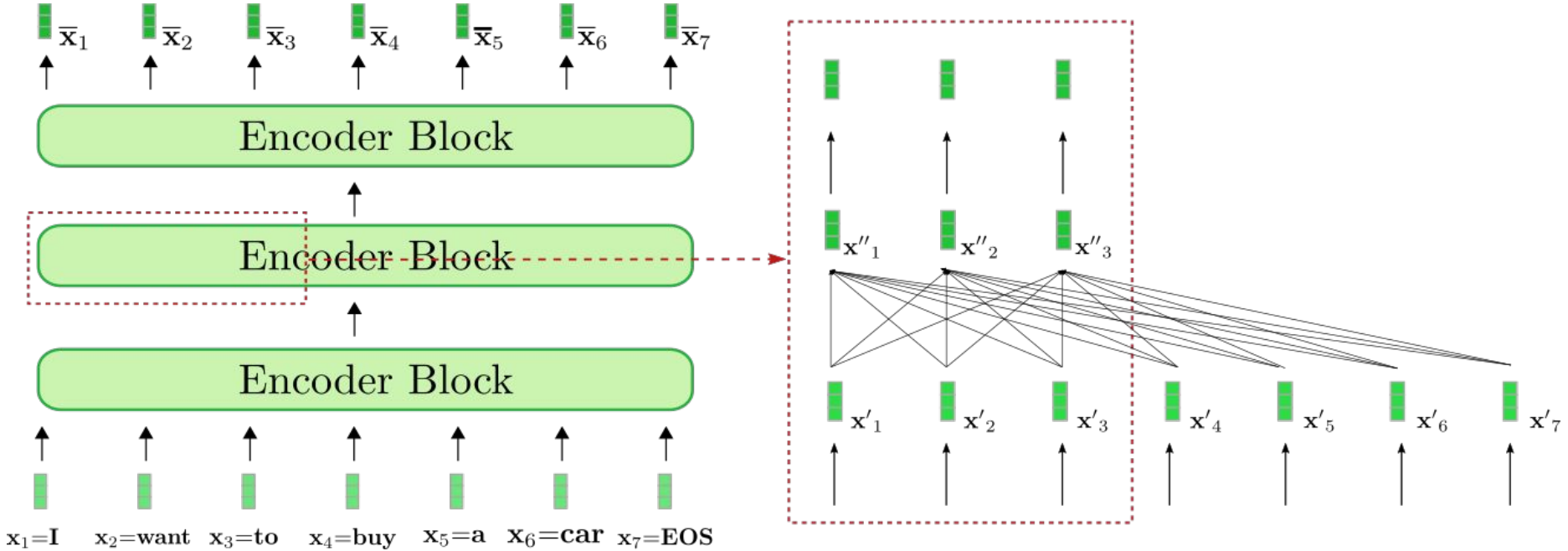
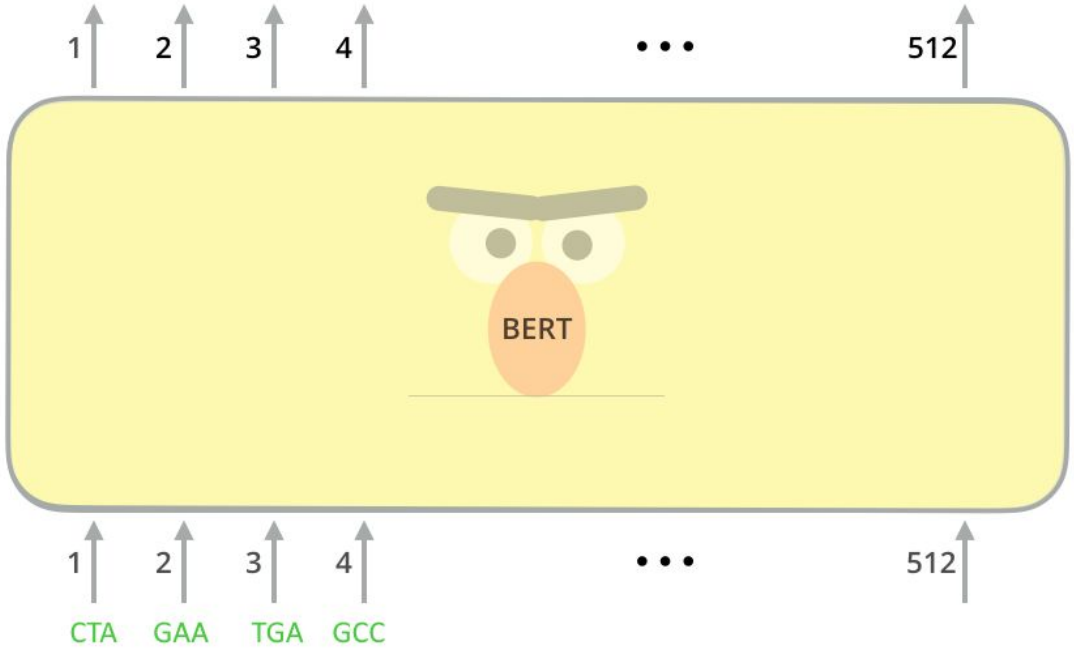


Image source

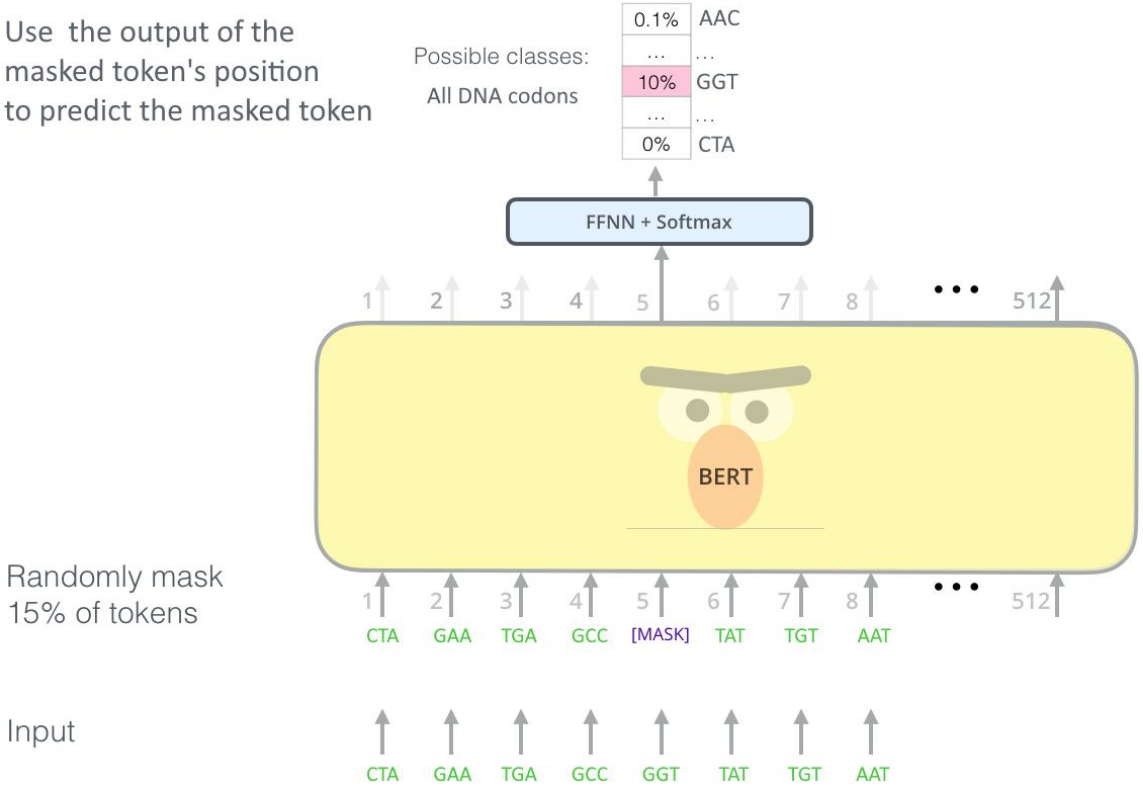
Transformer encoder: BERT for modeling text

[BERT notebook]



https://jalammar.github.io/illustrated-bert/

Transformer encoder: Self-supervised learning

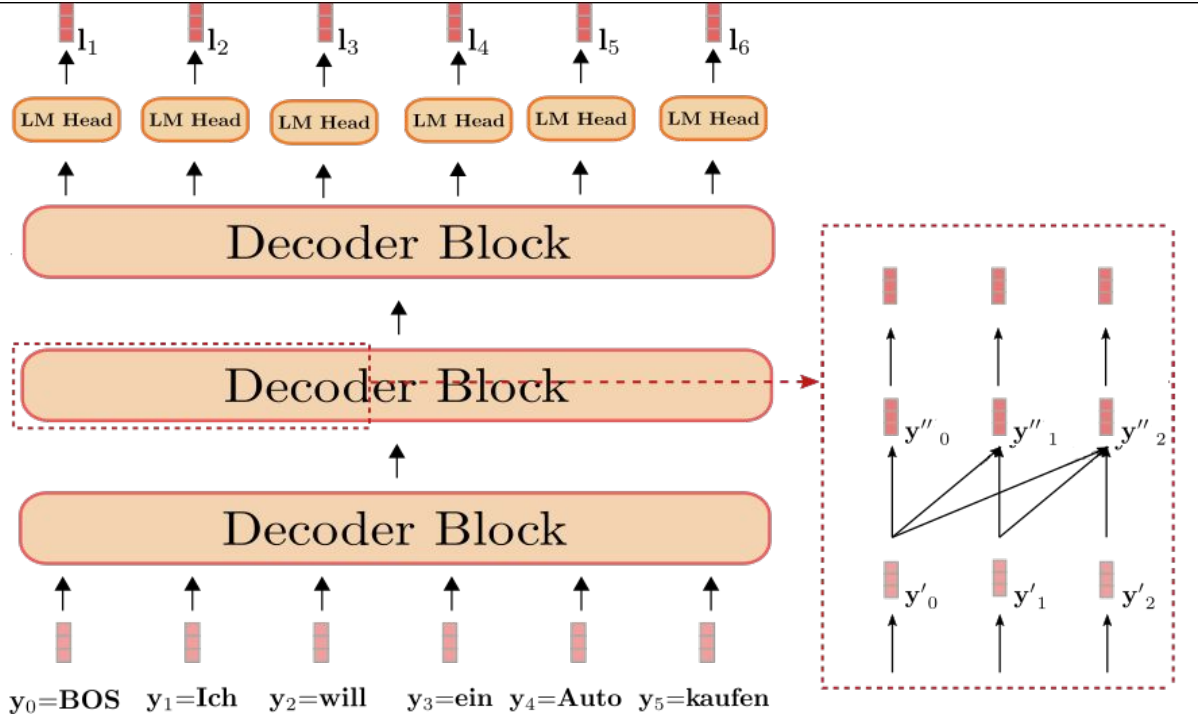


Self-supervised learning
We are able to effectively learn from unlabeled data by learning a *denoising* objective. This is useful when we have large amounts of unlabeled data.

This is a common approach for *pre-training*, where the goal is to obtain "good" weights without relying on supervised data. This can be used as a starting point for *downstream* tasks (e.g. classification tasks on DNA).

https://jalammar.github.io/illustrated-bert/

Transformer decoder: Masked self-attention



Causal Attention Mask
Every token can only "see" previous tokens and "extract" information from them.

The *first* token "BOS" contains no information about the sequence.

The *last* token "kaufen" can theoretically combine the information of the entire sequence into its hidden representation (i.e. embedding) without an additional explicit pooling operation.

Image source

Transformer - relevance and weaknesses

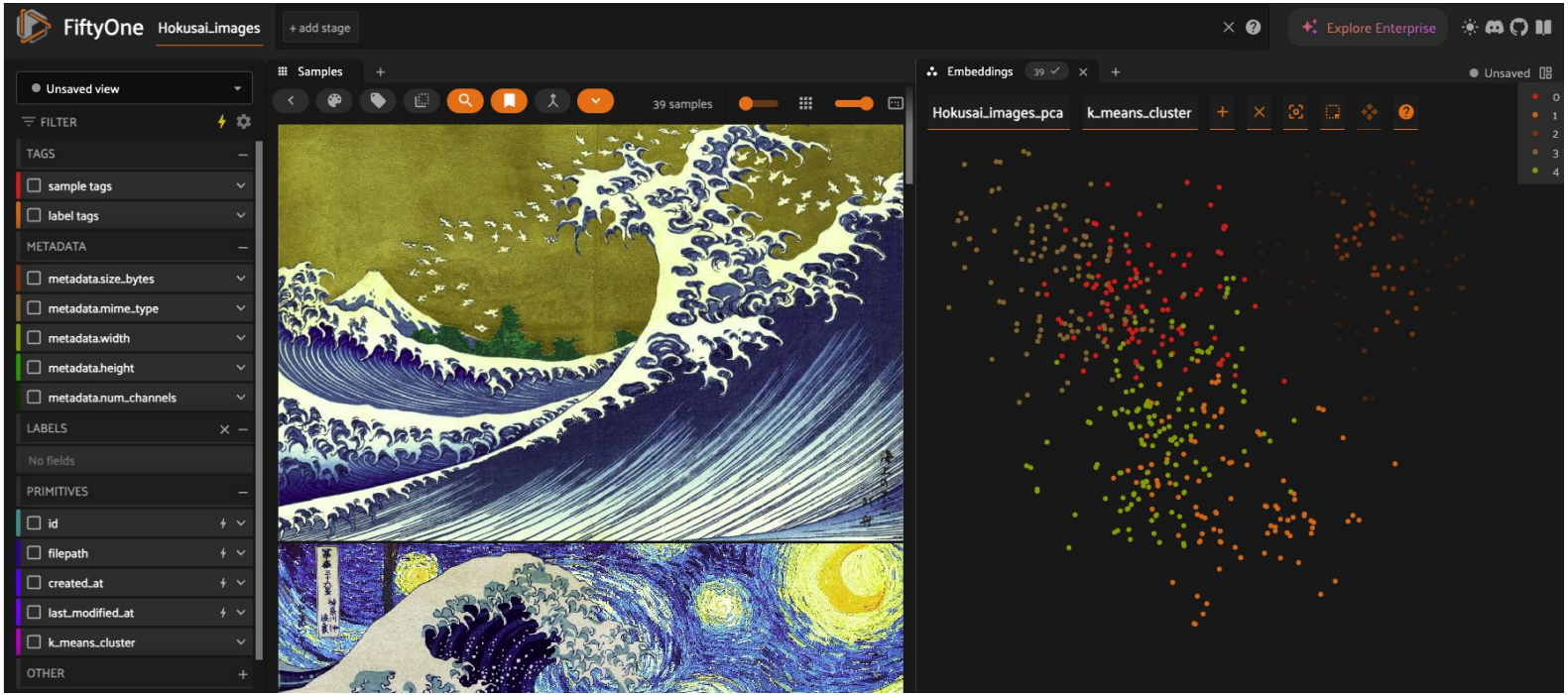
Relevance

- Solves long-term dependency issue present in RNNs.
- Can parallelize training more easily than RNNs (every input token can be processed in parallel).
- Transformer encoders are often used for embedding models (e.g. CLIP or SBERT).
- Transformer decoders are used for LLMs like ChatGPT.
- Vision transformers are currently SOTA for ImageNet classification.

Weaknesses

- The attention mechanism has quadratic runtime complexity with respect to the sequence length. This limits both training and inference speed.
- Requires large amounts of data for effective training.

Preview: dataset curation with FiftyOne



[Visualize and Cluster Embeddings with FiftyOne]

Review questions

- What is a loss function? Is the f1 measure a loss function or a performance metric?
- Can we use accuracy as a loss function? Can we use precision or recall as loss functions?
- Is the accuracy guaranteed to increase as the value of the loss function goes down?
- Which parts of the model need to be differentiable for an architecture to be trainable?
- What are we trying to minimize when using gradient descent? What is the learning rate?
- What is a GPU? Why are GPUs useful for deep learning?

Review questions

- What is an epoch? What is a batch? What is the maximum size of a batch? How does the batch size influence training?
- What is overparametrization?
- What is the difference between stochastic gradient descent (SGD) and ‘vanilla’ AKA ‘regular’ gradient descent?
- What is momentum? What is the difference between ADAM and SGD?
- Suppose that we have a ground truth for a ‘cat’ label that is encoded as [0, 1]. We can assume that the ground truth for a ‘dog’ label is [1, 0] (**cat** is the training point, **y = cat**).
    - The network first outputs [0.75, 0.25] as prediction (after softmax)
    - In the next epoch it outputs [0.99, 0.01] as prediction (after softmax)
    - Has the cross entropy loss increased or decreased?

Review questions

- What does it mean for a dataset to be ‘linearly separable’?
- How many layers does a neural network need in order to learn non-linear correlations?
- What’s the difference between a performance metric and a loss function?
- Why should we shuffle the data in every training epoch?
- Explain the bias vs variance tradeoff.
  - Effect of increasing the number of training epochs
  - Increasing the number of layers
  - Increasing the amount of dropped out connections
- How would you define ‘regularization’?
- What is the purpose of dropout? How does it work? Should dropout be active during inference time?

Review questions

- What is the vanishing gradient problem?
- How does a Resnet avoid the vanishing gradient problem?
- What does it mean to ‘freeze’ a layer?
- How does batch normalization work?
- Why should we call model.eval() before evaluating data from the validation set?
- What does it mean to “transfer a tensor” to the GPU?
- What is CUDA?
- How does the amount of RAM available in the GPU affect the types of models and data that we can use to train?
- What should we try when torch reports ‘CUDA out of memory’ errors?

Review questions

- Why do we need to call `optimizer.zero_grad()` in the training loop?
- What's the difference between a parameter and a hyperparameter? What's another common name for 'parameter' in a neural network?
- How can we know what's the "optimal" number of units in a hidden layer?
- How can we know the "optimal" number of layers in a network?
- Is data augmentation a regularization technique? Why should we apply data augmentations probabilistically?
- Why should we shuffle the data inside a training batch?
- If a model is increasing its loss through several epochs should we increase or decrease the learning rate?
- How do we evaluate training and validation loss to do early stopping?

Review questions

- What is weight decay? What is learning rate decay?
- Suppose that after going through certain number of epochs the training loss is higher than the validation loss. Is this an example of underfitting or overfitting?
- What is the vanishing gradient problem? Why does it appear?
- Are vanishing gradients more likely to appear when representing Jacobians and weight vectors with 8 bits, 16 bits, or 32 bits of precision?
- Why are ReLU activation functions less likely to display vanishing gradients than sigmoid activation functions?
- What is a skip connection?
- What is the difference between a Resnet and a Densenet?

Review questions

- Why do we resize images before training?
- If we retrain or do inference on a network pre-trained on Imagenet, why do we need to normalize its image channels using the mean and std of the intensities of the original dataset?
- What is a receptive field?
- What is a convolution?
- Why do we apply padding to images before doing convolutions?
- When calling conv2d(...,..., 64) how many output channels are we producing? How are they different from each other?
- Why is the vanishing gradient a ‘numerical’ problem? What does that mean?
- Are we more likely to get vanishing gradients when using fp16 or fp32 data types for our weights?

Resources

- Github Repository
- YouTube playlist
-